Venture Capital and Engineering Quality: Evidence from Software Startups*

João Dimas† Labros Pnevmatikos‡

November 10, 2026

Abstract

We document a new dimension of venture capital (VC) professionalization: the way startups build and maintain software. We use monthly snapshots of public source code from 569 firms, covering 13.2 billion lines to estimate startups' technical debt—the dollar cost of correcting problems that make code harder to maintain. Following the first institutional VC entry, funded firms' critical code issues per line decline by 15% and technical debt per line by 12% relative to their pre-funding averages, compared with otherwise similar firms without VC backing. Moreover, engineers already active before VC financing become less likely to introduce bugs, and later funding rounds bring substantial hiring without detectable code quality gains. Descriptions accompanying 5.6 million code contributions reveal more systematic documentation and organization of development work after a VC's arrival. Together, the findings suggest that VC professionalization extends beyond governance and leadership to the engineering work of startups' existing teams.

JEL Classification: G24, G34, D82, L86, L26
Keywords: Venture capital, corporate governance, intangible assets, technical debt, software quality

*We are grateful to Todd Gormley and Margarita Tsoutsoura for their invaluable comments and guidance throughout this project. We also thank Julie Fu, Armando Gomes, Gerard Hoberg, Arthur Korteweg, Linda Schilling, Anjan Thakor, and participants in the 12th Annual USC Marshall Ph.D. Conference in Finance for their insightful feedback and suggestions. This paper was previously circulated with the title: Venture Capital, Engineering Quality, and Technical Debt. We acknowledge support from the Google Research Program, and Pnevmatikos thanks the Onassis Foundation for support. The authors have no other relationships or financial interests to disclose.

†Olin Business School at Washington University in St. Louis. Send correspondence to João Dimas; e-mail: joaodimas@wustl.edu.
‡Olin Business School at Washington University in St. Louis.

1 Introduction

An important question in corporate finance concerns how venture capitalists (VCs) contribute to the success of the startups they finance (Gompers et al., 2020; Amornsiripanitch, Gompers, and Xuan, 2019; Hsu, 2004). While VCs select promising firms, their monitoring and advice after investment can also shape performance (Bernstein, Giroud, and Townsend, 2016). Beyond monitoring, VCs help professionalize startups' leadership, compensation, and human resource policies (Hellmann and Puri, 2002). These findings raise the question of how far professionalization extends within the firm. In software startups, does it also change how engineers build and maintain the product?

Software startups often face a trade-off between bringing products to market quickly and keeping future maintenance costs low. Shortcuts that speed development can create technical debt: the additional work required to maintain or extend the software later. It can shape a startup's ability to scale reliably, innovate, and manage operational risk (Sculley et al., 2014, 2015). VC funding and oversight may help firms reduce technical debt, but pressure to grow quickly may encourage them to prioritize new features over maintenance.

The economic burden of technical debt is difficult to assess from financial statements alone (Banker, Liang, and Ramasubbu, 2021; Krasner, 2022; McKinsey & Company, 2020, 2023). Measuring technical debt consistently across firms is also challenging: source code is usually proprietary, and estimates depend on which code issues a method identifies and how much effort it assigns to fixing them (Krasner, 2022). Our setting allows us to apply a common measurement framework to firms' source code and construct a dollar-denominated estimate of the cost of correcting identified problems that make code harder to modify or extend.

We analyze 13.2 billion lines of code across 60,495 monthly codebase snapshots from 569 open-core technology firms between 2010 and 2024. These firms commercialize software with publicly available core source code, allowing us to trace the development of an economically relevant part of their products. We evaluate each snapshot with SonarQube, a tool that automatically identifies maintainability issues, software errors, and security vulnerabilities. We link the snapshots to financing records from PitchBook and employment histories from LinkedIn. First institutional VC entry is a firm's first completed Seed round or, if PitchBook records no Seed round, its first completed Series A round.

We examine changes in code quality during the year after first institutional VC entry. We estimate a stacked difference-in-differences model using firms without VC backing as controls. We count issues per line of code and express estimated changes as percentages of funded firms' pre-funding averages. Relative to controls, serious issues that SonarQube rates critical decline by 15%. The overall frequency of maintainability issues declines by 16%.

As with financial debt, technical debt has a principal amount: the estimated cost of correcting maintainability issues (Avgeriou et al., 2021). SonarQube estimates the hours required for these

corrections. We apply a common hourly developer wage and calibrate the resulting costs to an external economic benchmark. Relative to controls, technical debt declines by $1.57 per line of code, 12% of funded firms' pre-funding average.

Event-study estimates show that critical issues per line of code start declining soon after first VC entry, with no evidence of diverging trends beforehand. This improvement coincides with substantial codebase expansion. One possibility is that issue density falls mainly because firms add relatively clean new code, diluting problems in the existing codebase. We therefore repeat the analysis among firms selected for low prior codebase growth. The funded firms in this group expand their codebases at a rate similar to controls around VC investment, yet their code quality still improves relative to controls.

Before VC entry, funded firms have similar code quality to controls but are already growing faster. Unobserved differences in managerial capabilities could explain both their earlier growth and later quality gains, even without VC backing. To address this concern, we match firms on pre-funding growth in employment and codebase size, varying the matching methods and thresholds. As an additional check, we repeat the analysis with controls matched on prior changes in code quality. The declines in issues per line of code persist across these comparisons.

Starting to serve paying customers may prompt firms to improve their engineering practices, offering another explanation for the quality gains around VC entry. Yet two-thirds of funded firms already generate revenue at entry, and quality improves among these firms as well. We find no evidence of larger gains among firms not yet generating revenue. Development activity expands, but developers' descriptions of code changes show no detectable shift toward billing, payments, or subscriptions. These findings weigh against initial commercialization as the explanation for the quality gains.

We next examine whether investor oversight contributes to the quality gains. Staged financing, board rights, and advice give VCs opportunities to review progress and influence how firms operate (Gompers, 1995; Kaplan and Strömberg, 2003; Casamatta, 2003). When more investors share the benefits of monitoring, each may have less incentive to bear its costs. Consistent with this prediction, critical issues per line of code decline by an estimated 1% among firms whose first round includes more than four investors, compared with 25% among the remaining firms.

Capital alone may also improve code quality by financing more engineering resources. Under this explanation, we would expect larger improvements following larger financing rounds. We find no evidence of this pattern. Later rounds provide a second comparison because they bring fresh capital to firms that already have institutional investors. Following firms' first Series B-or-later round, total employment and the number of software engineers rise substantially, but we find no evidence of improved code quality or lower technical debt per line. The significantly larger gains at first entry are consistent with a role for monitoring and professionalization accompanying the arrival of the first institutional investor.

We then examine engineering processes using messages accompanying 5.6 million individual code contributions (commits). Following first VC entry, commits more often credit a coauthor and more often carry copyright and licensing assurances. A larger share of commit messages references formal records of tasks, reported bugs, or proposed changes, and a smaller share of commits falls on weekends. These patterns suggest more systematic documentation and organization of development work.

Recruiting more experienced engineers offers another explanation for the quality gains. If recruitment drives these gains, improvements should be larger among firms that hire technical leaders or whose new hires include a greater proportion of senior engineers during the year after financing. The hiring comparisons do not reveal this pattern. We next examine changes in the work of engineers active before financing. Using documented bug fixes, we trace repaired lines back to the earlier commits that last modified them. Among these incumbent engineers, the share of commits linked to subsequent fixes declines after VC entry. Taken together, our findings suggest that first institutional VC entry improves code quality and reduces technical debt per line, with the evidence pointing to professionalization of existing engineering teams.

Our results contribute to two strands of the literature. First, we add to the venture capital literature on post-investment value creation. While prior work has emphasized governance and leadership professionalization (Lerner, 1995; Hellmann and Puri, 2002; Ewens and Marx, 2018), we provide evidence on the technical practices through which startups build their core products. Our findings also complement evidence that productivity grows faster at VC-backed manufacturing firms after financing (Chemmanur, Krishnan, and Nandy, 2011) and that operating practices change following private equity buyouts (Bernstein and Sheen, 2016). We extend this inquiry to engineering practices at early-stage software firms.

Second, our measurement of software maintenance liabilities informs research on intangible assets and software-intensive firms (Eisfeldt and Papanikolaou, 2013; Peters and Taylor, 2017). We construct a dollar-denominated measure of technical debt that is comparable across firms. To our knowledge, this is the first such measure in the finance literature. Combining this measure with development histories allows researchers to study how firms build and maintain software assets, with applications to technology-firm valuation and due diligence in mergers and acquisitions.

The remainder of the paper proceeds as follows. Section 2 describes the data, sample construction, and measures of engineering quality and technical debt. Section 3 presents the empirical design, results, and mechanism evidence. Section 4 examines matching, later rounds, commercialization, and specification sensitivity. Section 5 concludes.

2 Data and Background

We combine three data sources to follow software development, employment, and financing at open-core technology firms: **(i)** our comprehensive evaluations of source code obtained from GitHub and GitLab; **(ii)** employee–firm matched data from LinkedIn; and **(iii)** funding and deal records from PitchBook.

Firms typically restrict access to proprietary source code, making their software assets difficult to study. We therefore focus on open-core firms: they make the core of their product publicly available, allowing us to observe its development history and technical choices.

2.1 The Open-Core Business Model

The open-core model combines the collaborative advantages of open-source software with the revenue potential of proprietary licensing. Under this model, a firm releases its core product under an open-source license, making the source code freely accessible to developers and researchers. The firm retains exclusive rights to enterprise-grade features, proprietary modules, or support services, which it monetizes through subscription or licensing agreements.

A prominent example is Posit (formerly RStudio, Inc.), the company behind RStudio, a programming environment for R. While the core software is free and open source, the company generates revenue by licensing enhanced versions for commercial applications. More broadly, research on open-source software documents knowledge spillovers among projects and productivity gains for the firms that use it (Fershtman and Gandal, 2011; Nagle, 2019).

Unlike closed-source startups, open-core firms develop much of their codebase in public view. Many also keep public issue trackers, where users report bugs and request changes. This openness is what makes engineering quality observable for our study.

2.2 Codebase Data

We define our sample universe using OSSRank, a leading analytics platform for the open-source ecosystem, to identify 649 unique companies that commercialize open-source repositories. These firms primarily operate in the technology sector.

We match each firm to the repositories on GitHub and GitLab that hold the code underlying its core product. A repository is the central storage location for a software project's source code. GitHub and GitLab are industry-standard platforms for software development and version control, widely adopted across both startups and large enterprises. These platforms record changes to the code as timestamped "commits" that identify the author and carry a message describing the change. While most repositories are hosted on GitHub, some firms use GitLab. Since both platforms offer nearly identical version control features, we pool these data sources into a combined dataset.

We construct a monthly panel of codebases by retrieving the final version of each repository for every month in our sample period (2010–2024). We evaluate codebases with SonarQube using a consistent measurement procedure across firms and over time.

Because a firm can maintain several repositories, we measure engineering quality at the firm–month level by aggregating across its available repositories. We sum lines of code[1], estimated repair time, and issue counts, and recompute ratio measures from these totals. This construction captures development across the firm's covered projects while retaining the firm as the unit of analysis. Section 4.4 shows that the main estimates change little when we exclude firms with more than five scanned repositories.

Of the 649 companies, we link 609 to at least one repository, 703 repositories in all, and 569 of these firms contribute at least one covered codebase month. We analyze 60,495 repository–month snapshots containing 13.2 billion lines of code. We fill gaps between observed firm–months by linearly interpolating the aggregated metrics within each firm. The monthly panel contains 61,768 firm–months with a positive line count. For estimation, we retain firm–months in which at least one repository is observed and the line count is positive. Appendix N reports the sample flow and comparisons using complete observation windows and fixed pre-financing repository sets.

2.3 Funding and Firm Data

We date VC entry using PitchBook financing records. For each firm, we extract its investment history, including deal dates, transaction types, investor composition, and funding stages. We first remove rounds recorded as rumored, failed or cancelled, announced or in progress, upcoming, or postponed. We then identify the first completed Seed round, or the first completed Series A round when no completed Seed is recorded. To focus on early-stage venture investment, we exclude firms that enter the data only through a public listing, debt issuance, management buyout, acquisition, or Series B or later round with no prior Seed or Series A financing. We separately examine later financing rounds, Series B and beyond ("Series B+"), in Section 4.2.

Since the firms in our sample are predominantly private and early stage, standard financial disclosures and coverage in databases such as Compustat are limited. To measure firm size and growth, we use employee–firm matched LinkedIn records to construct monthly observations of total headcount and software-engineering headcount. These histories track organizational expansion alongside changes in the codebase.

[1] Our line count excludes blank lines and lines containing only comments (notes written for people reading the code).

2.4 Measuring Engineering Quality

We measure engineering quality using SonarQube, a widely used analysis tool that scans source code and flags deviations from common coding and security practices. For each historical snapshot, we apply language-specific checks directly to its source code. Using the same measurement procedure across firms and months supports comparisons of firms using different programming languages. Appendix B details the configurations for selected languages.

2.4.1 Taxonomy of Coding Problems

Each SonarQube rule checks for a specific pattern in the code. When the scanner finds that pattern, it records an "issue", that is, a violation of the rule; these issues are the basic unit behind our engineering-quality measures.

SonarQube identifies three types of coding problems: bugs, security vulnerabilities, and maintainability violations. Bugs can cause failures or incorrect behavior; security vulnerabilities can expose software to attack. Maintainability violations make software harder to understand, modify, or extend. We measure the frequency of bugs and maintainability violations per 1,000 lines of code; for security, we measure the frequency of security hotspots, defined below.

SonarQube also classifies violations by severity. Our principal quality measure, Critical Violations, counts serious deviations from coding best practices that the tool classifies as critical. Most critical violations in our data are maintainability violations.[2] We also measure total violations across all severity levels.

2.5 Technical Debt

Technical Debt measures the stock of maintenance liabilities embedded in a codebase. In the software engineering literature, technical debt captures development choices that save time today but raise the cost of future changes (Sculley et al., 2014). Firms effectively pay interest through additional maintenance work and face costly paydowns when refactoring—reorganizing existing code without changing its intended functionality—becomes unavoidable. For example, duplicating code can accelerate a feature's introduction but requires developers to update several copies whenever that feature changes. Although this tradeoff is familiar in software engineering, it has received little direct measurement in finance; our approach expresses these software liabilities in dollars so they can be compared across firms and over time.

We measure technical debt using SonarQube's estimate of the time required to resolve main-

[2] SonarQube's traditional measures do not split critical violations by type. We therefore infer the issue-type split from its alternative classification framework, which reports high-severity issues by the quality they affect: reliability, security or maintainability. See Appendix M.2.

tainability violations. We sum these times across a firm's covered repositories each month, yielding estimated repair time in hours. Violations that require more work to resolve receive greater weight.

The estimated repair time depends on the coding rules used in the scans and the time assigned to resolve each violation. We therefore apply a common scaling factor, κ, to calibrate this measure to an external economic benchmark. This produces a dollar-denominated proxy for maintenance liabilities, with a consistent unit of measurement across firms and monthly codebase evaluations.

The calibration proceeds in three steps. First, we construct a proxy for the replacement cost of the observed software assets. We set development time at $\ell = 0.5$ hours (30 minutes) per line of code. For a codebase with $\text{LoC}_{i,t}$ lines of code, our proxy for replacement effort is:

$$\text{Redevelopment Effort}_{i,t} = \ell \times \text{LoC}_{i,t}. \tag{1}$$

Multiplying by an assumed common developer wage of $w = \$55$ per hour yields Software Assets in dollars:

$$\text{Software Assets}_{i,t} = w \times \text{Redevelopment Effort}_{i,t}. \tag{2}$$

Second, we choose the scaling factor κ to place estimated repair time on an economic scale. Surveyed technology executives estimate technical debt at 20–40% of the value of their technology assets before depreciation (McKinsey & Company, 2023). We use the lower endpoint as a calibration target, setting median debt equal to 20% of the median replacement-cost proxy. Let $H_{i,t}$ denote estimated repair time in hours for firm i in month t. Letting $\varphi = 0.20$, we choose κ such that:

$$\text{Median}\left(w \times \kappa \times H_{i,t}\right) = \varphi \times \text{Median}\left(w \times \ell \times \text{LoC}_{i,t}\right), \tag{3}$$

where we compute medians over nonmissing observations in the firm–month panel. Simplifying yields:

$$\kappa = \varphi \times \frac{\text{Median}\left(\ell \times \text{LoC}_{i,t}\right)}{\text{Median}\left(H_{i,t}\right)}. \tag{4}$$

In the main sample, this produces $\kappa = 68.60$, which is constant across firms and time.

Third, we apply this calibration to construct Technical Debt for each firm–month:

$$\text{Technical Debt}_{i,t} = w \times \kappa \times H_{i,t}. \tag{5}$$

Because κ is constant across firms and time, it rescales estimated repair time proportionally, leaving statistical significance and changes relative to the sample mean unchanged. The calibration makes the estimated maintenance burden comparable in dollar terms, with its level determined by the common

wage and calibration target.

We also report the Technical Debt Ratio, which expresses calibrated debt as a percentage of the replacement-cost proxy:

$$\text{Technical Debt Ratio}_{i,t} = 100 \times \frac{\text{Technical Debt}_{i,t}}{\text{Software Assets}_{i,t}} = 100 \times \frac{\kappa \times H_{i,t}}{\ell \times \text{LoC}_{i,t}}. \tag{6}$$

Both measures use the same observations with positive lines of code. The wage cancels from the replacement-cost ratio, while its level reflects the calibration target.[3] Changes in the ratio, measured in percentage points, indicate how the maintenance burden evolves relative to the size of the software asset.

2.6 Key Variables

We express all issue counts per 1,000 lines of code (kLoC). SonarQube reports each issue under two classification frameworks: its traditional classification assigns each rule one type (bug, security vulnerability, or maintainability violation) and one severity level, while its alternative classification records a rule's impact on reliability, security, and maintainability, each with its own severity. Our primary measures use the traditional classification. The high- and low-severity maintainability and reliability measures use the alternative classification, described in Appendix M.2. *Reliability Issues* and *Bugs* therefore assess related risks of incorrect software behavior using different classifications. The severity-specific measures are not a breakdown of the traditional totals.

Security Hotspots identify security-sensitive code segments that require human review, such as code handling authentication or sensitive data. A hotspot marks a location that warrants inspection; it becomes evidence of a vulnerability only after the security context has been assessed. This distinction separates our measure of security-review needs from confirmed security failures.

In addition to these engineering measures, we use the economic measures of Technical Debt described in Section 2.5: debt in dollars per line of code and the Technical Debt Ratio, which scales debt by software assets. We measure codebase size using lines of code and organizational scale using total and software-engineering headcount. Regressions use code volume as log(LoC), total headcount as log(Headcount), and software-engineering headcount as log(SWE).

The VC indicator, $VC_{i,c,t}$, equals one for a firm treated in cohort c from its qualifying financing month onward. To examine heterogeneity, *High Violations* identifies firms whose critical violation density in the month before the cohort's event exceeds the median across baseline firm–cohort observations of firms treated in any cohort; *Large Round* identifies financing rounds above the treated-firm median deal size. Table 17 in Appendix A provides the full variable definitions.

[3] The calibration equates median Technical Debt with 20% of median Software Assets. The median firm–month ratio is 30.2%, since the median of a ratio need not equal the ratio of medians.

2.7 Summary Statistics

Table 1 reports summary statistics for the firm–month panel before stacking. The median covered firm–month has 5.16 critical violations per 1,000 lines of code and a Technical Debt Ratio of 30.2%. The distributions are right-skewed: the corresponding means are 10.45 and 43.4%.

Table 1: Summary Statistics

		Distribution				
Variable	**N**	**Mean**	**SD**	**P25**	**Median**	**P75**
Panel A: Level Variables						
Technical Debt (in $)	61,768	5,263,622.30	11,987,074.44	164,190.37	938,639.44	4,201,349.26
Comment Lines	61,768	117,976.35	329,153.82	4,707.75	21,377.50	98,217.00
Total Violations	61,768	11,732.45	26,172.37	399.00	2,131.00	10,016.25
Critical Violations	61,768	4,133.78	9,799.54	83.00	619.00	3,210.00
Bugs	61,768	593.95	1,920.59	5.00	49.00	279.00
Security Hotspots	61,768	275.21	632.14	9.00	56.00	223.00
Maintainability Violations (All)	61,768	10,967.73	24,626.64	363.00	2,042.00	9,368.92
Maintainability Issues (High)	61,768	4,106.40	9,747.16	79.00	608.00	3,196.00
Maintainability Issues (Low)	61,768	3,445.93	8,202.71	63.00	472.00	2,522.00
Reliability Issues (Total)	61,768	1,135.75	4,286.75	13.00	114.00	579.00
Reliability Issues (High)	61,768	28.33	75.54	0.00	3.00	20.67
Reliability Issues (Low)	61,768	463.55	3,109.39	0.00	11.00	134.00
Panel B: Per-LoC Variables						
Technical Debt per LoC ($)	61,768	11.92	15.99	3.55	8.29	14.26
Technical Debt Ratio (%)	61,768	43.36	58.15	12.90	30.16	51.84
Violations per 1,000 LoC	61,768	29.44	40.29	8.14	16.84	34.90
Critical Violations per 1,000 LoC	61,768	10.45	18.74	1.44	5.16	9.75
Bugs per 1,000 LoC	61,768	1.77	5.47	0.04	0.45	1.39
Security Hotspots per 1,000 LoC	61,768	1.37	10.60	0.10	0.43	1.05
Maintainability Violations per 1,000 LoC	61,768	27.38	37.68	7.72	16.04	32.81
Maintainability Issues (High) per 1,000 LoC	61,768	10.35	18.68	1.37	5.09	9.67
Maintainability Issues (Low) per 1,000 LoC	61,768	7.80	15.76	1.24	3.86	7.92
Reliability Issues (Total) per 1,000 LoC	61,768	3.28	9.91	0.15	0.83	2.53
Reliability Issues (High) per 1,000 LoC	61,768	0.10	0.40	0.00	0.02	0.09
Reliability Issues (Low) per 1,000 LoC	61,768	1.36	5.73	0.00	0.09	0.51
Panel C: Scale Variables (Log)						
Log(Lines of Code)	61,768	11.78	2.07	10.66	12.05	13.22
Log(Headcount)	32,313	3.58	2.59	1.79	3.14	4.88
Panel D: Treatment Indicators (Post-Funding)						
Post Seed Round (=1)	111,780	0.16	0.37	–	–	–
Post Series A (=1)	111,780	0.10	0.30	–	–	–

Note: The table reports summary statistics for the firm–month panel before stacking. SonarQube counts and estimated repair time are summed across each firm's covered repositories, with gaps interpolated between observed months. Panels A–C use firm–months with a positive number of lines of code: the code-quality and both debt measures cover 61,768 firm–months, while headcount covers 32,313 firm–months. SD denotes standard deviation; P25 and P75 denote the 25th and 75th percentiles. Panel A reports levels. Panel B divides issue counts by thousands of lines of code and Technical Debt by lines of code; the Technical Debt Ratio is calibrated debt divided by the replacement-cost proxy, multiplied by 100. Both debt measures use $\kappa = 68.60$ (Section 2.5). Panel C reports the natural-log scale transformations defined in Table 17. Panel D reports indicators equal to one from the relevant financing month onward, for every firm–month in the panel. Maintainability Violations use the traditional classification; the high- and low-severity maintainability issues use the alternative classification described in Appendix M.2 and are not a breakdown of the total. Bugs also use the traditional classification; the reliability issues use the alternative classification and are not a breakdown of Bugs.

3 Estimation and Empirical Results

To estimate how first institutional VC entry changes engineering quality, we exploit differences in financing dates across firms using a stacked difference-in-differences (DiD) design. With staggered treatment, a conventional two-way fixed-effects regression can compare newly funded firms with firms whose outcomes already reflect earlier financing. Under heterogeneous treatment effects, these comparisons can contaminate the estimated response and even reverse its sign (Callaway and Sant'Anna, 2021; Sun and Abraham, 2021). Following Gormley and Matsa (2011), we construct a separate comparison group for each financing cohort using firms that remain without a first round throughout its event window.

We group firms by the calendar month of their first completed Seed round, or their first completed Series A round when no Seed is recorded. For each cohort, controls are firms with no qualifying first round through the end of the window. The cohort panel covers 25 months, from event month $e = -12$ through $e = +12$ relative to financing. We then stack these panels.[4] Every estimation sample excludes fixed-effect singletons (observations that are the only member of a fixed-effect group) and cohorts without both treated and control firms. The stacked cohort panels contain 99 cohorts, 274 treated firms, 412 firms appearing as controls, and 484 unique firms. The baseline regression uses 487,577 observations from 483 unique firms, including 273 treated firms, after excluding 18 fixed-effect singletons. Appendix C describes the estimand and its relation to the stacked DiD regression.

We estimate

$$M_{i,c,t} = \beta VC_{i,c,t} + \chi_{c,i} + \delta_{c,t} + \gamma X_{i,c,t} + \varepsilon_{i,c,t}, \tag{7}$$

where i, c, and t index firm, cohort, and calendar month. $M_{i,c,t}$ is an engineering outcome, and $VC_{i,c,t}$ is $\text{Treated}_{i,c} \times \text{Post}_{c,t}$, which equals one for treated firms from the financing month onward. Cohort × Firm fixed effects, $\chi_{c,i}$, absorb persistent firm differences within each cohort; Month × Cohort fixed effects, $\delta_{c,t}$, absorb common monthly shocks within that cohort. $X_{i,c,t}$ denotes additional regressors in the heterogeneity specifications, and $\varepsilon_{i,c,t}$ is the error term. The baseline omits $X_{i,c,t}$.

The coefficient β captures the post-financing change in treated firms relative to their cohort controls. The DiD coefficient weights these comparisons according to the observations and residual treatment variation in the stacked sample; Appendix C provides the formal interpretation. We cluster standard errors by cohort and firm to account for dependence within cohorts and across repeated appearances of the same firm.

We also examine two hypotheses about the scope of the improvement. Firms with more critical violations before financing may have greater room to improve, while firms raising larger rounds can devote more resources to engineers, tools, and remediation. The heterogeneity columns therefore

[4] A control firm–month can appear in several cohort panels, and a not-yet-treated firm can later enter as treated in its own cohort. The stacked observation count therefore exceeds the number of underlying firm–months.

interact treatment with *High Violations*, defined using critical-violation density at relative month -1, and *Large Round*, defined using the treated-firm median round size. These comparisons distinguish where the gains are concentrated and motivate the more direct mechanism tests below. Appendix F extends the analysis to founding-team characteristics using LinkedIn career histories, and Table 17 provides the complete variable definitions.

To examine the timing of the response, we interact treated status with event-quarter indicators while retaining monthly fixed effects. We index the event-quarter bins from $q = -3$ to $q = 5$, omitting $q = 0$, and plot them at their month-bin midpoints. Each bin contains three consecutive months, except the end bins $q = -3$ and $q = 5$, which contain months $\{-12, -11\}$ and $\{11, 12\}$. The omitted quarter contains relative months $\{-4, -3, -2\}$, and the quarter around financing, $q = 1$, contains $\{-1, 0, 1\}$. The latter bin includes one pre-financing month, so a change that begins only after the round closes is averaged with an untreated month. Quarters $q = 2$ through $q = 5$ contain only post-financing observations. These estimates compare pre-financing trajectories and trace how the improvement develops during the first year of institutional backing.

3.1 Identification Strategy

Venture capitalists select firms whose teams, products, and growth prospects they find promising. A central identification concern is therefore that VCs select firms whose engineering quality is already improving, or whose founders have planned improvements that have yet to appear in the code. Our identification does not require financing to be randomly assigned. It requires that, absent first VC entry, treated firms and their cohort controls would have followed parallel trends in engineering quality, together with no anticipatory response and no spillovers to controls.

Several features of our data and design address selection on an improving engineering trajectory. First, the event studies show little divergence in critical violations per 1,000 lines and technical debt per line before financing, followed by reductions around the first round (Figures 1 and 6). To explain this pattern, an omitted change in product demand, management, or engineering priorities would need to coincide with financing and produce the observed change in quality. A gradual improvement already under way before investor entry would instead predict a divergence in the pre-treatment coefficients.

Second, the timing of engineering quality differs from that of firm growth. Treated firms expand their headcount and codebases more rapidly than controls before financing, but their critical-violation density does not show a corresponding improvement. This distinction matters because VC selection on fast-growing firms need not imply selection on improving code quality. We develop this argument in the firm-scale results and test it directly by matching on pre-financing headcount growth in Section 4.1.

Third, we examine specific alternatives to a change in engineering practices. Commercial-stage tests ask whether the quality improvement is concentrated in firms making the initial transition to revenue generation. A Series B+ placebo asks whether the same quality gains recur when firms that

already have institutional backing receive additional financing. Section 4 presents these tests and their implications for the interpretation of first VC entry.

Baseline Balance

Table 2 reports covariate means at the last pre-treatment month ($t = -1$). Treated and control firms enter the event with similar engineering quality: the standardized difference is 0.05 for critical violations per 1,000 lines of code, −0.03 for total violations, and −0.01 for both technical debt per line and the Technical Debt Ratio. The absence of a substantial baseline quality gap is informative for two competing selection stories. Treated firms do not start with markedly better code. Nor do they have systematically more violations to remove, as a simple room-for-improvement explanation would predict.

This balance also has an institutional rationale. Survey evidence indicates that VCs place particular weight on the management team and assess the product, technology, and market when selecting investments (Gompers et al., 2020). These assessments concern the commercial promise and capabilities of the venture; the detailed rule violations we measure capture a different margin, namely the implementation of coding practices within the existing product. An investor can therefore select a promising technology firm without selecting a codebase with low violation density. The observed balance is consistent with this distinction between commercial due diligence and the specific engineering outcomes in our analysis.

We next test whether baseline engineering quality or codebase size predicts which firms within a cohort receive financing. Using each firm's measures in the month before financing ($t = -1$), as in Table 2, we regress an indicator equal to one for the cohort's newly financed firms and zero for its controls on three quality measures (critical and total violations per 1,000 lines and technical debt per line), and then on those measures plus log lines of code. Both linear probability models include cohort fixed effects and cluster standard errors by firm. The quality measures are not jointly statistically significant ($p = 0.24$) and explain 0.02% of the within-cohort variation in financing. Doubling a firm's lines of code is associated with a 0.1 percentage-point lower probability of financing ($p < 0.01$), but adding size raises the explained share only to 0.1%. Selection on these observables therefore concerns scale, while baseline engineering quality has little predictive power for financing.

The balance table shows these scale differences directly. Treated firms have lower total headcount (standardized difference −0.65), lower software-engineering headcount (−0.69), and smaller codebases (−0.31 in log lines of code). Cohort × Firm fixed effects absorb persistent differences in levels, and the matched specification in Section 4.1 addresses the different pre-financing growth trajectories. These comparisons support the design's focus on changes in quality around financing.

Table 2: Baseline Balance: Treated vs. Control Firms at $t = -1$

	Treated	**Control**	**Std. Diff.**
Log(Lines of Code)	11.07	11.70	−0.31
Log(Headcount)	2.05	3.58	−0.65
Log(SWE Headcount)	1.07	2.58	−0.69
Critical Violations / kLoC	12.37	11.16	0.05
Technical Debt per LoC	12.45	12.68	−0.01
Violations / kLoC	30.77	31.86	−0.03
Technical Debt Ratio	45.28	46.09	−0.01
# Firm–cohort observations	247	19,686	

Note: This table reports covariate means at the last pre-treatment period ($t = -1$) for treated and control firms, using all control firms available in the stacked DiD panel. Firms can appear as controls in more than one cohort, so the observation counts refer to firm–cohort pairs. kLoC denotes 1,000 lines of code; SWE denotes software engineering. Log transformations follow Table 17. Technical Debt per LoC is measured in dollars per line, and the Technical Debt Ratio is calibrated debt as a percentage of the codebase's replacement cost. The observation-count row applies to code volume and quality; total and SWE headcount each have 201 treated and 10,080 control observations. Std. Diff. = (Treated Mean − Control Mean) / pooled standard deviation. See Appendix A, Table 17, for full variable definitions. Section 4.1 reports the matched-growth comparison.

3.2 Critical Violations

We first examine *Critical Violations*, severe deviations from coding best practices that can compromise the stability, security, or maintainability of a product. Unlike a deliberate decision to postpone a minor improvement, allowing serious code problems to accumulate can make subsequent development both riskier and more costly. A decline in this category therefore speaks to the discipline with which firms manage consequential engineering problems.

Critical violations fall by 1.85 per 1,000 lines of code after financing relative to controls (14.9% of the pre-treatment mean; $p = 0.03$; Table 3, Column (1)). More broadly, total rule violations across all severity levels decline by 4.74 per 1,000 lines (14.4% of the pre-treatment mean; $p = 0.02$; Column (4)). The critical-violation decline is concentrated among firms whose critical-violation density in the month before financing exceeds the treated-firm median: their decline is 3.9 per 1,000 lines larger than that of other treated firms ($p = 0.04$; Column (2)). Defining the groups by the six-month pre-financing average gives a difference of 3.6 ($p = 0.03$). Appendix N reports these alternatives and sensitivity to excluding firms with the highest initial violation densities.

Table 3: Difference-in-Differences Estimates for Critical Violations and All Violations

	Critical Violations			**Violations**		
	(1)	(2)	(3)	(4)	(5)	(6)
Treated × Post	−1.85**	−0.28	−2.29**	−4.74**	−4.60	−3.81**
	(0.86)	(0.39)	(1.15)	(2.02)	(2.98)	(1.72)
Post × High Violations (=1)		−1.31***			−2.17**	
		(0.32)			(1.00)	
Treated × Post × High Violations (=1)		−3.87**			−0.81	
		(1.89)			(3.74)	
Post × Large Round			0.02			−0.58
			(0.43)			(0.86)
Treated × Post × Large Round			1.00			−1.74
			(1.57)			(4.23)
Cohort × Firm FE	Yes	Yes	Yes	Yes	Yes	Yes
Month × Cohort FE	Yes	Yes	Yes	Yes	Yes	Yes
# Obs	487,577	477,107	487,577	487,577	477,107	487,577
R^2	0.88	0.89	0.88	0.86	0.86	0.86

This table reports the coefficient β from the specification $M_{i,c,t} = \beta VC_{i,c,t} + \chi_{c,i} + \delta_{c,t} + \gamma X_{i,c,t} + \varepsilon_{i,c,t}$, where $VC_{i,c,t}$ is the Treated × Post indicator. Critical Violations are severe SonarQube rule violations per 1,000 lines of code. Violations aggregate rule violations across all severity levels per 1,000 lines of code. See Appendix A, Table 17, for full variable definitions.

The treated pre-treatment means over months −12 to −1 in the estimation sample are 12.40 critical violations and 32.89 total violations per 1,000 lines.

High Violations is a dummy equal to one if the firm's pre-treatment critical violation density (at $t = -1$) exceeds the median across baseline firm–cohort observations of firms treated in any cohort, and zero otherwise.

Large Round is a dummy equal to one if the VC deal size (first Seed or Series A round amount, in U.S. dollars) exceeds the treated-firm median, and zero otherwise. Treated firms with no recorded round amount are in the reference group.

Note: Standard errors in parentheses. Stars report unadjusted pointwise tests: * p<0.1, ** p<0.05, *** p<0.01. Standard errors are clustered by cohort and firm.

Figure 1 traces the timing of this improvement. The pre-financing estimates show no systematic divergence between treated and control firms, and a joint test of the monthly pre-financing coefficients is not statistically significant ($p = 0.93$; Appendix Table 52), while negative coefficients emerge around financing and become larger over the post-financing quarters. The timing supports a shift in development practices following investor entry: it is not the pattern predicted by a gradual decline in critical-violation density already under way before the round. The fact that the improvement develops over subsequent quarters is also consistent with engineering practices taking time to affect

the accumulated codebase.

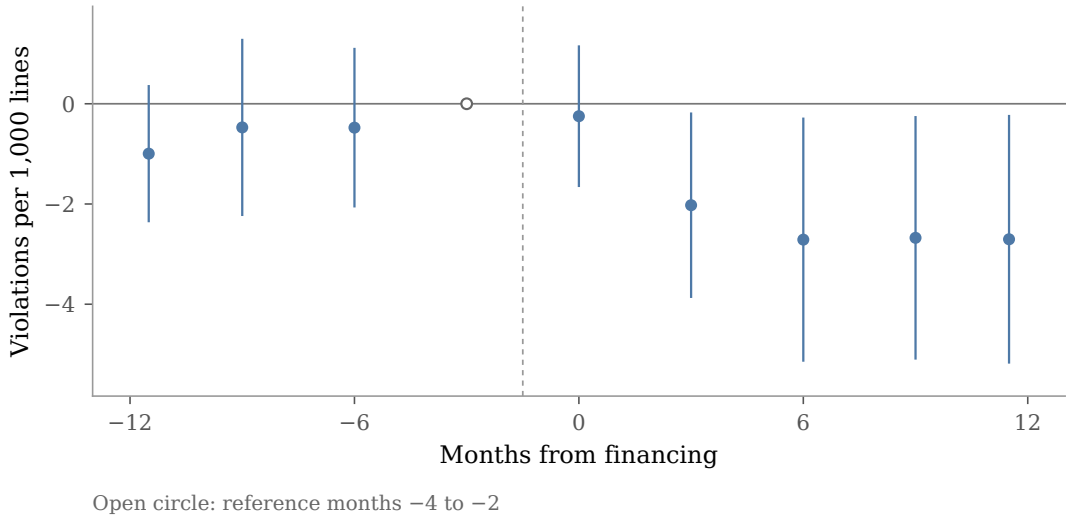


Figure 1: Critical Violations per 1,000 Lines of Code. Treated firms complete their first Seed round, or their first Series A round when no completed Seed is recorded; controls remain without a qualifying first round through the event window. Quarterly estimates are relative to months −4 to −2; the bin plotted at month 0 spans months −1 to +1. The dashed line marks the end of the reference months. Bars show 95% confidence intervals; standard errors are clustered by cohort and firm.

Figure 2 shows that the decline extends beyond critical violations and grows over the post-financing period. The pre-financing estimates are noisy, and a joint test of the three pre-financing coefficients is not statistically significant ($p = 0.78$). When we match firms on their pre-financing change in violation density, the pre-financing estimates move closer to zero while the post-financing decline remains (Figure 17).

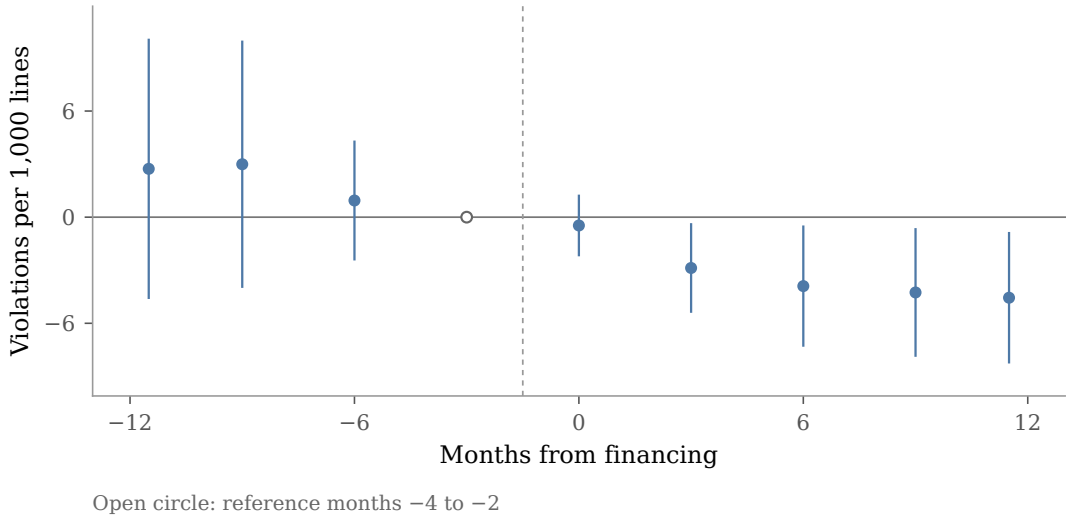


Figure 2: Violations (All Severities) per 1,000 Lines of Code. Treated firms complete their first Seed round, or their first Series A round when no completed Seed is recorded; controls remain without a qualifying first round through the event window. Quarterly estimates are relative to months −4 to −2; the bin plotted at month 0 spans months −1 to +1. The dashed line marks the end of the reference months. Bars show 95% confidence intervals; standard errors are clustered by cohort and firm.

3.3 Maintainability Violations and Security Hotspots

We next examine whether first VC entry improves the maintainability of software: the ease with which a codebase can be repaired, modified, or extended. Maintainability matters economically because an engineering shortcut that accelerates one release can increase the work required for every subsequent change. SonarQube flags code structures associated with this burden. Alongside the overall measure under the traditional classification, we examine high- and low-severity maintainability issues under the alternative classification described in Appendix M.2.

Maintainability violations decline by 4.8 per 1,000 lines of code after financing relative to controls (15.6% of the pre-treatment mean; $p = 0.01$; Table 4, Column (1)). When controls are matched on their pre-financing change in maintainability density, the decline is 3.3 per 1,000 lines ($p < 0.01$). High-severity issues under the alternative classification, which closely track critical violations (Appendix M.2), fall by 1.84 per 1,000 lines (15.0% of the pre-treatment mean; $p = 0.03$; Column (4)). Low-severity issues decline by 0.73 per 1,000 lines (10.0% of the pre-treatment mean; $p = 0.10$; Column (7)). The improvement thus includes the code structures that make future development most costly.

Table 4: Difference-in-Differences Estimates for Maintainability Violations

	Maintainability Violations (All)			**Maintainability Issues (High)**			**Maintainability Issues (Low)**		
	(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)
Treated × Post	−4.80**	−4.71	−3.92**	−1.84**	−0.27	−2.29**	−0.73	−0.68*	−1.03*
	(1.99)	(2.98)	(1.67)	(0.86)	(0.39)	(1.14)	(0.45)	(0.35)	(0.54)
Post × High Violations (=1)		−1.99**			−1.32***			−0.03	
		(0.95)			(0.32)			(0.51)	
Treated × Post × High Violations (=1)		−0.67			−3.86**			−0.14	
		(3.66)			(1.89)			(0.68)	
Post × Large Round			−0.67			0.01			−0.62*
			(0.81)			(0.43)			(0.35)
Treated × Post × Large Round			−1.58			1.03			1.11**
			(4.14)			(1.57)			(0.51)
Cohort × Firm FE	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Month × Cohort FE	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
# Obs	487,577	477,107	487,577	487,577	477,107	487,577	487,577	477,107	487,577
R^2	0.85	0.85	0.85	0.88	0.88	0.88	0.84	0.85	0.84

This table reports the coefficient β from the specification $M_{i,c,t} = \beta VC_{i,c,t} + \chi_{c,i} + \delta_{c,t} + \gamma X_{i,c,t} + \varepsilon_{i,c,t}$, where $VC_{i,c,t}$ is the Treated × Post indicator. Maintainability Violations (All) use the traditional classification. The high- and low-severity measures use the alternative classification described in Appendix M.2 and are not a breakdown of the traditional overall count. All three measures are counts per 1,000 lines of code; high and low distinguish the assessed impact on maintainability. See Appendix A, Table 17, for full variable definitions.

See Table 3 for the definitions of the High Violations and Large Round dummies.

The treated pre-treatment means over months −12 to −1 in the estimation sample are 30.83 for all maintainability violations, 12.29 for high-severity issues and 7.26 for low-severity issues, per 1,000 lines.

Note: Standard errors in parentheses. Stars report unadjusted pointwise tests: * p<0.1, ** p<0.05, *** p<0.01. Standard errors are clustered by cohort and firm.

Figure 3 traces the maintainability response around financing. In the baseline comparison, the early pre-financing estimates are noisy (Appendix D, Figure 18), although a joint test of the monthly pre-financing coefficients is not statistically significant ($p = 0.74$; Appendix Table 52). Figure 3 therefore shows the matched comparison, which leaves 193 of the 273 treated firms. In this comparison, the pre-financing estimates are close to zero. Figure 4 traces high-severity issues.

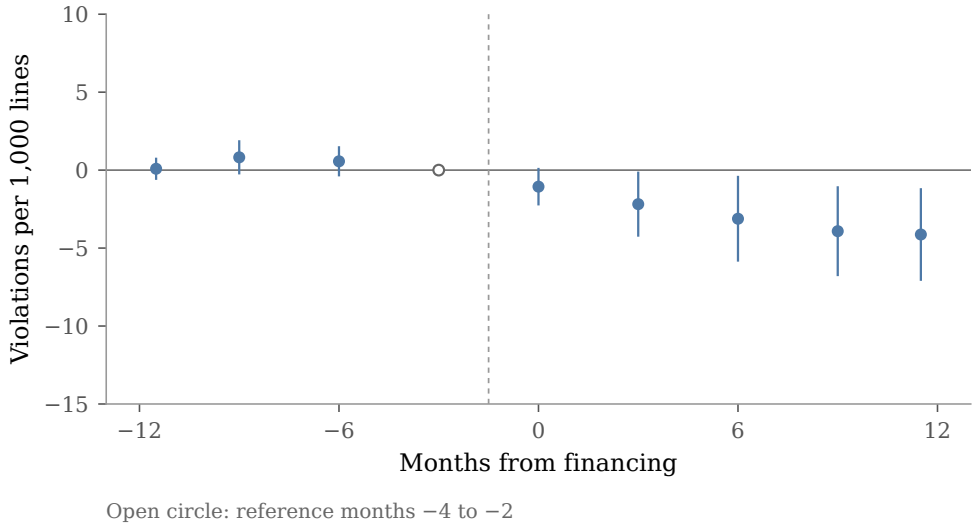


Figure 3: Maintainability Violations per 1,000 Lines of Code, Matched on Pre-Financing Density Change. Controls are matched on the change in maintainability violations per 1,000 lines between months $[-12, -11]$ and $[-4, -2]$ (193 treated firms in 90 cohorts; 335,092 stacked observations). Appendix D reports the matching procedure and the baseline comparison. Treated firms complete their first Seed round, or their first Series A round when no completed Seed is recorded; controls remain without a qualifying first round through the event window. Quarterly estimates are relative to months -4 to -2; the bin plotted at month 0 spans months -1 to $+1$. The dashed line marks the end of the reference months. Bars show 95% confidence intervals; standard errors are clustered by cohort and firm.

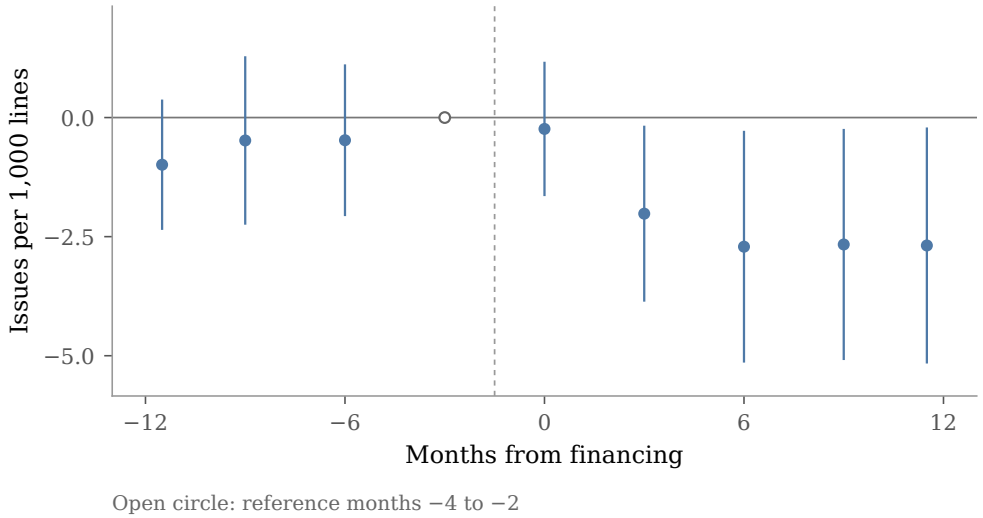


Figure 4: Maintainability Issues (High-Severity) per 1,000 Lines of Code. This measure uses the alternative classification described in Appendix M.2. Treated firms complete their first Seed round, or their first Series A round when no completed Seed is recorded; controls remain without a qualifying first round through the event window. Quarterly estimates are relative to months −4 to −2; the bin plotted at month 0 spans months −1 to +1. The dashed line marks the end of the reference months. Bars show 95% confidence intervals; standard errors are clustered by cohort and firm.

We also examine *Security Hotspots*, security-sensitive code segments that require human review to determine whether a vulnerability exists. Table 5 reports the estimates.

We find no statistically significant change in security hotspots after first VC entry (Column (1)), and the large-round interaction (Column (3)) is also not statistically significant ($p = 0.27$ for each).

Table 5: Difference-in-Differences Estimates for Security Hotspots

	Security Hotspots		
	(1)	(2)	(3)
Treated × Post	−1.18	−2.08	−0.02
	(1.09)	(1.93)	(0.09)
Post × High Violations (=1)		0.10	
		(0.09)	
Treated × Post × High Violations (=1)		2.05	
		(1.92)	
Post × Large Round			−0.10
			(0.11)
Treated × Post × Large Round			−2.59
			(2.38)
Cohort × Firm FE	Yes	Yes	Yes
Month × Cohort FE	Yes	Yes	Yes
# Obs	487,577	477,107	487,577
R^2	0.72	0.70	0.72

This table reports the coefficient β from the specification $M_{i,c,t} = \beta VC_{i,c,t} + \chi_{c,i} + \delta_{c,t} + \gamma X_{i,c,t} + \varepsilon_{i,c,t}$, where $VC_{i,c,t}$ is the Treated × Post indicator. Security Hotspots measure security-sensitive code segments requiring human review per 1,000 lines of code; unlike rule violations, hotspots represent potential risk areas. See Appendix A, Table 17, for full variable definitions.

See Table 3 for the definitions of the High Violations and Large Round dummies.

The treated pre-treatment mean over months −12 to −1 in the estimation sample is 2.16 hotspots per 1,000 lines.

Note: Standard errors in parentheses. * p<0.1, ** p<0.05, *** p<0.01. Standard errors are clustered by cohort and firm.

3.4 Bugs and Reliability

We next examine bugs and reliability issues, which concern incorrect behavior or failures during software execution. SonarQube's Bugs measure identifies code patterns that are likely to produce an incorrect result or unexpected behavior. The reliability measures capture related risks under the

alternative classification, with high-severity issues carrying greater potential impact and low-severity issues carrying smaller risks. Both assess problems identifiable from source code, allowing us to examine whether the gains in maintainability extend to the software's expected behavior in use.

We find no statistically significant change in bugs or reliability (Table 6): the Bugs estimate in Column (1) has $p = 0.95$, and the high- and low-severity reliability estimates in Columns (4) and (7) have $p = 0.70$ and $p = 0.45$, respectively. Figure 5 presents the corresponding event studies. Together with the security-hotspot results, these findings indicate that the measurable gains are concentrated in rule compliance and maintainability. Firms face a lower burden of modifying and extending the product, while we detect no comparable change in the code patterns associated with software failures.

Table 6: Difference-in-Differences Estimates for Bugs and Reliability Issues

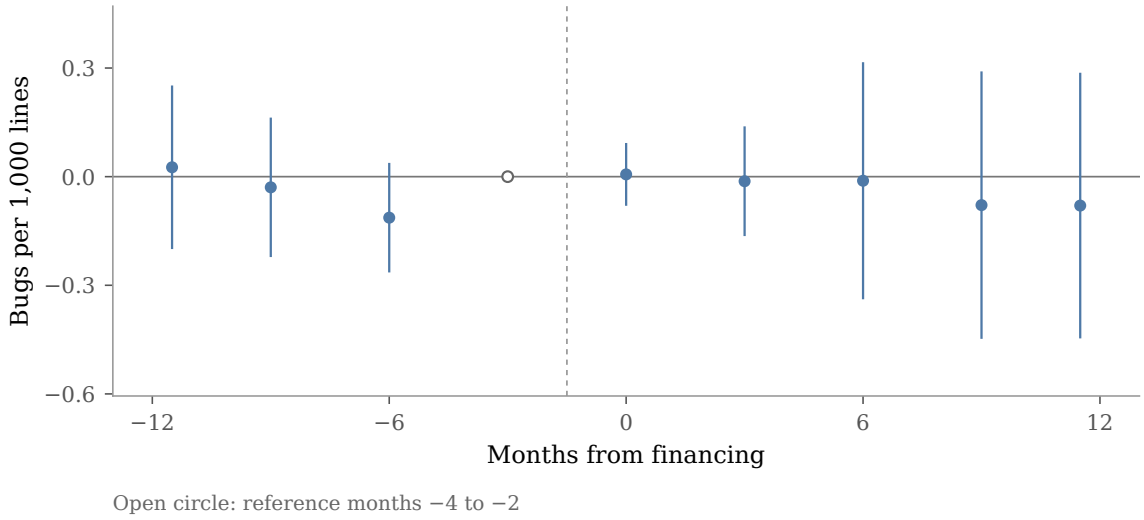
	Bugs			**Reliability Issues (High)**			**Reliability Issues (Low)**		
	(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)
Treated × Post	−0.01	−0.01	0.04	−0.01	0.00	0.00	−0.12	−0.31	−0.18
	(0.12)	(0.09)	(0.15)	(0.02)	(0.01)	(0.02)	(0.16)	(0.25)	(0.22)
Post × High Violations (=1)		−0.23**			0.01			−0.08	
		(0.11)			(0.02)			(0.10)	
Treated × Post × High Violations (=1)		−0.02			−0.02			0.43	
		(0.27)			(0.03)			(0.28)	
Post × Large Round			0.10			0.01			−0.03
			(0.08)			(0.01)			(0.07)
Treated × Post × Large Round			−0.18			−0.02			0.17
			(0.25)			(0.03)			(0.22)
Cohort × Firm FE	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Month × Cohort FE	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
# Obs	487,577	477,107	487,577	487,577	477,107	487,577	487,577	477,107	487,577
R^2	0.90	0.90	0.90	0.51	0.51	0.51	0.90	0.90	0.90

This table reports the coefficient β from the specification $M_{i,c,t} = \beta VC_{i,c,t} + \chi_{c,i} + \delta_{c,t} + \gamma X_{i,c,t} + \varepsilon_{i,c,t}$, where $VC_{i,c,t}$ is the Treated × Post indicator. Bugs count, per 1,000 lines of code, patterns classified by SonarQube as likely to cause incorrect or unexpected behavior. High- and Low-Severity Reliability Issues count, per 1,000 lines of code, issues at the stated severity associated with a risk of incorrect behavior. Bugs use the traditional classification; the reliability measures use the alternative classification described in Appendix M.2 and are not a breakdown of Bugs. See Appendix A, Table 17, for full variable definitions.

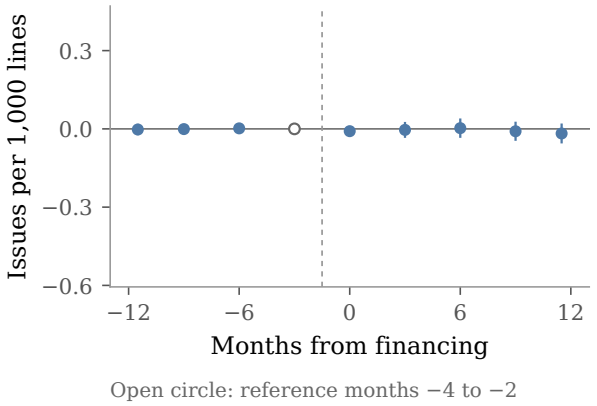
See Table 3 for the definitions of the High Violations and Large Round dummies.

The treated pre-treatment mean of Bugs over months −12 to −1 in the estimation sample is 1.82 per 1,000 lines.

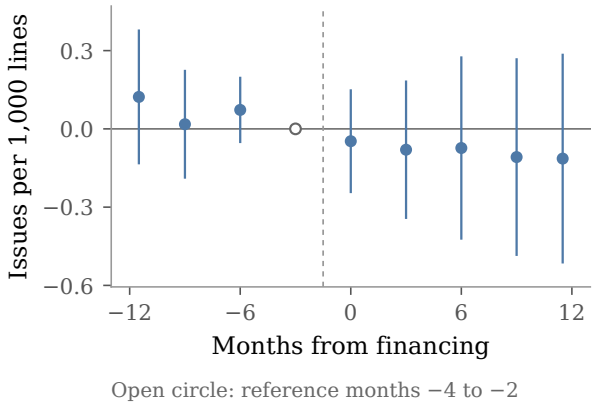
Note: Standard errors in parentheses. * p<0.1, ** p<0.05, *** p<0.01. Standard errors are clustered by cohort and firm.



(a) Bugs per 1,000 Lines of Code



(b) Reliability Issues (High Severity) per 1,000 lines of code



(c) Reliability Issues (Low Severity) per 1,000 lines of code

Figure 5: Bugs and Reliability Issues. Bugs use the traditional classification; the reliability measures use the alternative classification described in Appendix M.2. Treated firms complete their first Seed round, or their first Series A round when no completed Seed is recorded; controls remain without a qualifying first round through the event window. Quarterly estimates are relative to months −4 to −2; the bin plotted at month 0 spans months −1 to +1. The dashed line marks the end of the reference months. Bars show 95% confidence intervals; standard errors are clustered by cohort and firm.

3.5 Technical Debt

Better coding discipline can reduce the future cost of maintaining software. We measure this liability with *Technical Debt*, which converts SonarQube's estimated time required to resolve maintainability violations into dollars using the calibration in Section 2.5. The *Technical Debt Ratio* expresses the

same liability as a percentage of the software asset's estimated replacement cost. Both measures translate engineering problems into an estimated maintenance burden.

Table 7, Column (1), shows that technical debt declines by $1.57 per line of code after financing relative to controls (12.0% of the pre-treatment mean; $p = 0.04$). Correspondingly, the Technical Debt Ratio falls by 5.7 percentage points (12.0% of the pre-treatment mean; $p = 0.04$; Column (4)). Under the common calibration, this ratio decline means that the estimated maintenance liability falls by $5.69 for every $100 of software replacement cost. The improvement therefore represents a meaningful reduction in the future work associated with maintaining a given amount of software, in addition to the decline in unweighted violation counts. Figure 6 shows how technical debt per line evolves around financing.

Because we examine several quality outcomes, we also adjust their p-values for multiple testing with the Holm method, which controls the probability of any false rejection across a group of outcomes. The declines in critical violations and technical debt per line each have an adjusted p-value of 0.06, and adding the other violation measures to the group leaves the adjusted values between 0.06 and 0.07 (Appendix Table 54).

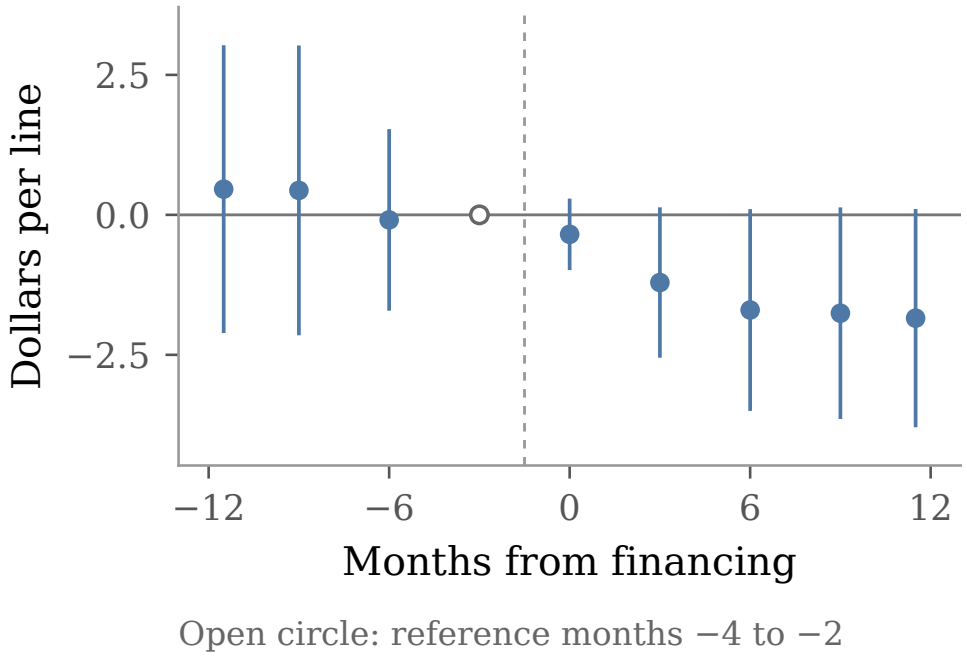


Figure 6: Technical Debt per Line of Code ($). Treated firms complete their first Seed round, or their first Series A round when no completed Seed is recorded; controls remain without a qualifying first round through the event window. Quarterly estimates are relative to months −4 to −2; the bin plotted at month 0 spans months −1 to +1. The dashed line marks the end of the reference months. Bars show 95% confidence intervals; standard errors are clustered by cohort and firm.

Table 7: Difference-in-Differences Estimates for Technical Debt

	Technical Debt per LoC ($)			**Technical Debt Ratio**		
	(1)	(2)	(3)	(4)	(5)	(6)
Treated × Post	−1.57**	−1.45	−1.25	−5.69**	−5.28	−4.54
	(0.77)	(0.98)	(0.81)	(2.78)	(3.56)	(2.96)
Post × High Violations (=1)		−0.52			−1.91	
		(0.52)			(1.88)	
Treated × Post × High Violations (=1)		−0.40			−1.45	
		(1.46)			(5.30)	
Post × Large Round			−0.19			−0.71
			(0.36)			(1.32)
Treated × Post × Large Round			−0.60			−2.16
			(1.51)			(5.49)
Cohort × Firm FE	Yes	Yes	Yes	Yes	Yes	Yes
Month × Cohort FE	Yes	Yes	Yes	Yes	Yes	Yes
# Obs	487,577	477,107	487,577	487,577	477,107	487,577
R^2	0.83	0.83	0.83	0.83	0.83	0.83

This table reports the coefficient β from the specification $M_{i,c,t} = \beta VC_{i,c,t} + \chi_{c,i} + \delta_{c,t} + \gamma X_{i,c,t} + \varepsilon_{i,c,t}$, where $VC_{i,c,t}$ is the Treated × Post indicator. Technical Debt per Line of Code is the calibrated repair cost in dollars per line. The Technical Debt Ratio is calibrated technical debt as a percentage of the firm's software assets, the replacement cost of its codebase. Section 2.5 defines both outcomes. See Table 3 for the definitions of the High Violations and Large Round dummies.

The treated pre-treatment means over months −12 to −1 in the estimation sample are $13.09 per line and 47.6% for the ratio. The ratio equals 40/11 times debt dollars per line on the same observations.

Note: Standard errors in parentheses. Stars report unadjusted pointwise tests: * p<0.1, ** p<0.05, *** p<0.01. Standard errors are clustered by cohort and firm.

Figure 7 summarizes the baseline estimates across the issue measures: critical, total and maintainability violations decline, while bugs, reliability issues and security hotspots show no statistically significant change.

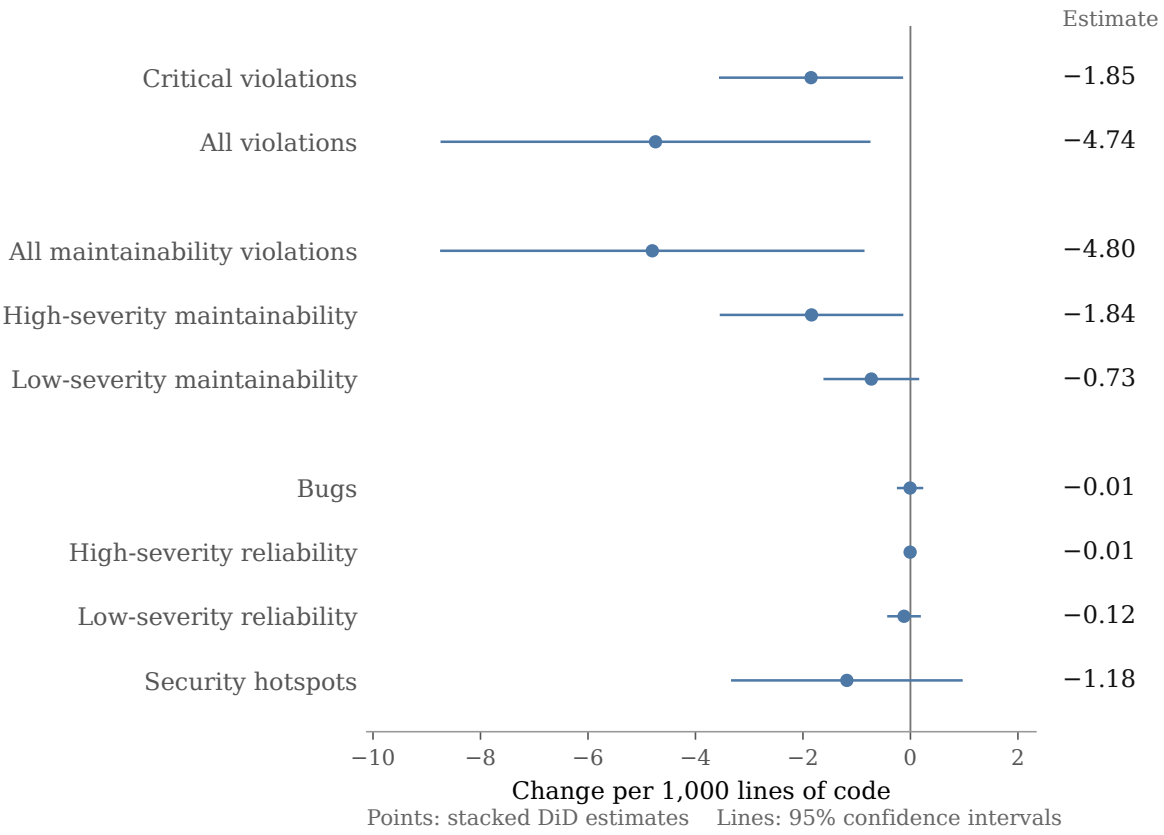


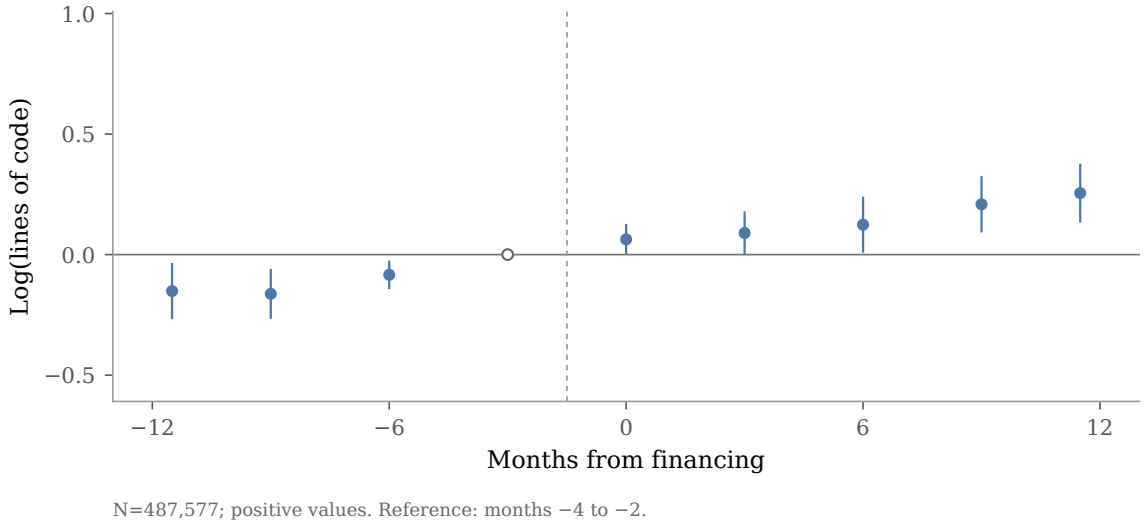
Figure 7: The Scope of the Engineering-Quality Improvement. Baseline estimates from Equation (7), in issues per 1,000 lines of code. Treated firms complete their first Seed round, or their first Series A round when no completed Seed is recorded; controls remain without a qualifying first round through the event window. Issue categories overlap. High- and low-severity maintainability and reliability use the alternative classification described in Appendix M.2 and are not breakdowns of the overall maintainability count or of Bugs. Bars show 95% confidence intervals; standard errors are clustered by cohort and firm.

3.6 Firm Scale and Engineering Capacity

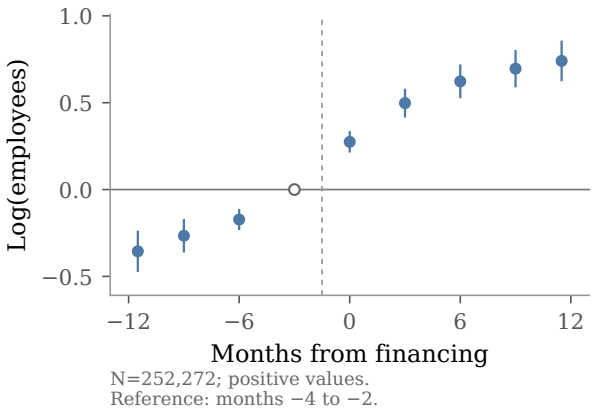
We next examine the expansion in firm scale and engineering capacity that accompanies first VC entry. This expansion provides the setting in which the quality improvements occur: firms are adding employees and extending their products while reducing violation density. Figure 8 plots the event-study estimates for code volume in Panel (a), total headcount in Panel (b), and software-engineering headcount in Panel (c).

The pre-treatment coefficients in Figure 8 show that treated firms were already expanding more rapidly in all three scale measures before financing. Parallel trends therefore do not hold for these outcomes, consistent with VC selection on fast-growing firms. The scaling estimates describe the

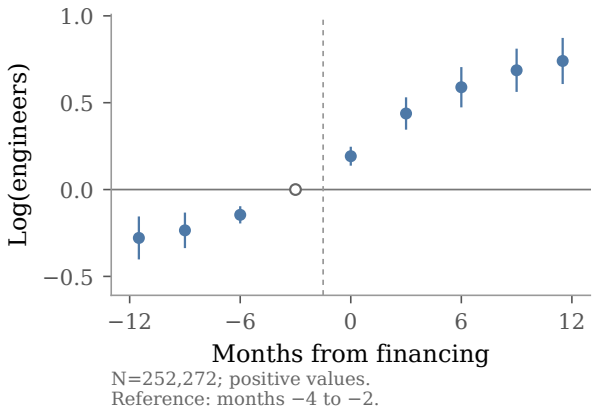
resource expansion surrounding investor entry; the identification of our quality results depends on the trajectories of the engineering outcomes themselves.



(a) Log(Lines of Code)



(b) Log(Total Employees)



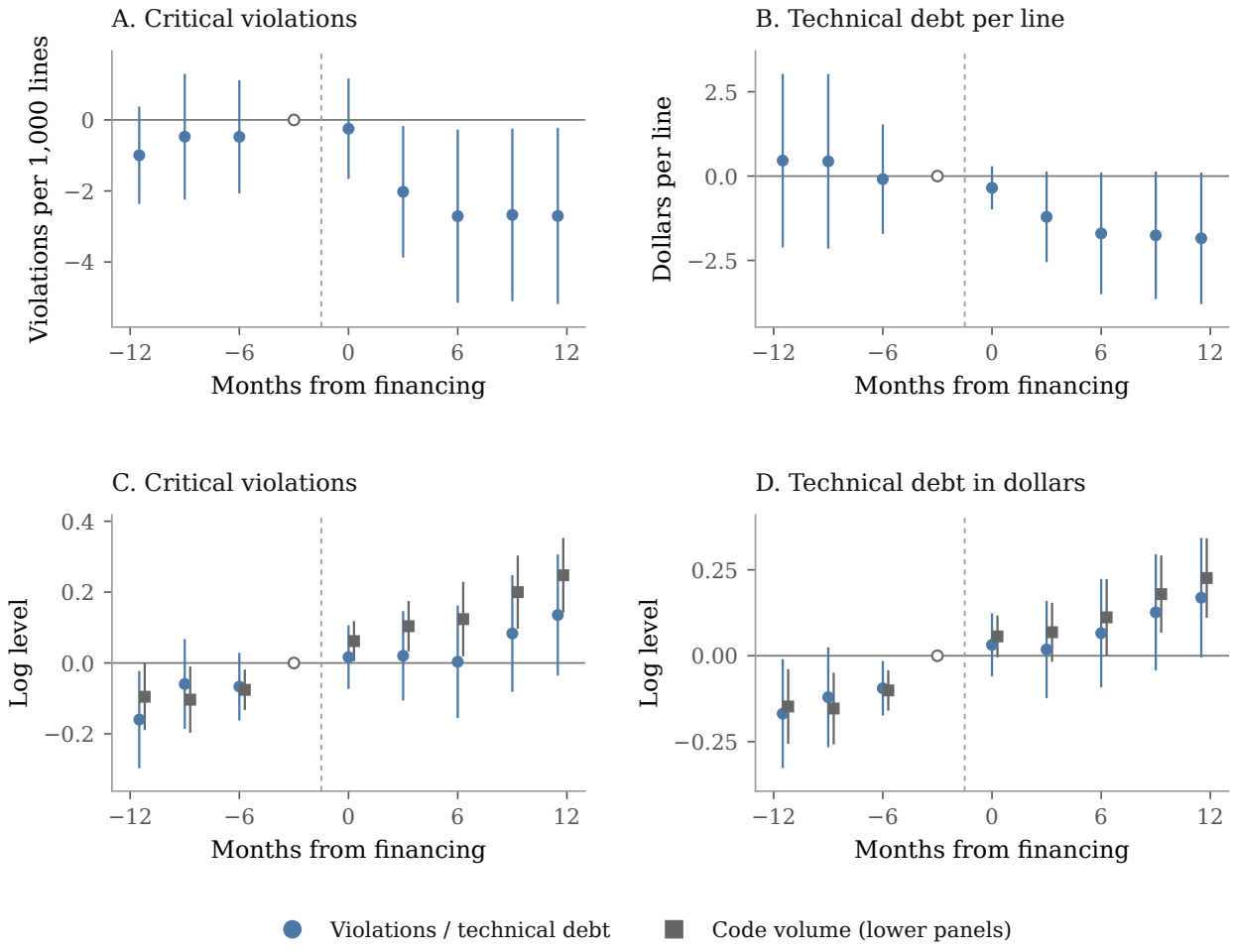
(c) Log(Software Engineers)

Figure 8: Firm and Product Scaling. Treated firms complete their first Seed round, or their first Series A round when no completed Seed is recorded; controls remain without a qualifying first round through the event window. Quarterly estimates are relative to months −4 to −2; the bin plotted at month 0 spans months −1 to +1. The dashed line marks the end of the reference months. Bars show 95% confidence intervals; standard errors are clustered by cohort and firm.

Organizational scale rises substantially after financing. By the end of the event window, the growth rates in total headcount and software-engineering headcount are each approximately 74 percentage points greater than in controls ($p < 0.01$ for each). Code volume also expands, although by less than headcount. This gap is consistent with the demands of building a larger engineering organization—including onboarding, review, and coordination—that need not translate immediately

into proportionate additions to the codebase. It also cautions against treating the number of lines produced as a sufficient measure of engineering output.

Figure 9 shows how quality changes as the codebase expands. The upper panels report critical violations per 1,000 lines and technical debt per line; the lower panels compare total critical violations and total technical debt with code volume. After financing, total critical violations and total technical debt grow by less than code volume relative to controls, so both fall per line of code. Appendix G discusses the relationship between these measures.



Open circle: reference months −4 to −2. Bars: 95% pointwise confidence intervals.

Figure 9: Engineering Quality During Codebase Expansion. Panels A–B show critical violations per 1,000 lines and technical debt in dollars per line. Panels C–D compare critical-violation counts and technical-debt levels with code volume, using the transformations defined in Table 17. Each lower-panel code-volume series uses the same observations as its stock series. Log specifications exclude zero stock values. Treated firms complete their first Seed round, or their first Series A round when no completed Seed is recorded; controls remain without a qualifying first round through the event window. Quarterly estimates are relative to months −4 to −2; the bin plotted at month 0 spans months −1 to +1. The dashed line marks the end of the reference months. Bars show 95% confidence intervals; standard errors are clustered by cohort and firm.

Growth and quality follow different trajectories before financing. If lower critical-violation density continued the same gradual improvement in firm capabilities that supported earlier growth, we would expect quality to improve before the round as well. Headcount and code volume diverge before financing, while the decline in critical violations emerges around investor entry. This separation in timing supports a change in development practices beyond the continuation of pre-existing growth. We test whether pre-existing growth explains the decline by matching treated firms to controls with similar pre-financing headcount growth in Section 4.1.

Rapid expansion also creates engineering challenges: more developers must coordinate changes to an increasingly complex product. Quality need not improve automatically as scale increases. The decline in violation density during this expansion is consistent with firms adopting practices that help them manage a larger development effort.

3.7 Capital and Governance: Mechanism Evidence

The results thus far show that first institutional VC entry is followed by lower violation density as firms expand. We next examine whether these improvements reflect the resources financing provides, the recruitment of senior technical talent, or changes in governance and development practices. The explanations imply different roles for investors. Additional capital can finance engineers, better tools, and technical-debt remediation; if these resources account for the gains, the improvement need not depend on how investors organize oversight. A governance channel instead predicts that quality responds to the monitoring and professionalization accompanying investment—including board oversight, operating advice, and more formal development procedures. We examine these predictions using investor structure and the composition of post-financing hiring.

We first examine the amount raised. If resources account for the gains, firms raising larger rounds should improve more. Firms raising rounds above the treated-firm median show no statistically significant difference in the critical-violation decline ($p = 0.52$; Table 3, Column (3)).

We next exploit variation in investor count. In larger syndicates, each investor bears the private cost of monitoring while sharing its benefits with more co-investors (Lerner, 1994; Casamatta and Haritchabalet, 2007; Kang, Li, and Oh, 2022). If these dispersed incentives weaken oversight, firms financed by larger syndicates should experience smaller quality improvements. To test this prediction, we classify a round as a large syndicate if it has more than four investors, the treated-firm median. This yields 120 treated firms with large syndicates among the 260 with nonmissing counts. We interact this indicator with treatment and estimate:

$$M_{i,c,t} = \beta_1 VC_{i,c,t} + \beta_2 VC_{i,c,t} \times \text{Syndicate}_i + \beta_3 \text{Post}_{c,t} \times \text{Syndicate}_i + \chi_{c,i} + \delta_{c,t} + \varepsilon_{i,c,t}, \quad (8)$$

where β_1 is the estimate for the reference group and $\beta_1 + \beta_2$ is the estimate for rounds with large

syndicates. A positive β_2 indicates smaller quality improvements when more investors share the round, as predicted by weaker individual monitoring incentives.[5]

Table 8 reports the predicted attenuation in critical violations. In Column (2), the decline among the remaining firms is 3.11 per 1,000 lines, 25% of the pre-treatment mean, and the large-syndicate interaction is +3.0 violations per 1,000 lines ($p = 0.07$), leaving a net estimate of −0.11 for larger syndicates, about 1% of the mean. Thus, firms financed by fewer co-investors experience the larger estimated reduction in critical violations. The technical-debt and code-volume interactions are not statistically significant ($p = 0.20$ and $p = 0.38$, respectively).

Table 8: Investor Structure and Engineering Outcomes

	Critical Violations / kLoC		**Technical Debt per LoC ($)**		**Log(Lines of Code)**	
	(1)	(2)	(3)	(4)	(5)	(6)
Treated × Post	−1.85**	−3.11**	−1.57**	−2.25*	0.23***	0.22***
	(0.86)	(1.42)	(0.77)	(1.22)	(0.05)	(0.07)
Post × Large Syndicate		−0.09		−0.20		0.16***
		(0.48)		(0.43)		(0.05)
Treated × Post × Large Syndicate		3.00*		1.74		−0.10
		(1.67)		(1.38)		(0.11)
Cohort × Firm FE	Yes	Yes	Yes	Yes	Yes	Yes
Month × Cohort FE	Yes	Yes	Yes	Yes	Yes	Yes
# Obs	487,577	487,577	487,577	487,577	487,577	487,577
R^2	0.88	0.88	0.83	0.83	0.93	0.93

This table reports heterogeneity in the treatment effect by syndicate size. Large Syndicate equals one if the first Seed or Series A treatment round has more than four investors, the treated-firm median. This definition classifies 120 treated firms as large syndicates among the 260 with nonmissing investor counts. The 14 treated firms with missing counts remain in the reference group, together with firms with four or fewer investors. Critical violations are counts per 1,000 lines of code; technical debt is calibrated dollars per line; code volume is log(LoC). Columns (1), (3), and (5) reproduce the baseline estimates. Columns (2), (4), and (6) introduce the large-syndicate interaction. All specifications include Firm × Cohort and Month × Cohort fixed effects. See Appendix A, Table 17, for variable definitions.

Note: Standard errors in parentheses, clustered by cohort and firm. * p<0.1, ** p<0.05, *** p<0.01.

Senior Technical Hiring

An alternative explanation is that VCs improve code quality by facilitating the hiring of senior technical talent. We examine this possibility using two measures constructed from LinkedIn employment histories. *Hired Technical Leader* identifies a chief technology officer (CTO), vice president (VP) of

[5]The reference group includes firms at or below the cutoff and firms with missing investor counts.

engineering, or head of engineering hire during the first 12 calendar months after financing. *High Senior Ratio* identifies firms whose senior engineering hires as a share of all new hires over that window exceed the treated-firm median of $1/7$. We restrict these comparisons to firms with LinkedIn coverage and estimate:

$$M_{i,c,t} = \beta_1 VC_{i,c,t} + \beta_2 VC_{i,c,t} \times H_i + \beta_3 \text{Post}_{c,t} \times H_i + \chi_{c,i} + \delta_{c,t} + \varepsilon_{i,c,t}, \tag{9}$$

where H_i denotes the hiring-composition indicator. If senior technical recruitment accounts for the quality gains, firms that add this talent should improve more, producing a negative interaction.

The hiring comparisons do not show larger quality gains among firms that recruit senior technical talent (Table 9). For critical violations, the differences between firms that hire a new technical leader and those that do not, and between firms with high and low senior-hire shares, are not statistically significant ($p = 0.31$ and $p = 0.69$, respectively). Appendix F presents the hiring definitions and confidence intervals for the hiring-group effects.

Table 9: Engineering Quality by First-Year Hiring Composition

	Critical Violations		**All Violations**		**Technical Debt per Line ($)**		**Maintainability**	
	Leader	Senior	Leader	Senior	Leader	Senior	Leader	Senior
Treated × Post	−2.87	−2.01	−4.16	−2.51	−1.50	−0.88	−4.07	−2.74
	(3.13)	(3.49)	(4.05)	(4.47)	(1.98)	(2.47)	(3.85)	(4.35)
Post × Hiring Group	0.20	0.03	−3.33	−2.22	−1.44	−0.74	−2.43	−1.89
	(1.29)	(1.25)	(3.25)	(2.56)	(1.31)	(1.25)	(2.79)	(2.25)
Treated × Post × Hiring Group	3.38	1.63	8.29	3.56	3.21	1.61	7.16	3.43
	(3.30)	(4.07)	(5.14)	(5.06)	(2.57)	(2.92)	(4.75)	(4.85)
Hiring-group effect: base + interaction	0.51	−0.38	4.12	1.04	1.71	0.73	3.09	0.69
	(2.00)	(2.51)	(5.07)	(4.35)	(2.28)	(1.94)	(4.54)	(3.94)
Firm × Cohort FE	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Month × Cohort FE	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
# Obs	31,925	26,451	31,925	26,451	31,925	26,451	31,925	26,451
# Firms	161	159	161	159	161	159	161	159
# Treated Firms	137	135	137	135	137	135	137	135

Critical, total and maintainability violations are counts per 1,000 lines; technical debt is the calibrated repair cost in dollars per line. Leader equals one for a new CTO, VP of Engineering or Head of Engineering hired from the cohort event date up to, but excluding, its 12-month anniversary. Senior equals one when senior engineering hires as a share of all new hires over that window exceed the treated-firm median of $1/7$. Both indicators use the cohort event date for treated and control firms. The leader and senior comparisons use separate samples of firms with LinkedIn coverage, each with 63 cohorts. Cohorts retain both treated and control firms after sample restrictions and fixed-effect singleton removal. Group effects sum the treatment and interaction coefficients, using their covariance for the standard error.

Note: Standard errors in parentheses, clustered by cohort and firm. * p<0.1, ** p<0.05, *** p<0.01.

We next examine hiring composition as an outcome. For technical and nontechnical leadership, we compare the probability of at least one qualifying hire over two equally long periods: months $[-12, -1]$ and $[0, 11]$ relative to financing. We retain firm–cohort pairs with observations throughout both periods. The software-engineering share and mean seniority of new hires are measured at quarterly frequency. Appendix Table 25 reports these comparisons, with cohort-by-firm fixed effects and either a period effect or cohort-by-quarter effects.

The software-engineering share of new hires rises by 5.8 percentage points (19.0% of the pre-treatment mean; $p = 0.04$). Changes in the probability of a technical- or nontechnical-leadership hire are not statistically significant ($p = 0.89$ and $p = 0.32$, respectively), nor is the change in the mean seniority of newly hired nonmanagerial software engineers ($p = 0.26$). Financing therefore shifts recruitment toward engineering.

Mechanism Synthesis

The mechanism tests are more consistent with governance than with resources or senior hiring. Critical violations decline more in rounds with fewer co-investors, where each investor captures more of the benefit of monitoring. Firms that raised larger rounds or that hired senior technical talent in the year after the round show no statistically significant difference in the decline, although financing shifts recruitment toward engineering. The governance channel also predicts more formal development procedures. Section 3.8 examines this prediction using the development histories, which record changes in procedures and in contributions from the existing organization. This evidence complements studies of VC involvement in portfolio-company operations (Gorman and Sahlman, 1989; Hellmann and Puri, 2002; Bernstein, Giroud, and Townsend, 2016).

3.8 Changes in the Development Process

We examine changes in development practices using individual code contributions from firms in the main sample. Version-control histories record when engineers submit changes, how they document and integrate them, and which earlier changes last modified subsequently repaired lines. We aggregate contributions across the repositories covered for each firm. Coverage varies with the data available for each outcome. The commit-message comparisons cover 5.6 million commits. Appendix L reports the construction rules, estimation samples, and comparisons estimated on identical observations.

Following VC entry, teams leave a more detailed record of how they develop software. To organize their work, teams can keep numbered records of tasks, reported bugs, and proposed changes. A commit message such as "Fix login error, #123" can then connect a change to the problem or discussion it addresses, helping others reconstruct what was done and why. The share of commit messages containing such references rises by 5.5 percentage points (14.1% of the pre-treatment mean; $p < 0.01$; Table 10).

Teams can also collect proposed changes in a pull request, allowing others to discuss them before they are incorporated into the main software product. Appendix Table 38 shows that references to pull requests rise by 6.2 percentage points ($p < 0.01$). The share of commits marked as merges or squashes—two ways of incorporating separately developed changes into the main product—rises by 6.5 percentage points ($p < 0.01$). The records also more frequently identify collaboration: the share of commits explicitly crediting a coauthor rises by 1.3 percentage points (44.4% of the pre-treatment mean; $p < 0.01$; Figure 10, Panel A). Together, these patterns indicate more systematic documentation of the work being undertaken, how changes are incorporated, and who contributed to them.

This greater emphasis on documentation also extends to copyright and licensing compliance. Table 10 reports a 1.9-percentage-point increase in the share of commits carrying a developer sign-off (25.2% of the pre-treatment mean; $p = 0.05$). A sign-off is a named declaration that the developer is entitled to contribute the code, either because it is their own work to provide or because they have

permission to contribute someone else's work under the project's license. The procedure originated in the Linux community during litigation over allegations that proprietary code had been incorporated without permission, establishing a documented trail of contributions and assurances about their legitimate use (The Linux Foundation, 2026). Projects can enforce this requirement through automated checks that prevent contributions from being accepted until the declarations are supplied (Dapr Authors, 2026). The increase after financing is consistent with professionalization of the procedures governing the legal acceptance of code, alongside the more systematic documentation of development work.

Table 10: Engineering Process Measures After First VC Entry

Measure	Estimate (pp)	p-value	# Obs	# Firms	# Treated
Contribution documentation (combined)	3.06***	< 0.01	300,380	371	213
	(0.95)				
Developer sign-offs	1.86*	0.05	300,380	371	213
	(0.94)				
Coauthor credits	1.35***	< 0.01	300,380	371	213
	(0.50)				
Review acknowledgements	0.02	0.77	300,380	371	213
	(0.05)				
Numbered issue references	5.51***	< 0.01	300,380	371	213
	(1.89)				
Headquarters-local weekend commits	−1.93***	< 0.01	171,116	273	199
	(0.71)				

Each row estimates Equation (7), with firm-by-cohort and calendar-month-by-cohort fixed effects. The estimation sample excludes fixed-effect singletons and cohorts without both treated and control observations. Each outcome is a monthly share of qualifying contributions, pooling numerator and denominator commits across each firm's covered projects and counting each commit once.
Contribution documentation consists of standardized lines in commit messages that record review, sign-off, acknowledgement, approval or coauthorship; these markers can overlap. Sign-offs record contributors' declarations about their right to supply the code; coauthorship credits another contributor. The first five rows use the same observations in 89 cohorts. Numbered references match issue or task numbers in messages. The weekend row uses Saturday and Sunday at the company's reported headquarters, applying date-specific time-zone rules; it uses 85 cohorts. Observations, firms and treated firms are reported for each row.
Coefficients and standard errors are in percentage points. Standard errors in parentheses are clustered by cohort and firm. * $p < 0.10$, ** $p < 0.05$, *** $p < 0.01$.

Following VC entry, developers record a smaller share of code contributions on weekends. We identify Saturdays and Sundays using the local time zone of each company's reported headquarters. Table 10 shows a 1.9-percentage-point decline in the share of weekend commits ($p < 0.01$). Relative to the treated firms' pre-financing weekend share of approximately 13%, this represents a reduction of about 15%.

Weekend contributions are also more frequently linked to subsequent software repairs. We trace later bug fixes back to earlier contributions to the affected code and compare contributions by the same developer within the same software project. Using the same headquarters-local definition of weekends, weekend commits are 0.7 percentage points more likely than weekday commits to be linked

to a bug fix within the following two years ($p < 0.01$; Appendix Table 39). Together, these findings are consistent with a more structured weekly development schedule after VC entry, with less activity on days when contributions are more often linked to later repairs.

The existing team becomes less likely to introduce bugs.[6] Among developers already active before financing, the probability that a contribution introduces a bug falls by 1.6 percentage points (10.6% of the pre-treatment mean; $p < 0.01$; Table 40). Decomposing this decline (Figure 10, Panel B), improvements within the same engineers account for 0.9 points ($p < 0.01$), while changes in who does how much work account for 0.6 points ($p = 0.07$). The latter reflects a shift in activity toward engineers with lower bug-introduction rates.

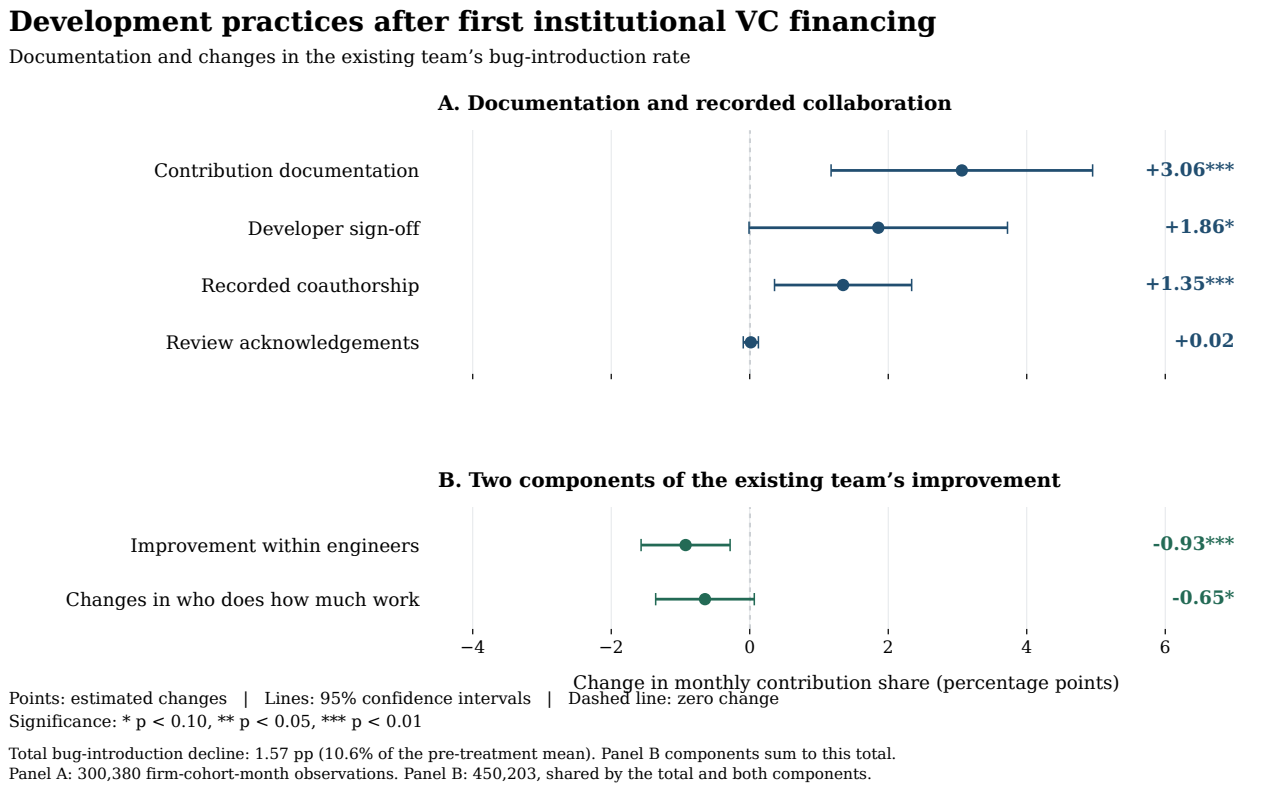


Figure 10: Development Practices and Bug Introduction Among Existing Engineers. Changes after first institutional VC entry, in percentage points. Panel A reports changes in the share of contributions documenting collaboration or developers' assurances about their right to supply the code. The combined measure includes sign-offs, coauthorship, review, acknowledgement and approval; these markers can overlap. Panel B divides the 1.6-percentage-point decline in bug introduction among engineers active before financing (10.6% of their pre-treatment mean) into improvement within engineers and changes in who does how much work. All comparisons pool each firm's covered repositories over months $[-12, 12]$. Tables 10 and 40 report construction details and estimation samples. Bars show 95% confidence intervals; standard errors are clustered by cohort and firm.

[6]This measure traces later bug fixes to the earlier contributions that last modified the repaired lines. It differs from SonarQube's Bugs measure in Section 3.4, which counts code patterns likely to cause failures.

Because the estimation samples differ across outcomes, we also estimate them on identical firm–cohort–month observations. More frequent contribution documentation, lower bug-introduction rates among pre-financing engineers, and lower critical-violation density appear there as well (Appendix Table 36).

Together, these findings indicate that institutional financing professionalizes how the existing team develops software: firms link code changes to recorded tasks, document copyright and licensing assurances, record collaboration more systematically, and concentrate less activity on weekends, while existing engineers become less likely to introduce bugs.

4 Robustness

4.1 Matching on Pre-Treatment Growth

A concern with the baseline comparison is that VC-backed firms are already growing faster before financing, as Figure 8 shows for both headcount and code volume. Faster growth may accompany managerial capabilities or product demand that also improve engineering outcomes. We address this concern by matching treated firms to controls with similar pre-treatment headcount growth. The resulting comparison asks whether quality improves after first VC entry even relative to other fast-growing startups that followed comparable trajectories but remained without institutional financing throughout the event window. The declines in critical, total and maintainability violations persist in this comparison.

Within each cohort, we retain controls whose pre-treatment headcount growth is close to the average growth of the treated firms. We measure growth as the difference in log average headcount between relative months $[-4, -2]$ and $[-12, -11]$:

$$G_i^c = \ln\left(\frac{1}{3}\sum_{t=-4}^{-2} \text{Headcount}_{i,c,t}\right) - \ln\left(\frac{1}{2}\sum_{t=-12}^{-11} \text{Headcount}_{i,c,t}\right).$$

Appendix D details the matching procedure. Matching to the treated-group average yields controls that resemble the typical newly financed startup in the cohort, while preserving a broader comparison group than one-to-one matching.

Improvements in software quality remain similar after matching firms on headcount growth before financing. Critical violations decline by 2.32 per 1,000 lines (14.4% of the pre-treatment mean; $p = 0.06$; Table 11, Column (1)). Total violations decline by 4.8 per 1,000 lines (13.1% of the pre-treatment mean; $p = 0.04$; Column (4)). Maintainability violations decline by 4.5 per 1,000 lines (12.9% of the pre-treatment mean; $p = 0.03$; Column (7)). The matching analysis covers 110 of the 274 treated firms, mainly because many firms lack the pre-financing data required for matching.

For these same 110 firms, critical violations decline by 2.57 per 1,000 lines with all eligible controls (16.0% of the pre-treatment mean; $p = 0.08$) and by 2.32 with matched controls (Appendix Table 51).

Table 11: Difference-in-Differences Estimates for Engineering Quality (Matched Sample)

	Critical Violations			**Violations (All)**			**Maintainability Violations**		
	(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)
Treated × Post	−2.32*	0.49	−2.76	−4.80**	−1.64	−4.72*	−4.46**	−1.46	−4.50*
	(1.22)	(0.83)	(1.71)	(2.37)	(1.21)	(2.44)	(2.11)	(1.16)	(2.31)
Post × High Violations (=1)		−1.80***			−4.58***			−4.16***	
		(0.45)			(1.67)			(1.48)	
Treated × Post × High Violations (=1)		−5.57*			−6.56			−6.19	
		(3.23)			(4.55)			(4.23)	
Post × Large Round			−0.42			−2.48*			−2.33*
			(0.51)			(1.33)			(1.22)
Treated × Post × Large Round			1.35			1.67			1.85
			(2.91)			(4.63)			(4.22)
Cohort × Firm FE	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Month × Cohort FE	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
# Obs	71,837	71,837	71,837	71,837	71,837	71,837	71,837	71,837	71,837
R^2	0.88	0.88	0.88	0.87	0.87	0.87	0.86	0.87	0.86

This table reports the coefficient β from the specification $M_{i,c,t} = \beta VC_{i,c,t} + \chi_{c,i} + \delta_{c,t} + \gamma X_{i,c,t} + \varepsilon_{i,c,t}$, where $VC_{i,c,t}$ is the Treated × Post indicator. The sample is restricted to treated firms and control firms matched on pre-treatment headcount growth. Critical Violations, Violations (All), and Maintainability Violations are measured per 1,000 lines of code. See Appendix D for details on the matching procedure and Appendix A, Table 17, for variable definitions.

See Table 3 for the definitions of the High Violations and Large Round dummies.

The matched treated pre-treatment means over months −12 to −1 in the estimation sample are 16.09 critical violations, 36.62 total violations and 34.54 maintainability violations per 1,000 lines.

Note: Standard errors in parentheses. * p<0.1, ** p<0.05, *** p<0.01. Standard errors are clustered by cohort and firm.

Technical debt declines by $1.29 per line and the Technical Debt Ratio by 4.7 percentage points in the matched sample (Table 12). Each decline equals 8.7% of its outcome's pre-treatment mean. These estimates retain much of their baseline magnitude but are less precise, with standard errors roughly 28% larger than in the full sample. Neither estimate is statistically significant ($p = 0.19$ for each).

Table 12: Difference-in-Differences Estimates for Technical Debt (Matched Sample)

	Technical Debt per LoC			**Technical Debt Ratio**		
	(1)	(2)	(3)	(4)	(5)	(6)
Treated × Post	−1.29	−0.09	−0.93	−4.70	−0.34	−3.39
	(0.98)	(0.59)	(1.07)	(3.57)	(2.15)	(3.91)
Post × High Violations (=1)		−1.58**			−5.74**	
		(0.77)			(2.80)	
Treated × Post × High Violations (=1)		−2.47			−8.97	
		(2.12)			(7.70)	
Post × Large Round			−0.88			−3.21
			(0.54)			(1.95)
Treated × Post × Large Round			−0.18			−0.65
			(1.89)			(6.87)
Cohort × Firm FE	Yes	Yes	Yes	Yes	Yes	Yes
Month × Cohort FE	Yes	Yes	Yes	Yes	Yes	Yes
# Obs	71,837	71,837	71,837	71,837	71,837	71,837
R^2	0.82	0.82	0.82	0.82	0.82	0.82

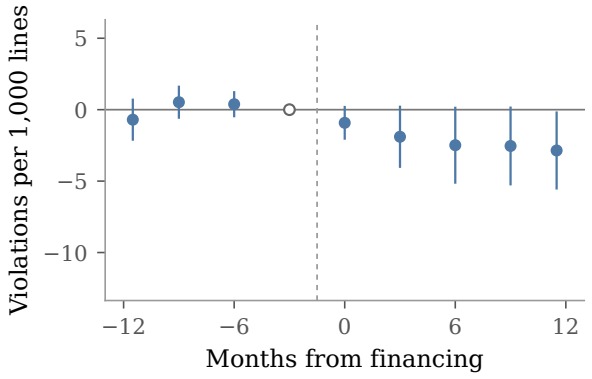
This table reports the coefficient β from the specification $M_{i,c,t} = \beta VC_{i,c,t} + \chi_{c,i} + \delta_{c,t} + \gamma X_{i,c,t} + \varepsilon_{i,c,t}$ on firms matched by pre-treatment headcount growth. Technical Debt per Line of Code is calibrated dollars per line; the Technical Debt Ratio is calibrated technical debt as a percentage of software assets. See Section 2.5 and Appendix D.

See Table 3 for the definitions of the High Violations and Large Round dummies.

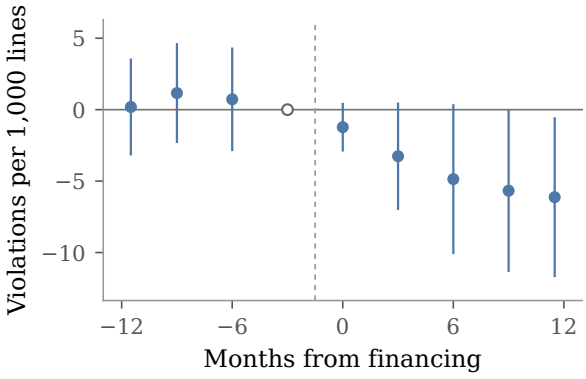
The matched treated pre-treatment means over months −12 to −1 in the estimation sample are $14.81 per line and 53.8% for the ratio.

Note: Standard errors in parentheses. * p<0.1, ** p<0.05, *** p<0.01. Standard errors are clustered by cohort and firm.

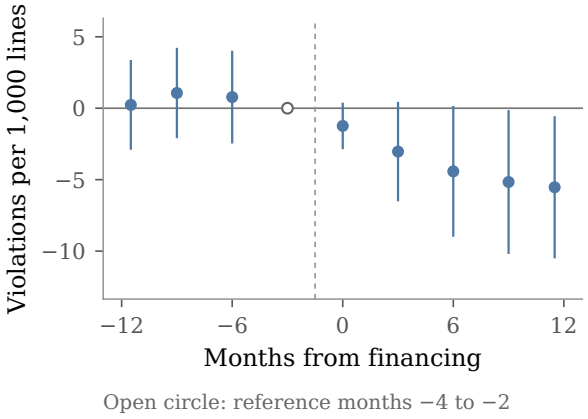
The matched event studies for critical, total, and maintainability violations (Figure 11) and technical debt per line (Figure 12) show a timing pattern similar to the baseline results. Pre-financing coefficients display no systematic divergence, and the post-financing estimates indicate lower violation densities and technical debt. Confidence intervals are wider in the smaller matched sample.



(a) Critical Violations per 1,000 LoC



(b) Violations (All) per 1,000 LoC



(c) Maintainability Violations per 1,000 LoC

Figure 11: Engineering Quality (Matched Sample). Treated firms complete their first Seed round, or their first Series A round when no completed Seed is recorded; controls remain without a qualifying first round through the event window. Controls are matched on pre-financing headcount growth within cohorts. Quarterly estimates are relative to months −4 to −2; the bin plotted at month 0 spans months −1 to +1. The dashed line marks the end of the reference months. Bars show 95% confidence intervals; standard errors are clustered by cohort and firm.

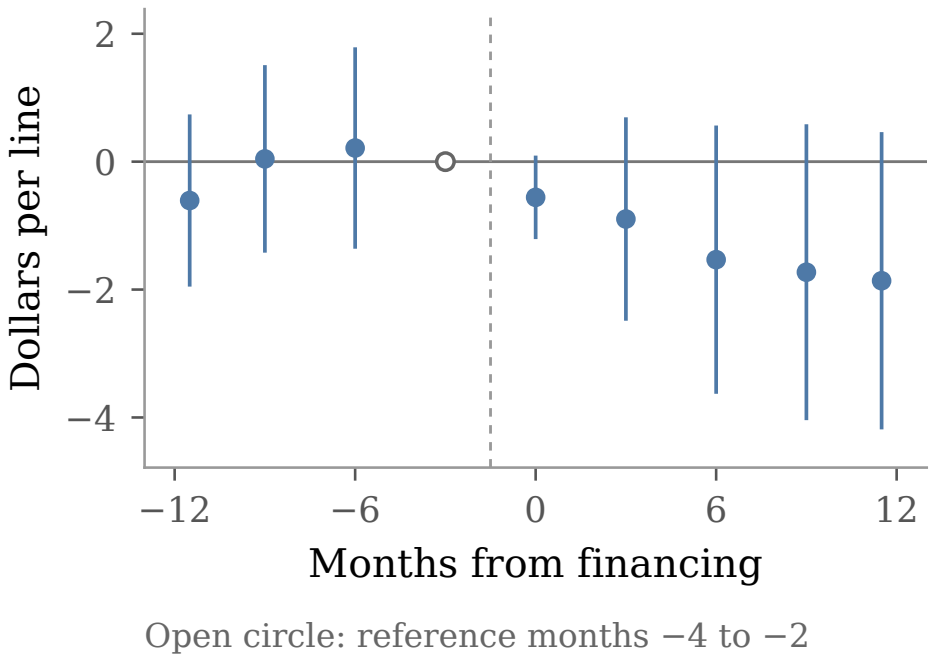
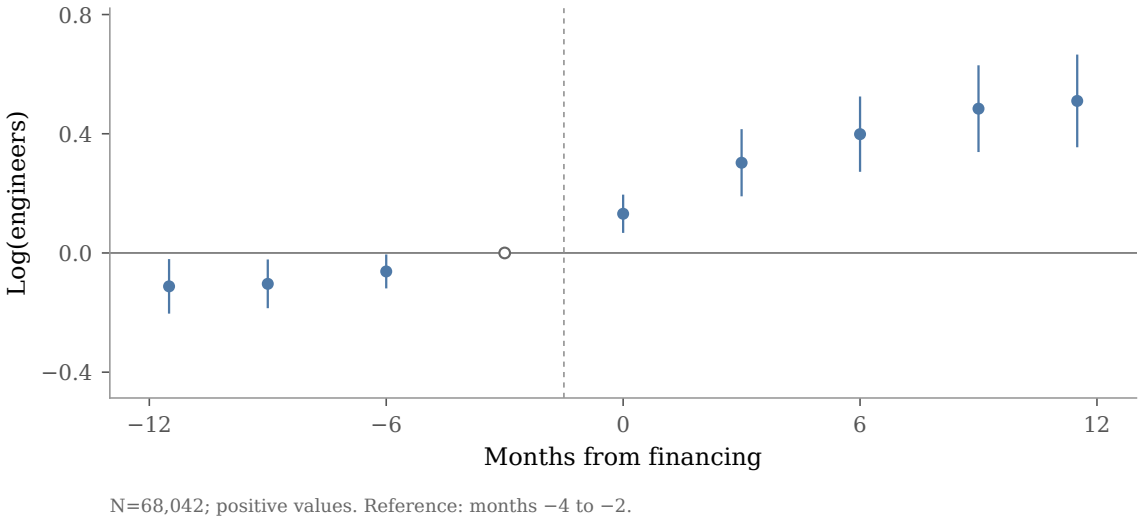
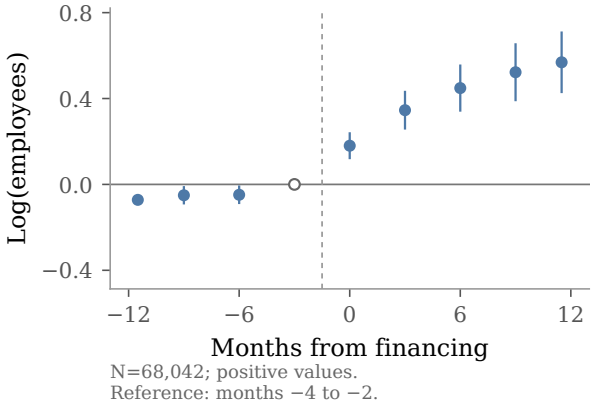


Figure 12: Technical Debt per Line of Code (Matched Sample). Treated firms complete their first Seed round, or their first Series A round when no completed Seed is recorded; controls remain without a qualifying first round through the event window. Controls are matched on pre-financing headcount growth within cohorts. Quarterly estimates are relative to months −4 to −2; the bin plotted at month 0 spans months −1 to +1. The dashed line marks the end of the reference months. Bars show 95% confidence intervals; standard errors are clustered by cohort and firm.

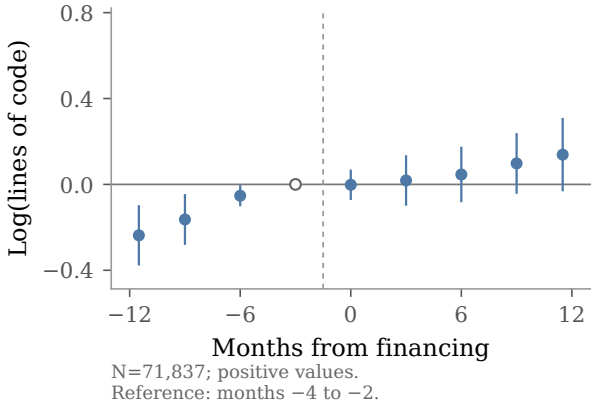
Matching on total-headcount growth improves the comparability of the growth trajectories, although pre-financing differences remain in engineering headcount and code volume (Figure 13). We therefore examine alternative matches based on engineering-headcount growth and on combinations of growth measures that include code volume (Appendices D and E). Matching directly on engineering-headcount growth yields a similar post-financing path: by the end of the event window, engineering-headcount growth is approximately 51 percentage points greater than in controls under the primary match and 53 percentage points greater under the engineering-growth match (Figure 16). Appendix Table 18 reports the corresponding difference-in-differences estimates for headcount.



(a) Log(Software Engineers)



(b) Log(Total Employees)



(c) Log(Lines of Code)

Figure 13: Firm and Product Scaling (Matched Sample). Treated firms complete their first Seed round, or their first Series A round when no completed Seed is recorded; controls remain without a qualifying first round through the event window. Controls are matched on pre-financing headcount growth within cohorts. Quarterly estimates are relative to months −4 to −2; the bin plotted at month 0 spans months −1 to +1. The dashed line marks the end of the reference months. Bars show 95% confidence intervals; standard errors are clustered by cohort and firm.

To assess whether the findings depend on how we select comparable controls, we compare estimates across 72 matching choices (Simonsohn, Simmons, and Nelson, 2020). We vary the growth measures used to select controls, whether controls are matched to the treated-group average or to individual firms, and how closely their prior growth must align. Across all 72 specifications, every critical-violation, total-violation, and maintainability coefficient is negative; 85%, 74%, and 79%, respectively, reach the 10% significance level, and 26%, 61%, and 61% reach the 5% level. Technical-

debt coefficients are negative in 68 of the 72 specifications. Appendix E reports the specification curves and full details.

Overall, the matching analysis supports the robustness of the main quality findings. The declines in critical, total and maintainability violations remain statistically significant ($p = 0.06$, $p = 0.04$ and $p = 0.03$, respectively). Technical debt retains most of its estimated decline, although that estimate is not statistically significant ($p = 0.19$). The improvement therefore persists in a comparison that substantially narrows the observed differences in pre-financing growth, making those differences an unlikely explanation for the baseline quality pattern.

4.2 Later Financing Rounds

We next compare engineering quality around first entry with quality around later financing rounds, which bring fresh capital to firms that already have institutional investors. First entry can introduce investor oversight and operating practices that persist into subsequent rounds (Hellmann and Puri, 2002; Kaplan and Strömberg, 2003). Later financing also supplies resources for engineers and tools. If capital alone accounts for the quality gains, later rounds should be followed by similar improvements. If the gains depend on the oversight and practices that arrive with first entry, they should not. We find that the improvements do not recur.

We construct the later-round comparison by re-centering the stacked difference-in-differences design on each firm's first completed Series B-or-later round recorded in PitchBook (Series B, C, D, and subsequent rounds). As in the main analysis, each cohort covers the 25 months from $e = -12$ through $e = +12$ relative to financing. For each Series B+ cohort, controls are firms with no Seed or Series A round through the end of the event window.

We reestimate Equation (7), with the treatment indicator equal to one for later-round recipients from their first Series B+ round onward. We retain cohort × firm and cohort × month fixed effects and cluster standard errors by cohort and firm. The outcomes are the same engineering-quality measures, together with code volume, total headcount, and software-engineering headcount.

Figure 14 compares the first-entry and later-round estimates on the same scale within each outcome. Headcount rises in both comparisons, whereas the later-round quality estimates are closer to zero. Figure 15 plots the later-round event studies for code volume in Panel (a), total headcount in Panel (b), and software-engineering headcount in Panel (c).

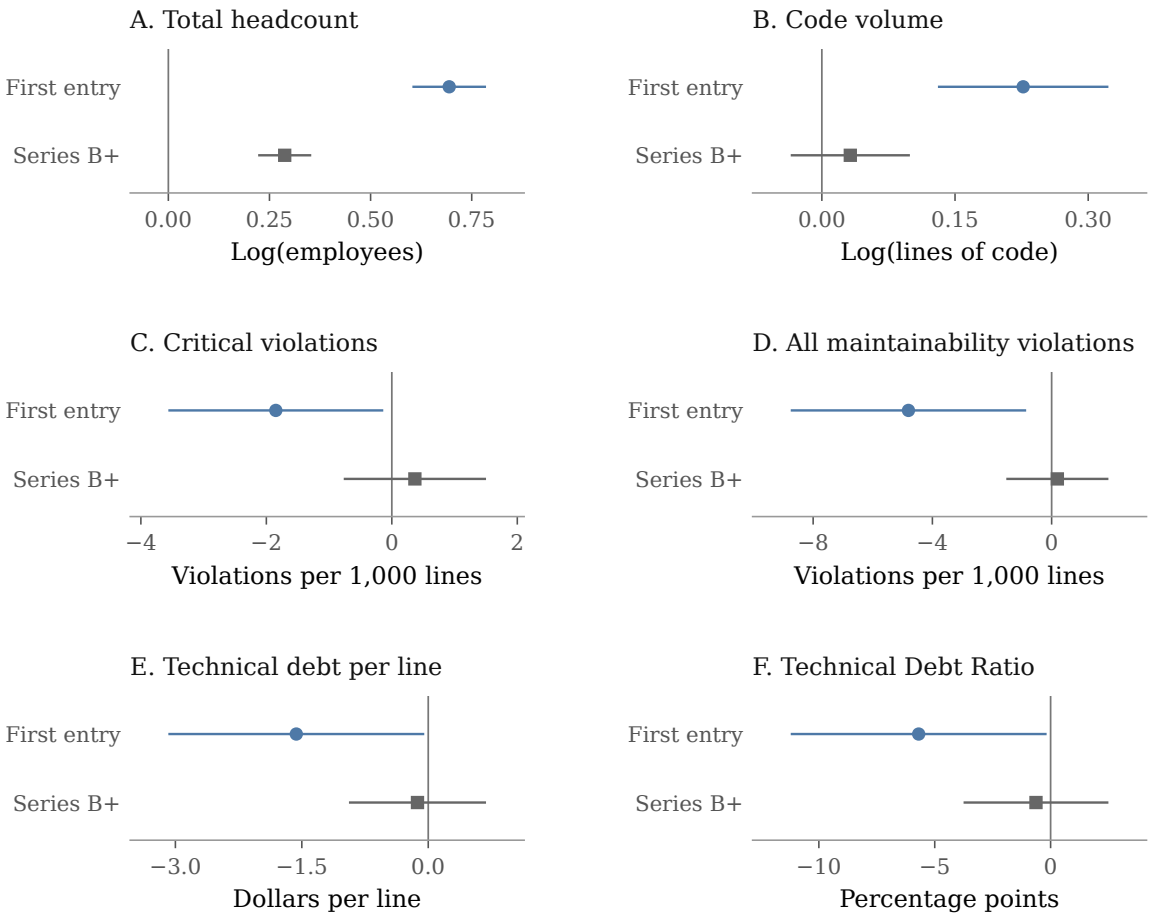
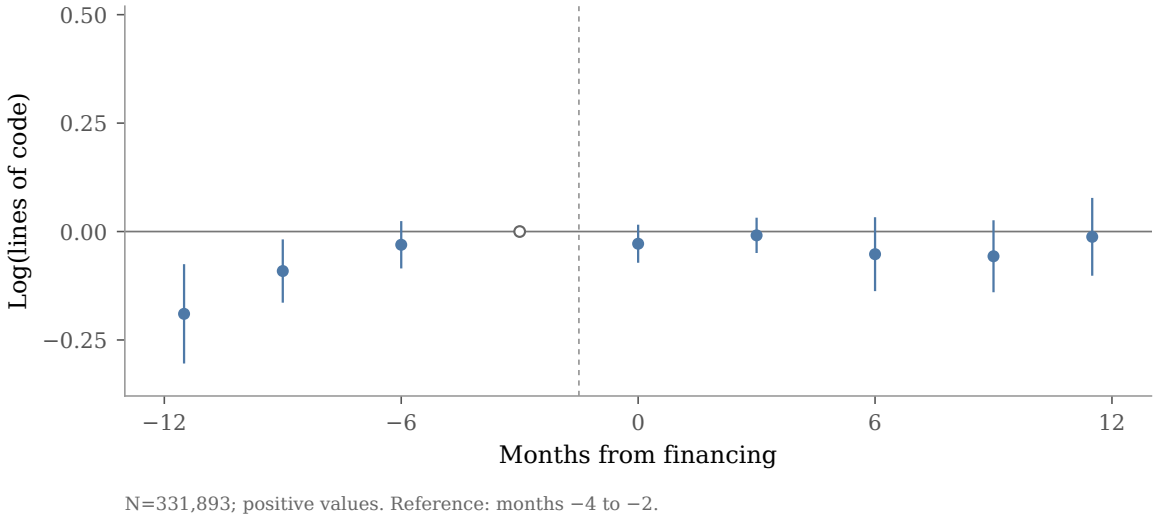
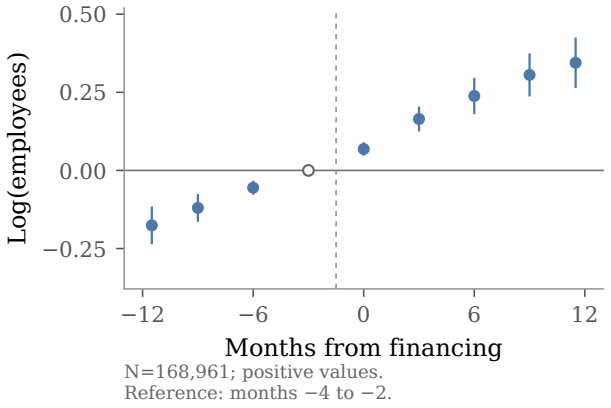


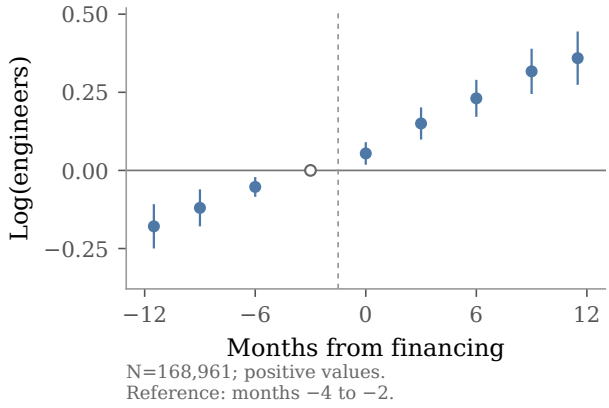
Figure 14: First Institutional Entry and Later Financing Rounds. Separate estimates within 12 months of first entry (circles) and the first Series B-or-later round (squares). Controls remain without a completed Seed or Series A round through each window. Units appear below each panel. Table 13 tests the differences between the two estimates for the quality outcomes. Bars show 95% confidence intervals; standard errors are clustered by cohort and firm.



(a) Log(Lines of Code)



(b) Log(Total Employees)



(c) Log(Software Engineers)

Figure 15: Firm and Product Scaling (Series B+ rounds). Treated firms complete their first Series B- or-later round. Controls remain without a completed Seed or Series A round through the event window. Quarterly estimates are relative to months −4 to −2; the bin plotted at month 0 spans months −1 to +1. The dashed line marks the end of the reference months. Bars show 95% confidence intervals; standard errors are clustered by cohort and firm.

Panels (b) and (c) show substantial headcount expansion after later financing, including a rise in the number of software engineers. Code volume in Panel (a), however, shows no statistically significant change ($p = 0.34$).

Later-round recipients show no statistically significant improvement in the quality measures relative to firms that remain without a Seed or Series A round through the event window (Figure 14; Table 13). For critical violations, the first-entry estimate is 2.2 violations per 1,000 lines lower than the later-round estimate (17.9% of the first-entry pre-treatment mean; $p < 0.01$). The difference between

stages is also statistically significant for every other quality measure that improves at first entry (largest $p = 0.03$). Appendix J reports the full estimates and event studies.

Table 13: First Entry versus Later-Round Coefficients

Outcome	**Baseline** (1)	**Series B+** (2)	**Difference** (1)−(2)	**SE**
Critical Violations	−1.85**	0.37	−2.22***	(0.80)
Violations (All)	−4.74**	0.13	−4.87**	(1.94)
Maintainability Violations (All)	−4.80**	0.19	−5.00**	(1.91)
Maintainability Issues (High)	−1.84**	0.38	−2.22***	(0.79)
Maintainability Issues (Low)	−0.73	−0.39	−0.34	(0.29)
Security Hotspots	−1.18	0.07	−1.26	(1.09)
Bugs	−0.01	−0.13	0.13	(0.14)
Reliability Issues (High)	−0.01	−0.01	0.00	(0.01)
Reliability Issues (Low)	−0.12	−0.15	0.03	(0.16)
Technical Debt	−1.57**	−0.13	−1.44**	(0.68)
Technical Debt Ratio	−5.69**	−0.46	−5.23**	(2.46)

Columns (1) and (2) report the Treated × Post coefficient for each outcome from the baseline Seed/Series A specification (Tables 3, 4, 5, 6, and 7) and from the Series B+ placebo specification (Tables 28, 29, 30, 31, and 32). The high- and low-severity maintainability and reliability rows count issues per 1,000 lines of code using the alternative classification documented in Appendix A; they are not breakdowns of Maintainability Violations (All) or Bugs, which use the traditional classification. Table 43 compares the two classifications on identical observations. The difference (1)−(2) is estimated in a single regression that stacks the two samples with sample-specific Cohort × Firm and Month × Cohort fixed effects. Because many firms appear in both samples, the two estimates are correlated; the standard error of the difference, clustered by cohort and firm, accounts for this covariance. The Technical Debt Ratio is 40/11 times debt per line on the same observations with positive lines of code; its coefficients are expressed in percentage points. Red text identifies the later-round estimates. * p<0.1, ** p<0.05, *** p<0.01.

The difference between the first-entry and later-round estimates is consistent with professionalization at first institutional entry. Section 3.8 supports this interpretation directly: firms record collaboration more systematically, and existing engineers become less likely to introduce bugs.

4.3 Lifecycle Stage and Commercialization

The first institutional round may also coincide with commercialization. As firms begin serving paying customers, they may adopt more demanding engineering practices. If the initial transition to revenue generation explains our findings, quality gains should be concentrated among firms making that transition.

We test this prediction using commercial status at financing: 183 of the 274 treated firms (67%) were already generating revenue at first entry, and the estimated quality gains are not larger among the remaining pre-commercial firms.

Table 14 shows that several main outcomes improve among firms already generating revenue. Within this group, total violations decline by 5.92 per 1,000 lines, technical debt by $2.23 per line, and the Technical Debt Ratio by 8.1 percentage points ($p = 0.04$ for each). Critical violations in this group decline by 1.90 per 1,000 lines ($p = 0.11$); among pre-commercial firms, they decline by 1.74 ($p = 0.07$).

Table 14: Treatment Effects by Commercial Stage at Entry: Subsample Estimates

	Critical Violations	**Violations**	**Technical Debt**	**Technical Debt Ratio**
	(1)	(2)	(3)	(4)
Panel A: Treated firms already generating revenue at entry (183 firms)				
Treated × Post	−1.90	−5.92**	−2.23**	−8.11**
	(1.20)	(2.90)	(1.07)	(3.91)
# Obs	390,179	390,179	390,179	390,179
Panel B: Treated firms pre-commercial at entry (91 firms)				
Treated × Post	−1.74*	−2.40	−0.24	−0.89
	(0.95)	(1.45)	(0.54)	(1.96)
# Obs	297,099	297,099	297,099	297,099
Cohort × Firm FE	Yes	Yes	Yes	Yes
Month × Cohort FE	Yes	Yes	Yes	Yes

Each panel estimates Equation (7) on treated firms with the stated commercial stage and the controls in their cohorts. Commercial stage is the PitchBook business status at the deal closest to treatment, matched within three months. Firms already generating revenue have the status Generating Revenue, Generating Revenue/Not Profitable, or Profitable. Cohorts left with only treated or only control firms are dropped. Critical Violations and Violations are counts per 1,000 lines of code; Technical Debt is calibrated dollars per line; the Technical Debt Ratio is calibrated technical debt as a percentage of software assets. Joint lead tests show no evidence of differential pre-treatment trends; none is statistically significant.
Note: Standard errors in parentheses. * p<0.1, ** p<0.05, *** p<0.01. Standard errors are clustered by cohort and firm.

Table 15 directly tests whether the response differs by commercial stage. The pre-commercial interaction is not statistically significant for critical and total violations ($p = 0.92$ and $p = 0.27$, respectively). For both debt measures, however, it is positive, indicating a smaller decline among pre-commercial firms ($p = 0.08$ for each). Excluding firms with more than five repositories yields similar estimates. The point estimates therefore do not show the predicted concentration of gains among firms entering commercialization; for technical debt, the direction of the difference is the opposite.

Table 15: Commercial Stage at Entry: Treatment-Effect Differences

	Critical Violations	**Violations**	**Technical Debt**	**Technical Debt Ratio**
	(1)	(2)	(3)	(4)
Panel A: Main sample				
Treated × Post	−1.90	−5.92**	−2.23**	−8.11**
	(1.20)	(2.90)	(1.07)	(3.90)
Treated × Post × Pre-commercial	0.14	3.49	1.97*	7.18*
	(1.50)	(3.16)	(1.13)	(4.11)
Implied effect, pre-commercial firms	−1.76*	−2.43*	−0.26	−0.93
	(0.95)	(1.45)	(0.54)	(1.96)
# Obs	487,577	487,577	487,577	487,577
Panel B: At most five repositories per firm				
Treated × Post	−1.91	−6.01**	−2.34**	−8.51**
	(1.23)	(2.99)	(1.11)	(4.02)
Treated × Post × Pre-commercial	0.12	3.56	2.10*	7.64*
	(1.55)	(3.26)	(1.17)	(4.24)
# Obs	447,156	447,156	447,156	447,156
Cohort × Firm FE	Yes	Yes	Yes	Yes
Month × Cohort FE	Yes	Yes	Yes	Yes

This table estimates $M_{i,c,t} = \beta_1 VC_{i,c,t} + \beta_3 \left(VC_{i,c,t} \times \text{Pre-commercial}_i\right) + \chi_{c,i} + \delta_{c,t} + \varepsilon_{i,c,t}$ on the full estimation sample, retaining every eligible control firm. Firms already generating revenue at entry are the omitted category: β_1 (Treated × Post) is the treatment effect for these firms, and β_3 is the incremental effect for pre-commercial firms. Pre-commercial equals one for treated firms not yet generating revenue at the financing deal closest to the treatment date (PitchBook deal business status, matched within three months) and zero for treated firms already generating revenue. It is identically zero for control firms, whose commercial stage at the cohort baseline is unobservable. The specification estimates treatment-effect heterogeneity across treated firms on the common control pool. The level and Post interactions of the indicator are absorbed by the fixed effects or collinear with the triple term and are not separately identified. The implied effect for pre-commercial firms is $\beta_1 + \beta_3$; its standard error comes from the equivalent reparameterization with pre-commercial firms as the base group. Joint tests show no evidence of differential pre-treatment leads by commercial stage for any of the four outcomes; none is statistically significant. Panel B drops firms with more than five scanned repositories before aggregation. Critical and total violations are counts per 1,000 lines of code; technical debt is calibrated dollars per line, and the Technical Debt Ratio is calibrated debt as a percentage of software replacement cost.

Note: Standard errors in parentheses. * p<0.1, ** p<0.05, *** p<0.01. Standard errors are clustered by cohort and firm.

Commercialization can develop gradually even after a firm begins generating revenue. We therefore complement the stage comparison with direct measures of how the software asset changes around financing. Panel A of Table 16 uses developers' descriptions of code changes (commit messages) to track references to billing, payments, subscriptions, trials, and upgrades. Panel B measures changes to files whose names or folder locations refer to commercial features, payment processing, enterprise features, or growth, while Panel C records the scale and breadth of development.

Table 16: Commercialization Measures in the Software Asset

Measurement	Dependent Variable	Treated × Post	p-value
Panel A: Commit-message language			
	Any commercialization term (pp)	0.70	0.22
		(0.57)	
	Billing / payments / metering (pp)	0.03	0.67
		(0.08)	
	Subscription / trial / upgrade (pp)	0.54	0.33
		(0.55)	
Panel B: Names and locations of changed files			
	Commercial features (pp)	−0.04	0.22
		(0.03)	
	Payment-processing software (pp)	−0.01	0.86
		(0.05)	
	Enterprise / service-level agreements (pp)	0.13	0.32
		(0.13)	
	Growth / expansion (pp)	0.16	0.40
		(0.19)	
Panel C: Development activity			
	Log file changes per month	0.30***	<0.01
		(0.08)	
	Top-level folders with file changes	0.71**	0.01
		(0.29)	
	User-facing interface share of changed files (pp)	1.91**	0.03
		(0.89)	

This table estimates the baseline specification of Equation (7) on contribution-based outcomes for main-sample firms, pooling each firm's covered repositories. Each Panel A outcome is the fraction of a firm's commits in a month carrying the stated marker, computed from 5.6 million commit messages; these regressions use 300,380 observations. Panels B and C use file-change histories from 482 repositories. Panel B measures the fraction of changed files whose names or folder locations match each category. Panel C measures log(file changes), the number of top-level project folders with changed files (including the main project folder), and the fraction of changed files in the user-facing interface. For each repository, we take the natural logarithm of its positive monthly file-change count, then average these values across the firm's repositories within each month. The Panel B and C regressions use 302,768 observations. Coefficients and standard errors for share outcomes, marked pp, are reported in percentage points.

Note: Standard errors in parentheses. * p<0.1, ** p<0.05, *** p<0.01. Standard errors are clustered by cohort and firm.

The monetization markers show no statistically significant shift after financing, while development activity expands substantially. The growth rate in monthly file changes is approximately 30 percentage points greater than in controls ($p < 0.01$). Developers change files in 0.71 additional top-level project folders, and the share of changed files in the user-facing interface rises by 1.9 percentage points ($p = 0.01$ and $p = 0.03$, respectively). The increase in development therefore extends across the software asset without a detectable shift toward the measured monetization features.

Employment records show a similar expansion in engineering capacity. The sales-related share of headcount rises by 1.9 percentage points ($p < 0.01$), compared with 7.4 percentage points for software engineering ($p < 0.01$). Firms build commercial capacity, but the larger compositional shift is toward engineering. Appendix K reports the employment decomposition and additional development diagnostics.

The stage, codebase, and employment tests do not support a narrow lifecycle explanation in which quality improves because firms make the initial transition to selling a product. The gains are present among firms already generating revenue and accompany broad development activity, with a larger employment shift toward engineering than toward sales. These findings complement the process evidence by showing that the engineering improvement extends beyond the firms and activities most directly associated with the onset of commercialization.

4.4 Additional Robustness Tests

We next examine whether the quality improvement depends on the normalization of engineering outcomes, the months surrounding the financing event, or particular firms and cohorts in the stacked sample. These tests address distinct features of the design.

Density versus levels. Violations per line can fall because a codebase grows, not only because violations decline. We therefore study firms in the bottom quartile of pre-financing code growth (Appendix H). After financing, their code volume grows no faster than that of controls; the difference, -0.02 log points, is not statistically significant ($p = 0.73$). Their critical violations nevertheless decline by 2.85 per 1,000 lines ($p = 0.06$), more than the 1.85 decline in the full sample. Appendix G connects the density results to changes in issue counts and total technical debt.

Excluding months around financing. To check whether the gains depend on changes concentrated around the deal, we reestimate the baseline after excluding the financing month and either one or two months on each side. Quality improvements persist beyond the months immediately surrounding financing (Table 42). Under both exclusions, critical violations, total violations, and technical debt decline (largest $p = 0.04$), with estimates modestly larger in magnitude than the baseline.

Observation coverage and repository composition. We next examine whether changes in observation coverage or repository composition account for the quality gains. One comparison requires observations in every month of the 25-month event window. Another follows the pre-financing repositories, retaining only months when all remain observed. We also hold each pre-financing repository's share of the firm's lines at its value in the month before financing, while allowing quality within repositories to change. Critical violations and technical debt decline under each comparison (Tables 45 and 47). The declines also persist in comparisons of pre- and post-financing averages on complete windows. When we exclude firms with more than five scanned repositories, critical violations decline by 1.87 per 1,000 lines and technical debt by $1.63 per line ($p = 0.03$ and $p = 0.04$, respectively).

Alternative event windows. To check whether the findings depend on the observation period, we replace the baseline of 12 months on each side of financing with 6, 9, 18, or 24 months. Critical violations, total violations, and technical debt decline in every window (largest $p = 0.06$; Table 42).

Individual financing cohorts. To check whether a particular financing cohort drives the results, we reestimate the baseline 99 times, omitting one cohort each time. Critical violations, total violations, and technical debt decline in every case (Table 42).

Repeated use of control firms. The same firm can serve as a control for several financing cohorts. To check whether repeated use drives the findings, we weight control observations inversely by the number of cohorts in which each firm appears. Critical violations, total violations, and technical debt decline under this weighting (largest $p = 0.04$; Table 42).

Programming-language exclusions. To check whether a single programming language accounts for the gains, we successively exclude firms whose primary language is Go, JavaScript, C/C++, Java, or Python—the five most common languages among treated firms. The estimated changes in critical violations, total violations, and technical debt remain negative in every remaining sample.

5 Conclusion

Venture capital professionalization reaches the engineering practices through which startups build their products. Using longitudinal codebase data on 569 open-core technology firms, we find that critical violations fall by 1.85 per 1,000 lines and technical debt falls by $1.57 per line after first VC entry, approximately 15% and 12% of their treated pre-financing means. These improvements accompany substantial expansion of codebases and engineering teams. The critical-violation decline persists when we compare firms with similar prior growth, require complete observation windows, and hold the covered repositories fixed. Quality gains also extend to firms already generating revenue.

The contribution histories show how this change reaches the existing organization. Teams more frequently document copyright and licensing assurances, collaboration, and references to numbered development records, and they make a smaller share of contributions on weekends. Among engineers already active before financing, the probability of introducing a bug falls by 10.6% of its pre-financing mean, and improvements within the same engineers account for nearly three-fifths of this decline. The quality improvement is larger around first entry than around later financing rounds, placing these changes at the beginning of the VC relationship. The decline in critical violations is also larger in rounds with fewer co-investors, where each investor captures more of the benefit of monitoring, and we find no evidence of larger gains among firms that raised larger rounds or hired senior technical talent. Together, the codebase and contribution records connect improved engineering quality to the procedures through which existing engineers build and maintain software.

The open-core setting bears on the external validity of these estimates. Relative to closed-source startups, open-core firms expose substantial portions of their codebase (and often issue trackers) to public scrutiny, which can create baseline quality discipline through reputational concerns, external bug reporting, and community review. If this community discipline substitutes for some of the monitoring and process improvements that VCs bring, the marginal effect of VC entry on engineering quality could be attenuated in our setting and potentially larger in proprietary environments.

Our dollar-denominated measure of technical debt makes the estimated maintenance liability embedded in software comparable with other business costs. Combining this measure with development histories provides a way to study how firms accumulate and manage software liabilities alongside intangible assets. The findings connect VC value creation to the maintenance of a technology firm's core product, with applications to software-intensive firm valuation and mergers and acquisitions due diligence.

References

Amornsiripanitch, N., P. A. Gompers, and Y. Xuan. 2019. More than money: Venture capitalists on boards. *The Journal of Law, Economics, and Organization* 35:513–43.

Avgeriou, P., D. Taibi, A. Ampatzoglou, F. Arcelli Fontana, T. Besker, A. Chatzigeorgiou, V. Lenarduzzi, A. Martini, A. Moschou, I. Pigazzini, N. Saarimäki, D. Sas, S. Soares de Toledo, and A. Tsintzira. 2021. An overview and comparison of technical debt measurement tools. *IEEE Software* 38:61–71.

Banker, R., Y. Liang, and N. Ramasubbu. 2021. Technical debt and firm performance. *Management Science* 67:3174–94.

Bernstein, S., X. Giroud, and R. R. Townsend. 2016. The impact of venture capital monitoring. *The Journal of Finance* 71:1591–622.

Bernstein, S., and A. Sheen. 2016. The operational consequences of private equity buyouts: Evidence from the restaurant industry. *The Review of Financial Studies* 29:2387–418.

Callaway, B., and P. H. Sant'Anna. 2021. Difference-in-differences with multiple time periods. *Journal of Econometrics* 225:200–30. Themed Issue: Treatment Effect 1.

Casamatta, C. 2003. Financing and advising: Optimal financial contracts with venture capitalists. *The Journal of Finance* 58:2059–85.

Casamatta, C., and C. Haritchabalet. 2007. Experience, screening and syndication in venture capital investments. *Journal of Financial Intermediation* 16:368–98.

Chemmanur, T. J., K. Krishnan, and D. K. Nandy. 2011. How does venture capital financing improve efficiency in private firms? A look beneath the surface. *The Review of Financial Studies* 24:4037–90.

Dapr Authors. 2026. Contributing to dapr. Accessed October 9, 2026.

Eisfeldt, A. L., and D. Papanikolaou. 2013. Organization capital and the cross-section of expected returns. *The Journal of Finance* 68:1365–406.

Ewens, M., and M. Marx. 2018. Founder replacement and startup performance. *The Review of Financial Studies* 31:1532–65.

Fershtman, C., and N. Gandal. 2011. Direct and indirect knowledge spillovers: the "social network" of open-source projects. *The RAND Journal of Economics* 42:70–91.

Gompers, P. A. 1995. Optimal investment, monitoring, and the staging of venture capital. *The Journal of Finance* 50:1461–89.

Gompers, P. A., W. Gornall, S. N. Kaplan, and I. A. Strebulaev. 2020. How do venture capitalists make decisions? *Journal of Financial Economics* 135:169–90.

Gorman, M., and W. A. Sahlman. 1989. What do venture capitalists do? *Journal of Business Venturing* 4:231–48.

Gormley, T. A., and D. A. Matsa. 2011. Growing out of trouble? corporate responses to liability risk. *The Review of Financial Studies* 24:2781–821.

Hellmann, T., and M. Puri. 2002. Venture capital and the professionalization of start-up firms: Empirical evidence. *The Journal of Finance* 57:169–97.

Hsu, D. H. 2004. What do entrepreneurs pay for venture capital affiliation? *The Journal of Finance* 59:1805–44.

Kang, J.-K., Y. Li, and S. Oh. 2022. Venture capital coordination in syndicates, corporate monitoring, and firm performance. *Journal of Financial Intermediation* 50:100948.

Kaplan, S. N., and P. Strömberg. 2003. Financial contracting theory meets the real world: An empirical analysis of venture capital contracts. *The Review of Economic Studies* 70:281–315.

Krasner, H. 2022. Cost of poor software quality in the U.S. : A 2022 report. Working Paper, Consortium for Information & Software Quality (CISQ). Accessed: March 25, 2025.

Lerner, J. 1994. The syndication of venture capital investments. *Financial Management* 23:16–27.

———. 1995. Venture capitalists and the oversight of private firms. *Journal of Finance* 50:301–18.

McKinsey & Company. 2020. Tech debt: Reclaiming tech equity. https://www.mckinsey.com/capabilities/mckinsey-digital/our-insights/tech-debt-reclaiming-tech-equity. Accessed: March 25, 2025.

———. 2023. Breaking technical debt's vicious cycle to modernize your business. https://www.mckinsey.com/capabilities/mckinsey-digital/our-insights/breaking-technical-debts-vicious-cycle-to-modernize-your-business. Accessed: March 25, 2025.

Nagle, F. 2019. Open source software and firm productivity. *Management Science* 65:1191–215.

Peters, R. H., and L. A. Taylor. 2017. Intangible capital and the investment-q relation. *Journal of Financial Economics* 123:251–72.

Sculley, D., G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, and M. Young. 2014. Machine learning: The high interest credit card of technical debt. In *SE4ML: Software Engineering for Machine Learning (NIPS 2014 Workshop)*.

Sculley, D., G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison. 2015. Hidden technical debt in machine learning systems. In *Proceedings of the 29th International Conference on Neural Information Processing Systems - Volume 2*, NIPS'15, 2503–2511. Cambridge, MA, USA: MIT Press.

Simonsohn, U., J. P. Simmons, and L. D. Nelson. 2020. Specification curve analysis. *Nature Human Behaviour* 4:1208–14.

Sun, L., and S. Abraham. 2021. Estimating dynamic treatment effects in event studies with heterogeneous treatment effects. *Journal of Econometrics* 225:175–99. Themed Issue: Treatment Effect 1.

The Linux Foundation. 2026. Developer certificate of origin (dco). LF Guidance: Policy and Best Practices. Accessed October 9, 2026.

Appendix

A Variable Definitions

Table 17: Data and Variable Definitions

Variable	Description
A. Dimension of the data	
Firms	Open-core technology companies that commercialize open-source repositories and for which we observe longitudinal code histories, funding events, and employment data. The universe comprises 649 unique firms identified through OSSRank, an analytics platform for the open-source ecosystem; 569 of them contribute covered codebase months.
Repositories	Core product repositories associated with each firm. The company–repository mapping contains 703 repositories hosted on GitHub or GitLab. A repository functions as the central storage location for the code underlying a firm's core product.
Month	Monthly observations covering the period 2010–2024.
Unit of observation	The underlying data contain one observation per firm–month. The estimation panel is a firm–cohort–month because a firm can appear in more than one cohort.
Firm–month aggregation	We sum lines of code, estimated repair time and issue counts by severity across each firm's repositories within a month. Ratio measures, including the Technical Debt Ratio, are recomputed from the summed components rather than averaged across repositories.
B. Dependent variables (Engineering quality metrics)	
Critical Violations	Number of SonarQube rule violations at the critical severity level per 1,000 lines of code in month t. These severe issues can threaten system stability, security, or future maintainability. SonarQube also has a small Blocker level above critical; neither this measure nor the high-severity measures under the alternative classification include it.

Continued on next page

Table 17 (continued)

Variable	Description
Violations (All)	Number of rule violations across all severity levels per 1,000 lines of code. SonarQube flags code patterns associated with reliability, security, and maintainability concerns.
Maintainability Violations (All)	Number of maintainability-related rule violations across all severities per 1,000 lines of code. These rules flag code patterns that may make software harder to understand, modify, or extend.
Maintainability Issues (High)	Number of maintainability issues classified as high severity by SonarQube per 1,000 lines of code.
Maintainability Issues (Low)	Number of maintainability issues classified as low severity by SonarQube per 1,000 lines of code.
kLoC	One thousand lines of source code. Issue densities use this unit; technical debt per LoC uses one line.
Lines of code (LoC)	Lines of source code summed across the firm's covered repositories in month t.
SonarQube Security Hotspots	Number of security-sensitive code segments flagged for human review per 1,000 lines of code. A hotspot is not a confirmed vulnerability.
Bugs	Number of code patterns classified by SonarQube as likely to cause incorrect or unexpected behavior, per 1,000 lines of code.
Reliability Issues (High)	Number of reliability issues classified as high severity by SonarQube per 1,000 lines of code. The classification indicates a risk of incorrect behavior, rather than an observed application failure.
Reliability Issues (Low)	Number of reliability issues classified as low severity by SonarQube per 1,000 lines of code.

Continued on next page

Table 17 (continued)

Variable	Description
Technical Debt per Line of Code	Dollar value of estimated repair time per line of code. Let w denote the wage per hour ($55/hour) and let the SQALE Index be SonarQube's estimated repair time for maintainability violations (in hours). Let ℓ denote the development time per line (in hours) and set $\ell = 0.5$ (30 minutes), SonarQube's default. We approximate redevelopment time as $\ell \times \text{LoC}$ hours and scale the raw SQALE Index by κ to obtain calibrated repair time. We choose κ so that median calibrated debt equals a target share φ of median replacement-cost software assets (we set $\varphi = 0.20$): $$\kappa = \varphi \times \frac{\text{Median}(\ell \times \text{LoC})}{\text{Median}(\text{SQALE Index})}.$$ We then define $$\text{Technical Debt}_{i,t} = \kappa \times \text{SQALE Index}_{i,t} \times w,$$ and normalize by lines of code: $$\text{Technical Debt per LoC}_{i,t} = \frac{\text{Technical Debt}_{i,t}}{\text{LoC}_{i,t}}.$$
Technical Debt Ratio	Calibrated Technical Debt divided by Software Assets, multiplied by 100, where Software Assets is the replacement cost of the codebase ($55 per hour times 30 minutes per line times lines of code). Equivalently, 100κ times raw repair hours divided by estimated redevelopment hours; the wage cancels. The ratio is a percentage of replacement cost, and changes are reported in percentage points. For the same observations with positive lines of code, it equals $40/11$ times Technical Debt per LoC; the two measures describe one outcome in two units.
Technical Debt (level)	Calibrated dollar value of estimated repair time in month t, defined as $\kappa \times \text{SQALE Index}_{i,t} \times w$.

Continued on next page

Table 17 (continued)

Variable	Description
κ (Technical-debt calibration)	Scaling constant applied to SonarQube's raw SQALE Index before repair time is converted into dollars. It is chosen so that median calibrated debt equals 20% of median replacement-cost software assets ($\varphi = 0.20$). The main-sample calibration is $\kappa = 68.60$ and is held fixed across specifications using that sample, including the paired comparison of SonarQube's traditional and alternative classification systems in Table 43.
C. Main explanatory variables	
$VC_{i,c,t}$ (Treated × Post)	Indicator equal to 1 for a treated firm in cohort c beginning in the month of its first completed Seed round, or first completed Series A round if no Seed is recorded. Rumored, failed or cancelled, announced or in-progress, upcoming, and postponed deals are excluded when assigning the event.
Treated	Indicator equal to 1 for a firm whose first qualifying financing defines cohort c, and 0 for that cohort's controls. A firm can serve as a control in an earlier cohort before entering its own treated cohort.
Post	Dummy equal to 1 for months at or after the first VC deal for treated firms (and the corresponding aligned event-time months for controls in each cohort), and 0 otherwise.
High Violations	Dummy equal to 1 if the firm's pre-treatment critical violation density (at $t = -1$) exceeds the median across baseline firm–cohort observations of firms treated in any cohort, and 0 otherwise. A firm can contribute baseline observations in more than one cohort.
Large Codebase	Indicator equal to one if log code volume at event month -1 exceeds the median across baseline firm–cohort observations of firms treated in any cohort. The comparison uses the same baseline convention as High Violations.

Continued on next page

Table 17 (continued)

Variable	Description
Large Round	Dummy equal to 1 if the VC deal size (first Seed or Series A round amount, in U.S. dollars) exceeds the treated-firm median, and 0 otherwise; treated firms with no recorded round amount are in the reference group. Used to compare quality changes across financing amounts.
Large Syndicate (median split)	Indicator equal to one if the first Seed or Series A treatment round has more than four recorded investors, and zero otherwise. Four is the median among treated firms with a recorded investor count; missing counts are coded zero.
Founder Technical Leadership	Dummy equal to 1 if any member of the founding team (identified via LinkedIn job titles containing "founder" or "co-founder" and active at the firm within six months of first VC financing) simultaneously held a technical leadership role (CTO, VP of Engineering, Head of Engineering, or Director of Engineering) at the same firm around the deal date. Captures technical leadership within the founding team around financing.
Prior Executive Experience	Dummy equal to 1 if any founder of the firm held a prior CEO, Chief, Founder, or President role at a different company before the first VC deal date (excluding assistant and vice president roles). Captures prior senior leadership experience within the founding team.
Series B+ placebo treatment	In the Series B+ placebo sample of mature firms, dummy equal to 1 in months at or after a Series B or later funding round (conditional on observing earlier Seed/Series A history), and 0 otherwise. Treated firms already have institutional backing; controls remain without a first Seed or Series A round through the end of the event window.

Continued on next page

Table 17 (continued)

Variable	Description
Already generating revenue	Dummy equal to 1 if the firm was already generating revenue when venture capital arrived, based on the PitchBook business status recorded at the financing deal that defines treatment (matched within ±3 months of the treatment date) and taking the value *Generating Revenue*, *Generating Revenue/Not Profitable*, or *Profitable*; 0 otherwise. Defined for treated firms.
Pre-commercial	Complement of *Already generating revenue* for treated firms: dummy equal to 1 if the firm was not yet generating revenue at the treatment deal, and 0 otherwise. Used to test whether the effect of VC entry differs by the firm's commercial stage at entry.
D. Other covariates and constructed variables	
Total firm headcount	Total number of employees associated with the firm based on LinkedIn data in month t.
Software engineer headcount	Number of employees with software-engineering roles (based on LinkedIn job titles) in month t.
Senior engineering hires	Number of new hires in the stated 12-month window whose LinkedIn job titles indicate senior engineering roles: senior engineer/developer, staff engineer, principal engineer, architect, engineering director/VP, engineering manager, tech lead, or CTO.
Senior hire ratio	Ratio of senior engineering hires to total new hires in the stated 12-month window. Missing for firms with no new hires in the window.
High Senior Ratio	Indicator equal to one if the ratio of senior engineering hires to all new hires over the first 12 calendar months after the cohort event exceeds the treated-firm median of $1/7$. The endpoint 12 months after the event is excluded. Missing hiring measures are excluded from the interaction regressions.

Continued on next page

Table 17 (continued)

Variable	Description
Hired technical leader	Indicator equal to one if the firm hires a new CTO, VP of Engineering or Head of Engineering during [event, event+12 months). Hiring is measured relative to the cohort event for both treated and control firms. Hiring requires a recorded start date and no position at the firm before the event.
Technical-leader hiring outcome	Indicator equal to one if a firm hires a CTO, VP of Engineering, Head or Director of Engineering, or VP of Technology during the pre-financing period $[-12,-1]$ or post-financing period $[0,11]$. The direct comparison retains firm–cohort pairs observed in every month of both 12-month periods. This outcome is defined separately in each period; the first-year hiring indicator above instead classifies firms over the post-event year.
Nontechnical-leader hiring outcome	Analogous period-specific indicator for hiring a nontechnical executive (chief executive, operating, financial, marketing, product, revenue, or customer officer, or a head, vice president, or director of a non-engineering function).
SWE Share	Nonmanagerial software engineers as a share of distinct new hires in a firm–cohort–quarter, based on LinkedIn role classifications. Missing when no hires are recorded. SWE denotes software engineering.
SWE Seniority	Mean LinkedIn seniority, on a scale from 1 to 7, among newly hired nonmanagerial software engineers in a firm–cohort–quarter. Missing when no classified engineering hires are recorded.
$X_{i,c,t}$	Additional regressors in the heterogeneity specifications, including Post and Treated $\times$ Post interactions with baseline quality, financing size, and other stated firm characteristics. The baseline regression omits $X_{i,c,t}$.
Log LoC	Natural logarithm of positive lines of code summed across covered repositories.
Log total headcount	Natural logarithm of positive total firm headcount.

Continued on next page

Table 17 (continued)

Variable	Description
Log software-engineering headcount	Natural logarithm of positive software-engineering headcount.
Log critical violations (count)	Natural logarithm of positive critical-violation counts.
Log total violations (count)	Natural logarithm of positive total-violation counts.
Log technical debt in dollars	Natural logarithm of positive technical debt in dollars.
Log maintainability violations	Natural logarithm of positive maintainability-violation counts.
Log file-change activity	Natural logarithm of positive file-change activity, transformed at the repository–month level before averaging within firm–month.
Pre-treatment headcount growth	Difference in log average headcount between months $t \in [-4, -2]$ and $t \in [-12, -11]$ relative to the VC event month for firm i in cohort c. Used in Mahalanobis-distance matching to select controls with similar observed pre-treatment growth: $$G_i^c = \ln\left(\frac{1}{3}\sum_{t=-4}^{-2} \text{Headcount}_{i,c,t}\right) - \ln\left(\frac{1}{2}\sum_{t=-12}^{-11} \text{Headcount}_{i,c,t}\right).$$
Pre-treatment SWE headcount growth	Analogous pre-treatment growth measure constructed using software-engineering headcount instead of total headcount over the same pre-treatment windows.
Pre-treatment software growth	Analogous pre-treatment growth measure constructed using lines of code (LoC) over the same pre-treatment windows.

Continued on next page

Table 17 (continued)

Variable	Description
Software Assets	Estimated replacement cost of the codebase in month t. Software Assets approximate the capital required to re-develop the codebase from scratch: $$\text{Software Assets}_{i,t} = w\ell\,\text{LoC}_{i,t},$$ where $\ell = 0.5$ hours per line and $w = \$55$ per hour.
Event-time indicators	Set of dummy variables for monthly or quarterly bins within months -12 through $+12$ relative to the VC event date. Used to estimate dynamic difference-in-differences (event-study) coefficients and assess pre-treatment outcome paths.
Cohort × Firm FE ($\chi_{c,i}$)	Firm-by-cohort fixed effects that absorb time-invariant firm characteristics within each stacked cohort.
Month × Cohort FE ($\delta_{c,t}$)	Month-by-cohort fixed effects that control for common time-varying shocks affecting all firms within a cohort at month t.
Cohort eligibility	After each sample restriction, cohorts containing only treated firms or only control firms are excluded because they provide no identifying variation.
E. Contribution-history outcomes	
Contribution documentation share	Fraction of a firm's monthly commits containing Reviewed-by, Signed-off-by, Acked-by, Approved-by or Co-authored-by. Component shares count each marker separately and can overlap.
Numbered-reference share	Fraction of a firm's monthly commits whose messages contain a numbered issue or task reference. Explicit pull-request URLs and external task-tracking identifiers are measured separately.
Headquarters-local weekend share	Fraction of a firm's monthly commits recorded on Saturday or Sunday after converting contribution timestamps to the local time zone of the company's reported headquarters, including date-specific daylight-saving rules.

Continued on next page

Table 17 (continued)

Variable	Description
Pre-financing engineer	Engineer first active before the firm's financing event; control engineers are classified relative to their cohort event. The classification is based on repository contribution histories.
Bug-introduction rate	Share of contributions introducing bugs. We identify these contributions by tracing later bug fixes back to earlier changes to the affected code. We exclude bot accounts and contributors who cannot be classified, keep the first recorded fix linked to each introducing commit, and count the commit when that fix comes 0–24 calendar months after it. Fixed-roster comparisons instead count a commit if any recorded fix qualifies. The commit-level weekend comparison in Table 39 uses a 730-day window. The rate is missing in months without contributions.
Existing-engineer decomposition	Additive decomposition of the change in the bug-introduction rate for engineers active before financing. The within-engineer component captures changes in individual rates; the composition component captures changes in their shares of contribution activity. Both components use the same firm–month observations as the estimated total change.
Post-entry newcomer gap	We subtract the bug-introduction rate for engineers active before financing from newcomers' rate. We compare this difference between treated and control firms in post-event months when both groups contributed, using cohort-by-calendar-month fixed effects. Newcomers first contribute at or after the relevant cohort event; no pre-event newcomer rate enters this comparison.

Note: Engineering-quality metrics are measured using SonarQube Community Edition, which provides a catalogue of more than 10,000 language-specific rules. Severity-specific maintainability and reliability measures use the alternative classification described in Appendix M.2; they are not breakdowns of Maintainability Violations (All) or Bugs, which use the traditional classification. Technical Debt uses the SQALE (Software Quality Assessment based on Lifecycle Expectations) methodology. The universe comprises 649 open-core technology firms identified through OSSRank. The company–repository mapping links 703 repositories to 609 firms, 569 of which contribute at least one covered codebase month. The stacked difference-in-differences sample contains 484 unique firms and 99 event cohorts. Among these firms, 274 appear as treated and 412 as controls; the counts overlap because a not-yet-treated firm can serve as a control before its own financing event. Outcomes are observed over $t \in [-12, +12]$ around the first qualifying financing.

B Language-Specific Analysis Configurations

To reconstruct engineering quality over time, we analyze historical source code using SonarQube Community Edition[7] and its language analyzers, supplemented by the community *sonar-cxx* plugin for C and C++. Historical repositories often lack working build environments or compiled dependencies. We use the following language-specific settings to reduce reliance on those artifacts and accommodate large codebases.

- **C/C++:** The *sonar-cxx* plugin parses source text without rebuilding each historical project. It constructs a representation of the code's syntax and measures features such as branching complexity and duplication.
- **Java:** We analyze source code when compiled class files and dependency libraries are unavailable. This limits checks that require compiled information.
- **JavaScript/TypeScript:** The analysis disables bundle detection and the HTML analyzer.
- **Rust:** We analyze source code without the optional Clippy checks, which require an external toolchain.

The scan checks code against thousands of language-specific rules that reflect widely used software-quality frameworks and industry practice (e.g., ISO/IEC 25010). Three common Java rule violations illustrate what the tool measures. (i) Commented-out code makes it harder to understand what the software actually does and whether the implementation is complete. (ii) Lingering `TODO` markers are placeholders for unfinished work that often persist and embed a hidden backlog inside the code. (iii) Overly complex functions (high cognitive complexity) are harder to read, test, and modify, raising the cost of future changes and increasing the risk of mistakes.[8]

[7] SonarQube Community Edition is the freely available, open-source distribution used for all of our scans.
[8] For additional examples of common Java issues highlighted by SonarSource, see SonarSource, "Top issues in Java projects".

C Stacked Difference-in-Differences

Design. The objective is to estimate how first institutional VC entry changes engineering quality when firms receive financing at different dates. Standard two-way fixed-effects regressions can use already-treated firms as controls, producing misleading estimates when treatment effects vary across cohorts or evolve over time. We therefore use the cohort-stacking approach in Gormley and Matsa (2011), constructing a separate comparison group that remains untreated throughout each cohort's event window. This appendix defines the cohort-event causal contrasts, states the assumptions under which they are identified, and relates them to the stacked DiD regression used in the paper.

Cohorts and comparison firms. Let E_i denote the calendar month of firm i's first completed Seed round, or its first completed Series A round if no Seed is recorded. Set $E_i = \infty$ for firms without a qualifying financing. We exclude firms with a later-stage financing but no observed Seed or Series A history because their first institutional round cannot be dated.

For cohort month c, define the treated and eligible control sets as

$$\mathcal{T}_c = \{i : E_i = c\}, \qquad \mathcal{J}_c = \{j : E_j > c + L\},$$

where $L = 12$ is the event-window half-width. Event time is $e = t - c$, with $e \in [-L, L]$; treatment begins at $e = 0$. The control set includes never-funded firms and firms funded after the window closes.

We stack the available observations for $\mathcal{T}_c \cup \mathcal{J}_c$ over the event window. At construction, a cohort must contain at least one treated observation at every observed event month and an eligible control group. Formally, let $\mathcal{E}_c$ denote the observed event months and let $n^0_{T,c,e}$ and $n^0_{C,c,e}$ count treated and control observations before estimation-sample filters. The construction rule retains

$$C^0_L = \left\{ c : \min_{e \in \mathcal{E}_c} n^0_{T,c,e} \geq 1, \quad \sum_{e \in \mathcal{E}_c} n^0_{C,c,e} \geq 1 \right\}.$$

After each estimation-sample restriction, we drop any cohort left without both treated and control firms; $C_L \subseteq C^0_L$ denotes the retained cohorts. The sample is unbalanced because each firm's codebase need not be observed in every month. A firm can also appear as a control in several earlier cohorts before entering its own treated cohort, which is why the stacked regression count exceeds the number of underlying firm–months.

Potential outcomes and identification. Let Y^g_{it} denote the outcome for firm i in calendar month t if its first financing occurs in month g, and let Y^∞_{it} denote the outcome without financing. The observed outcome is $Y_{it} = Y^{E_i}_{it}$. This notation assumes that a firm's outcome is unaffected by the financing of

other sample firms. The cohort-event average treatment effect on the treated is

$$\theta_{c,e} = \mathbb{E}\left[Y^{c}_{i,c+e} - Y^{\infty}_{i,c+e} \mid E_i = c\right].$$

A causal interpretation requires the following assumptions for the firms represented in each comparison.

Assumption PT (Parallel Trends). Absent financing, treated and control firms would have the same expected outcome change. For each post-treatment month $e \geq 0$ and pre-treatment reference month $\bar{e} < 0$,

$$\mathbb{E}\left[Y^{\infty}_{i,c+e} - Y^{\infty}_{i,c+\bar{e}} \mid E_i = c\right] = \mathbb{E}\left[Y^{\infty}_{i,c+e} - Y^{\infty}_{i,c+\bar{e}} \mid i \in \mathcal{J}_c\right].$$

Assumption NA (No Anticipation). Financing does not affect outcomes before the recorded event month:

$$Y^{g}_{it} = Y^{\infty}_{it} \quad \text{for } t < g.$$

We use the exclusions of months around financing in Appendix M.1 to examine sensitivity to observations adjacent to the recorded event month. The monthly comparisons in Table 52 omit month -4, allowing a separate estimate for month -1.

Under PT and NA, the population difference-in-differences contrast identifies $\theta_{c,e}$. To see this, write the observed change for treated firms and subtract the change for their cohort controls:

$$\begin{aligned}
\theta^{\text{DID}}_{c,e} &= \mathbb{E}[Y_{i,c+e} - Y_{i,c+\bar{e}} \mid E_i = c] - \mathbb{E}[Y_{i,c+e} - Y_{i,c+\bar{e}} \mid i \in \mathcal{J}_c] \\
&= \mathbb{E}[Y^{c}_{i,c+e} - Y^{\infty}_{i,c+\bar{e}} \mid E_i = c] - \mathbb{E}[Y^{\infty}_{i,c+e} - Y^{\infty}_{i,c+\bar{e}} \mid i \in \mathcal{J}_c] \\
&= \mathbb{E}[Y^{c}_{i,c+e} - Y^{\infty}_{i,c+e} \mid E_i = c] \\
&\quad + \mathbb{E}[Y^{\infty}_{i,c+e} - Y^{\infty}_{i,c+\bar{e}} \mid E_i = c] - \mathbb{E}[Y^{\infty}_{i,c+e} - Y^{\infty}_{i,c+\bar{e}} \mid i \in \mathcal{J}_c] \\
&= \theta_{c,e}.
\end{aligned}$$

The second line uses no anticipation and the fact that controls remain untreated throughout the window. The third adds and subtracts the treated firms' counterfactual post-financing outcome, separating their treatment effect from the difference in untreated changes. Parallel trends makes the latter difference zero. For the paired comparisons in Table 45, we apply PT to the firms observed at both the reference month and the comparison month.

Aggregation and regression weights. A treated-firm-weighted summary of the cohort-event effects would be

$$\tau_e = \sum_{c \in C_L} \omega_c \theta_{c,e}, \qquad \omega_c = \frac{|\mathcal{T}_c|}{\sum_{c' \in C_L} |\mathcal{T}_{c'}|}, \qquad \tau_{\text{post}} = \frac{1}{L+1} \sum_{e=0}^{L} \tau_e,$$

where C_L denotes the retained cohorts. This aggregation weights cohorts by treated-firm counts and post-financing months equally. The stacked least-squares specification below uses weights determined by the available comparisons.

In the stacked estimation sample, define $D_{ice} = \mathbf{1}\{E_i = c\}\mathbf{1}\{e \geq 0\}$. The baseline specification is

$$Y_{ice} = \beta D_{ice} + \alpha_{ic} + \lambda_{ce} + \varepsilon_{ice}.$$

Here Y_{ice} corresponds to $M_{i,c,t}$ in Equation (7), with $t = c + e$; D_{ice} corresponds to $VC_{i,c,t}$, and α_{ic} and λ_{ce} correspond to $\chi_{c,i}$ and $\delta_{c,t}$. The fixed effects absorb firm differences and common monthly shocks separately within each cohort. Let $\widetilde{D}_{ice}$ be the residual from projecting D_{ice} on these fixed effects, and let O denote the set of observations in the estimation sample. The estimated coefficient satisfies

$$\widehat{\beta} = \frac{\sum_{(i,c,e) \in O} \widetilde{D}_{ice} Y_{ice}}{\sum_{(i,c,e) \in O} \widetilde{D}_{ice}^2}.$$

Writing $\tau_{ice} = Y^c_{i,c+e} - Y^\infty_{i,c+e}$ for a treated post-financing observation and using NA gives the exact decomposition

$$\widehat{\beta} = \frac{\sum_O \widetilde{D}_{ice} Y^\infty_{i,c+e}}{\sum_O \widetilde{D}_{ice}^2} + \sum_{\substack{(i,c,e) \in O \\ D_{ice}=1}} w_{ice} \tau_{ice}, \qquad w_{ice} = \frac{\widetilde{D}_{ice}}{\sum_O \widetilde{D}_{ice}^2}.$$

The weights sum to one because the fixed-effect projection satisfies $\sum_O \widetilde{D}_{ice} D_{ice} = \sum_O \widetilde{D}_{ice}^2$. A sufficient conditional-mean condition for the expected untreated-outcome term to vanish is

$$\mathbb{E}\left[Y^\infty_{i,c+e} \mid O, \{E_j\}\right] = a_{ic} + \lambda_{ce} \quad \text{on the retained observations.}$$

This condition puts the expected untreated path, conditional on the complete observation history and financing dates, in the span of the fitted fixed effects. More generally, the required restriction is that its inner product with $\widetilde{D}$ is zero. It specifies the weighted comparison underlying the stacked DiD estimator, separately from PT for a population cohort-event contrast.

The available observations determine the weights in the DiD coefficient. With balanced cohorts and equal window lengths, cohort weights are proportional to $|\mathcal{T}_c||\mathcal{J}_c|/(|\mathcal{T}_c| + |\mathcal{J}_c|)$.

In our unbalanced panel, firms are observed in different sets of months. A treated firm's weight depends on its coverage before and after financing and on the coverage of its cohort's controls. Missing months can therefore shift influence across firms, cohorts and event months. The coefficient summarizes the available comparisons, rather than giving equal weight to every treated firm or every post-financing month.

In the unbalanced baseline estimation sample, 97 treated post-financing observation weights are negative; their total absolute mass is less than 0.01% of the net weight. The largest absolute observation weight is 0.06% of the net weight. Concentration across treated firms, measured as $10{,}000 \sum_i (\sum_{c,e:D_{ice}=1} w_{ice})^2$, is 41.3; this diagnostic sums each treated firm's post-financing effect weights within its own cohort before computing concentration. The index would equal 10,000 if one firm carried all the weight and 36.6 if the weight were spread equally over the 273 treated firms. The 487,577 observations follow removal of 18 fixed-effect singletons from the stacked panel. We report $\widehat{\beta}$ as the DiD coefficient and cluster standard errors by cohort and firm to account for cohort shocks and firms that appear in several cohorts.

For comparison, we also retain firm–cohort units observed in all 25 months and subtract each firm's pre-period mean from its post-period mean. Cohorts receive weights proportional to their retained treated-firm counts, and the same total weight is assigned to controls within each cohort. The resulting aggregate has nonnegative weights and fixes each firm's observation history. A second comparison differences each post observation from that firm's month -1 value, gives each post month equal weight, and fixes cohort weights at baseline treated-firm counts. Both preserve firm-and-cohort clustering. Table 45 reports these estimators and the firms each covers. Appendix M.1 reports sensitivity to control reuse.

Event-study implementation. The quarterly event study replaces D_{ice} with interactions between cohort treatment status and event-quarter indicators:

$$Y_{ice} = \sum_{q \in Q,\, q \neq 0} \beta_q \mathbf{1}\{E_i = c\}\mathbf{1}\{q(e) = q\} + \alpha_{ic} + \lambda_{ce} + \varepsilon_{ice},$$

where Q is the set of observed event-quarter bins. The omitted bin $q = 0$ contains months $\{-4, -3, -2\}$. The next bin, $q = 1$, contains months $\{-1, 0, 1\}$ and combines one pre-treatment month with two months at or after financing. It therefore averages over the onset of treatment; later bins are entirely post-treatment. The coefficients are normalized relative to the omitted pre-treatment bin and inherit the stacked regression's weighting. The plots and joint tests describe the fitted pre-treatment coefficients. For a lead $q < 0$, let r^q_{ice} be the residual from projecting its treated-quarter indicator on the fixed effects and all other event-quarter interactions. Under the conditional untreated-outcome restriction above

and no anticipation,

$$\mathbb{E}[\widehat{\beta}_q \mid O, \{E_j\}] = \frac{\sum_{O:D_{ice}=1} r^q_{ice}\, \mathbb{E}[\tau_{ice} \mid O, \{E_j\}]}{\sum_O (r^q_{ice})^2}.$$

Zero-centered leads therefore also require this weighted post-treatment-effect term to vanish. A sufficient restriction is that the post-treatment effect vector lies in the span of the other included event interactions and fixed effects. With unbalanced coverage, heterogeneous responses can enter the fitted leads through this projection.

D Matching on Pre-Treatment Growth and Quality Trends

We match within financing cohorts on growth measured before the round. For firm i in cohort c, let $H_{i,c,e}$ denote headcount at event month e. The matching feature is

$$G^c_i = \ln\left(\frac{H_{i,c,-4} + H_{i,c,-3} + H_{i,c,-2}}{3}\right) - \ln\left(\frac{H_{i,c,-12} + H_{i,c,-11}}{2}\right).$$

This feature compares average headcount near the beginning and end of the pre-financing period, excluding the month immediately before the round to reduce sensitivity to deal-adjacent hiring. Sensitivity specifications add analogously constructed growth in software-engineering headcount and lines of code, making G^c_i a vector.

Using the notation of Appendix C, the candidate controls satisfy $E_j > c + 12$. We require an observation at $e = -1$ and nonmissing matching features for both treated firms and candidate controls. This eligibility step retains 112 treated firms and 206 candidate controls in 70 cohorts. The distance cutoff and the subsequent requirement that each cohort retain both treated and control firms leave 110 treated firms and 196 controls in 68 cohorts; the preferred matched regression has 71,837 observations. Table 51 compares the eligible and final treated populations before and after selecting controls. Let $\bar{G}^{\mathcal{T}}_c$ be the mean feature vector among eligible treated firms. In the primary matched specification, the distance between a candidate control and this treated-group mean is

$$d_c(j) = \sqrt{(G^c_j - \bar{G}^{\mathcal{T}}_c)' \widehat{\Sigma}^{-1}_c (G^c_j - \bar{G}^{\mathcal{T}}_c)},$$

where $\widehat{\Sigma}_c$ is the sample covariance of the candidate-control vectors together with the treated-group mean. With headcount growth as the sole feature, this is the absolute growth difference divided by the corresponding sample standard deviation.

We retain controls with $d_c(j) \leq 1$, ranked by distance, and keep the available event-window observations for eligible treated firms and selected controls. Each control enters at most once within a cohort but can appear in several cohorts, and we discard any cohort left without both groups. Matching

to the treated-group mean retains more eligible controls than individual one-to-one matching. In the primary matched estimation sample, the mean treated-minus-control headcount-growth gap is 0.06 log points, averaging within-cohort gaps with weights proportional to each cohort's treated-firm count in that sample. The distance cutoff therefore narrows the comparison without imposing equality of the two groups' mean growth.

Appendix E examines whether this result depends on the particular definition of a comparable firm. We vary the distance cutoff from 0.5 to 2.0, add growth in code volume and software-engineering headcount, and replace matching to the treated-group mean with individual nearest-neighbor matching at $K \in \{1, 3\}$, selecting controls sequentially without replacement within a cohort. These alternatives change both the dimensions along which firms are matched and how tightly their trajectories must align, allowing us to assess whether the quality improvement persists across plausible comparison groups.

Scale under the Primary Growth Match

Figure 13 in Section 4.1 shows the event-study paths of firm scale under the primary match. The DiD coefficients likewise show continued expansion after financing. In the matched sample, total-headcount growth is approximately 44 percentage points greater than in controls, and software-engineering-headcount growth is approximately 42 percentage points greater ($p < 0.01$ for each; Table 18). Thus, the post-financing expansion persists when controls are selected for similar prior headcount growth.

Table 18: Difference-in-Differences Estimates for Firm Scaling (Matched Sample)

	Log(Total Headcount)			**Log(SWE Headcount)**		
	(1)	(2)	(3)	(4)	(5)	(6)
Treated × Post	0.44***	0.46***	0.54***	0.42***	0.46***	0.43***
	(0.05)	(0.07)	(0.08)	(0.06)	(0.08)	(0.07)
Post × High Violations (=1)		0.00			0.03	
		(0.02)			(0.02)	
Treated × Post × High Violations (=1)		−0.04			−0.08	
		(0.10)			(0.11)	
Post × Large Round			0.15***			0.10*
			(0.04)			(0.06)
Treated × Post × Large Round			−0.33***			−0.08
			(0.12)			(0.12)
Cohort × Firm FE	Yes	Yes	Yes	Yes	Yes	Yes
Month × Cohort FE	Yes	Yes	Yes	Yes	Yes	Yes
# Obs	68,042	68,042	68,042	68,042	68,042	68,042
R^2	> 0.99	> 0.99	> 0.99	> 0.99	> 0.99	> 0.99

This table reports the coefficient β from the specification $M_{i,c,t} = \beta VC_{i,c,t} + \chi_{c,i} + \delta_{c,t} + \gamma X_{i,c,t} + \varepsilon_{i,c,t}$, where $VC_{i,c,t}$ is the Treated × Post indicator. The outcomes are log(Headcount) and log(SWE), using positive values of each measure. The sample is restricted to treated firms and control firms matched on pre-treatment headcount growth. The matching procedure is described above.
See Table 3 for the definitions of the High Violations and Large Round dummies.
Note: Standard errors in parentheses. * p<0.1, ** p<0.05, *** p<0.01. Standard errors are clustered by cohort and firm.

Matching on Engineering-Headcount Growth

We also match on pre-financing engineering-headcount growth alone, using the same within-cohort procedure, pre-treatment windows and distance cutoff as the primary match. Figure 16 compares engineering-headcount trajectories under matching on total-headcount growth (Panel A) and engineering-headcount growth (Panel B). By the end of the event window, the growth rate in engineering headcount is approximately 51 percentage points greater than in controls under the primary match and 53 percentage points greater under the engineering-growth match ($p < 0.01$ for each).

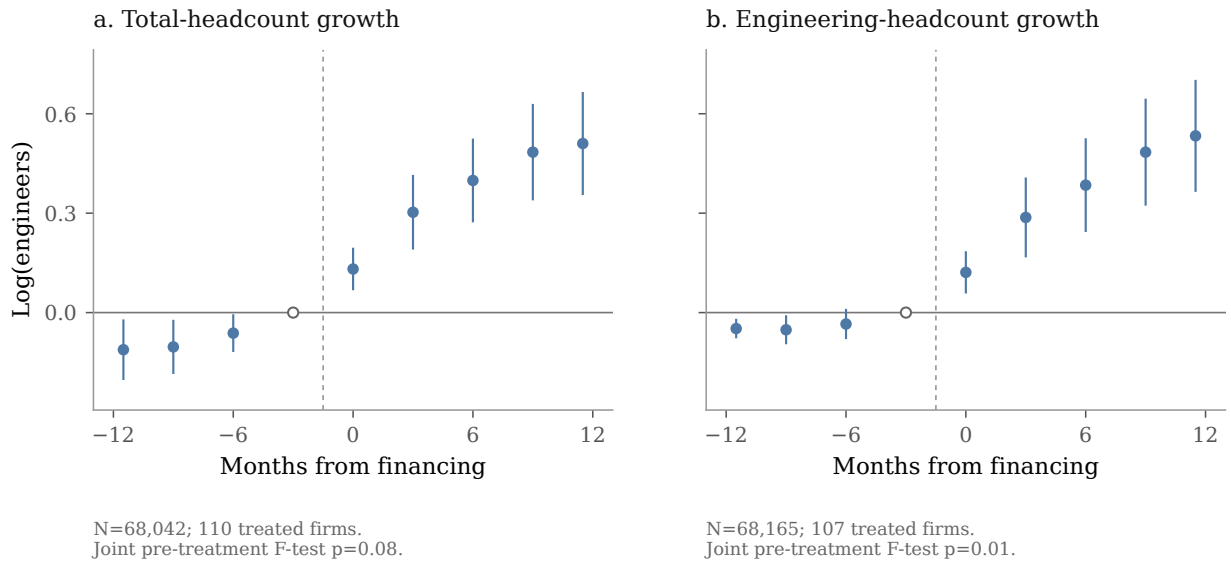


Figure 16: Engineering Headcount under Alternative Growth Matches. The outcome is the natural logarithm of positive software-engineering headcount. Panel A uses the primary match on total-headcount growth (110 treated firms in 68 cohorts; 68,042 stacked observations); Panel B matches on engineering-headcount growth (107 treated firms in 67 cohorts; 68,165 stacked observations). Joint tests of the three pre-treatment coefficients give $F(3, 67) = 2.31$ ($p = 0.08$) and $F(3, 66) = 3.93$ ($p = 0.01$), respectively. Both panels use the same axis scales. Controls are selected within financing cohorts using growth between months $[-12, -11]$ and $[-4, -2]$, group-mean matching and a Mahalanobis distance cutoff of one. Regressions include firm-by-cohort and month-by-cohort fixed effects. Quarterly estimates are relative to months -4 to -2; the bin plotted at month 0 spans months -1 to $+1$. The dashed line marks the end of the reference months. Bars show 95% confidence intervals; standard errors are clustered by cohort and firm.

Matching on Pre-Financing Violation-Density Changes

In the baseline comparison, the early pre-financing estimates for total violation density are noisy. We therefore examine whether the post-financing decline persists after matching controls on the pre-financing change in total violation density, measured as violations per 1,000 lines of code without taking logs. The matching feature is the mean density in months $[-4, -2]$ minus its mean in months $[-12, -11]$, requiring all five observations and an observation at month -1. Controls are selected once within each financing cohort using the treated-group mean and a Mahalanobis distance cutoff of one. Figure 17 compares the baseline event study with this additional match. After matching, the pre-financing estimates are closer to zero and the post-financing decline persists.

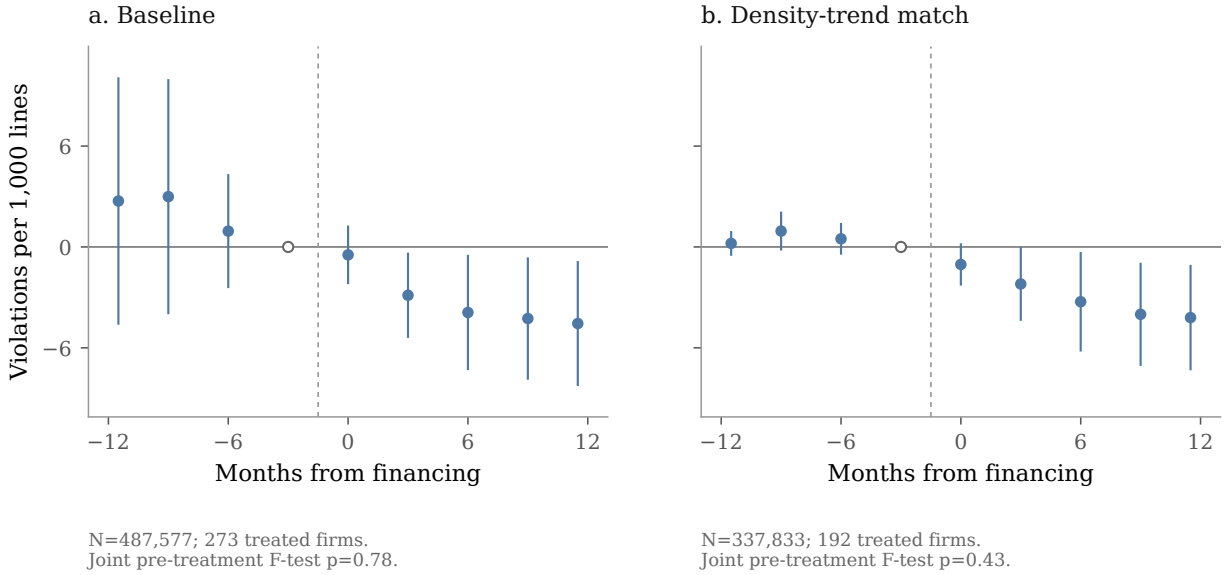


Figure 17: Total Violation Density under Pre-Financing Trend Matching. The outcome is total violations per 1,000 lines of code, without logs. Panel A reports the baseline (273 treated firms in 99 cohorts; 487,577 stacked observations). Panel B matches on the pre-financing density change (192 treated firms in 89 cohorts; 337,833 stacked observations). The panels use identical axis scales. Joint tests of the three pre-treatment coefficients give $F(3, 98) = 0.35$ ($p = 0.78$) and $F(3, 88) = 0.91$ ($p = 0.43$), respectively. Neither joint test is statistically significant. Regressions include firm-by-cohort and month-by-cohort fixed effects. Quarterly estimates are relative to months -4 to -2; the bin plotted at month 0 spans months -1 to $+1$. The dashed line marks the end of the reference months. Bars show 95% confidence intervals; standard errors are clustered by cohort and firm.

Matching on Pre-Financing Maintainability-Density Changes

We match on the change in average maintainability violations per 1,000 lines of code between months $[-12, -11]$ and months $[-4, -2]$, under the traditional classification used in Figure 3. We require all five monthly observations and an observation at month -1. Within each financing cohort, we select controls within a Mahalanobis distance of one from the treated-group mean and keep membership fixed throughout the event window. The data requirements leave 195 eligible treated firms; two more are excluded because their cohorts have no controls within the cutoff. The resulting sample contains 193 treated firms in 90 cohorts. Figure 18 compares the baseline and matched event studies on identical axes.

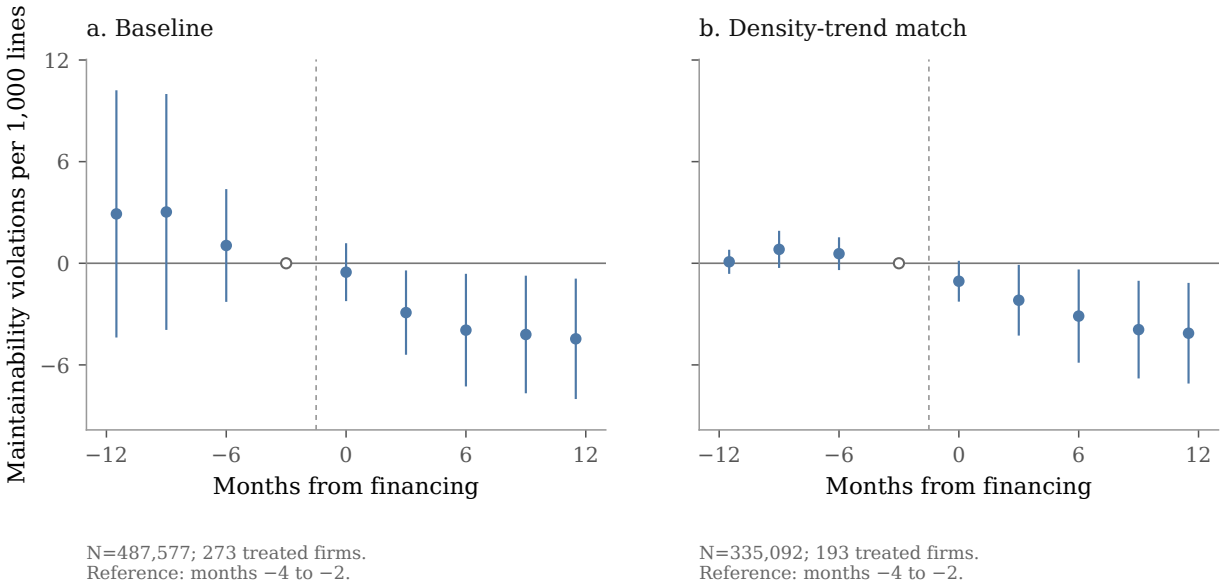


Figure 18: Maintainability Density under Pre-Financing Trend Matching. The outcome is maintainability violations per 1,000 lines of code under the traditional classification, without logs. Panel A reports the baseline (273 treated firms in 99 cohorts; 487,577 stacked observations). Panel B matches on the pre-financing density change (193 treated firms in 90 cohorts; 335,092 stacked observations). The panels use identical axis scales. The open marker denotes the omitted months −4 to −2, plotted at month −3. Regressions include firm-by-cohort and month-by-cohort fixed effects. Quarterly estimates are relative to months −4 to −2; the bin plotted at month 0 spans months −1 to +1. The dashed line marks the end of the reference months. Bars show 95% confidence intervals; standard errors are clustered by cohort and firm.

E Matching Sensitivity Analysis

We examine whether the matched engineering-quality estimates depend on how controls are selected. We construct a specification curve (Simonsohn, Simmons, and Nelson, 2020) over 72 configurations that vary three choices. First, the matching features include growth in total headcount alone, headcount plus lines of code, headcount plus software-engineering headcount, or all three measures. Second, we match controls either to the cohort's treated-group mean (centroid matching) or to individual treated firms, selecting one or up to three nearest neighbors. Third, the maximum Mahalanobis distance, or caliper, ranges from 0.5 to 2.0 ($\delta \in \{0.5, 0.75, 1.0, 1.25, 1.5, 2.0\}$). Within a cohort, each control is used at most once; it can appear in different cohorts. We re-estimate Equation (7) on the matched sample for each configuration.

Figures 19–22 present the specification curves for our four core engineering-quality outcomes. In each figure, the top panel plots the point estimate and 95% confidence interval for every specification, grouped by feature set, matching mode, and caliper in the same order across outcomes. The dashed

gray line marks the baseline (unmatched) estimate and its 95% confidence band (gray shading); the solid gray line marks zero. The orange diamond identifies our primary matched specification (centroid matching on headcount growth, caliper $\delta = 1$). The bottom panel displays an indicator matrix identifying the feature set, matching mode, and caliper active in each specification.

The direction of the quality estimates is stable across matching choices. Every critical-violation, total-violation, and maintainability-violation coefficient is negative, as are 68 of the 72 technical-debt coefficients. At the 5% level, 26% of critical-violation specifications, 61% of total-violation specifications, and 61% of maintainability-violation specifications reject the null; the corresponding shares at the 10% level are 85%, 74%, and 79%.

Matching substantially reduces the sample: the primary specification retains approximately 72,000 of 488,000 baseline observations, and some individual nearest-neighbor configurations retain fewer than 10,000. Despite this contraction, the median matched-to-baseline coefficient ratio is 1.38 for critical violations, 0.90 for total violations, and 0.85 for maintainability violations. The stability of these magnitudes, alongside wider confidence intervals, indicates that reduced statistical power contributes to the loss of significance. By contrast, technical debt is less stable, with a median ratio of 0.63.

Eligibility varies with the matching features. To distinguish this population change from control selection, we also retain the same 37 treated firms in 21 cohorts across all 72 configurations, require complete 25-month observations for both treated and control firms, and fix cohort weights at treated-firm counts. For each firm we compare the average of months 0–12 with that of months −12 to −1. Critical-violation estimates are negative in all 72 comparisons, ranging from −4.6 to −0.5 per 1,000 lines. Total-violation estimates range from −16.7 to +3.0 (46 negative), maintainability from −14.9 to +1.8 (59 negative), and debt from −7.10 to +1.09 dollars per line (20 negative). This fixed-population comparison uses nonnegative aggregation weights and a smaller treated population than the stacked DiD specification curves above.

Table 19 reports selected treated and control firms, cohorts and monthly observations for every configuration, distinguishing these membership counts from the observations that remain after singleton deletion.

Table 19: Sample Sizes across the 72 Matching Configurations

Features	Matching	Caliper	Treated	Controls	Cohorts	Observations
Headcount + SWE	Group mean	0.50	65	141	41	13,330
Headcount + SWE	Group mean	0.75	79	161	49	23,850
Headcount + SWE	Group mean	1.00	96	181	61	35,229
Headcount + SWE	Group mean	1.25	102	191	63	46,891
Headcount + SWE	Group mean	1.50	103	193	64	56,775
Headcount + SWE	Group mean	2.00	107	196	66	82,131
Headcount + SWE	One neighbor	0.50	87	52	50	3,864
Headcount + SWE	One neighbor	0.75	92	57	54	4,239

Table continued

Features	Matching	Caliper	Treated	Controls	Cohorts	Observations
Headcount + SWE	One neighbor	1.00	104	69	64	4,885
Headcount + SWE	One neighbor	1.25	106	71	65	5,035
Headcount + SWE	One neighbor	1.50	107	71	66	5,134
Headcount + SWE	One neighbor	2.00	107	72	66	5,209
Headcount + SWE	Three neighbors	0.50	87	95	50	6,419
Headcount + SWE	Three neighbors	0.75	92	103	54	7,194
Headcount + SWE	Three neighbors	1.00	104	114	64	8,340
Headcount + SWE	Three neighbors	1.25	106	123	65	9,065
Headcount + SWE	Three neighbors	1.50	107	126	66	9,464
Headcount + SWE	Three neighbors	2.00	107	128	66	9,989
Headcount + code	Group mean	0.50	78	151	48	19,668
Headcount + code	Group mean	0.75	97	176	58	35,204
Headcount + code	Group mean	1.00	103	189	62	47,487
Headcount + code	Group mean	1.25	108	195	66	60,053
Headcount + code	Group mean	1.50	108	198	66	72,707
Headcount + code	Group mean	2.00	109	201	67	97,785
Headcount + code	One neighbor	0.50	98	62	56	4,477
Headcount + code	One neighbor	0.75	103	63	61	4,802
Headcount + code	One neighbor	1.00	105	65	63	4,977
Headcount + code	One neighbor	1.25	108	68	66	5,201
Headcount + code	One neighbor	1.50	108	71	66	5,276
Headcount + code	One neighbor	2.00	109	74	67	5,376
Headcount + code	Three neighbors	0.50	98	113	56	7,746
Headcount + code	Three neighbors	0.75	103	123	61	8,655
Headcount + code	Three neighbors	1.00	105	126	63	9,155
Headcount + code	Three neighbors	1.25	108	129	66	9,677
Headcount + code	Three neighbors	1.50	108	133	66	9,927
Headcount + code	Three neighbors	2.00	109	136	67	10,352
All three	Group mean	0.50	47	93	28	5,624
All three	Group mean	0.75	62	138	36	13,090
All three	Group mean	1.00	70	164	42	23,043
All three	Group mean	1.25	87	181	54	32,694
All three	Group mean	1.50	97	185	60	41,756
All three	Group mean	2.00	106	194	65	65,941
All three	One neighbor	0.50	66	40	36	2,736
All three	One neighbor	0.75	78	49	42	3,386
All three	One neighbor	1.00	88	55	49	3,907
All three	One neighbor	1.25	97	63	58	4,482
All three	One neighbor	1.50	101	66	62	4,756
All three	One neighbor	2.00	106	68	65	5,031
All three	Three neighbors	0.50	66	78	36	4,267
All three	Three neighbors	0.75	78	91	42	5,317
All three	Three neighbors	1.00	88	107	49	6,488
All three	Three neighbors	1.25	97	114	58	7,586
All three	Three neighbors	1.50	101	123	62	8,360
All three	Three neighbors	2.00	106	128	65	9,510
Headcount	Group mean	0.50	109	183	67	42,188
Headcount	Group mean	0.75	109	193	67	58,222
Headcount	Group mean	1.00	110	196	68	71,837
Headcount	Group mean	1.25	110	200	68	84,865

Table continued

Features	Matching	Caliper	Treated	Controls	Cohorts	Observations
Headcount	Group mean	1.50	110	201	68	97,090
Headcount	Group mean	2.00	110	202	68	117,394
Headcount	One neighbor	0.50	109	70	67	5,230
Headcount	One neighbor	0.75	109	71	67	5,255
Headcount	One neighbor	1.00	110	74	68	5,379
Headcount	One neighbor	1.25	110	74	68	5,404
Headcount	One neighbor	1.50	110	74	68	5,404
Headcount	One neighbor	2.00	110	74	68	5,429
Headcount	Three neighbors	0.50	109	123	67	9,962
Headcount	Three neighbors	0.75	109	123	67	10,112
Headcount	Three neighbors	1.00	110	126	68	10,336
Headcount	Three neighbors	1.25	110	126	68	10,411
Headcount	Three neighbors	1.50	110	126	68	10,436
Headcount	Three neighbors	2.00	110	127	68	10,636

Note: Counts describe the selected samples before fixed-effect singleton deletion, after matching and requiring treated and control firms in each cohort. Treated and control counts are unique firms by role and can overlap across cohorts. Observations are firm–cohort–months. Features are pre-event growth in total headcount, lines of code and software-engineering headcount (SWE). Group mean matches to the cohort's average treated profile; neighbor methods select one or up to three controls per treated firm without replacement within cohort. Calipers are Mahalanobis-distance units. The preferred configuration is headcount, group mean, caliper one, with 71,837 observations. The separate common-population comparison fixes 37 treated firms and 21 cohorts across configurations.

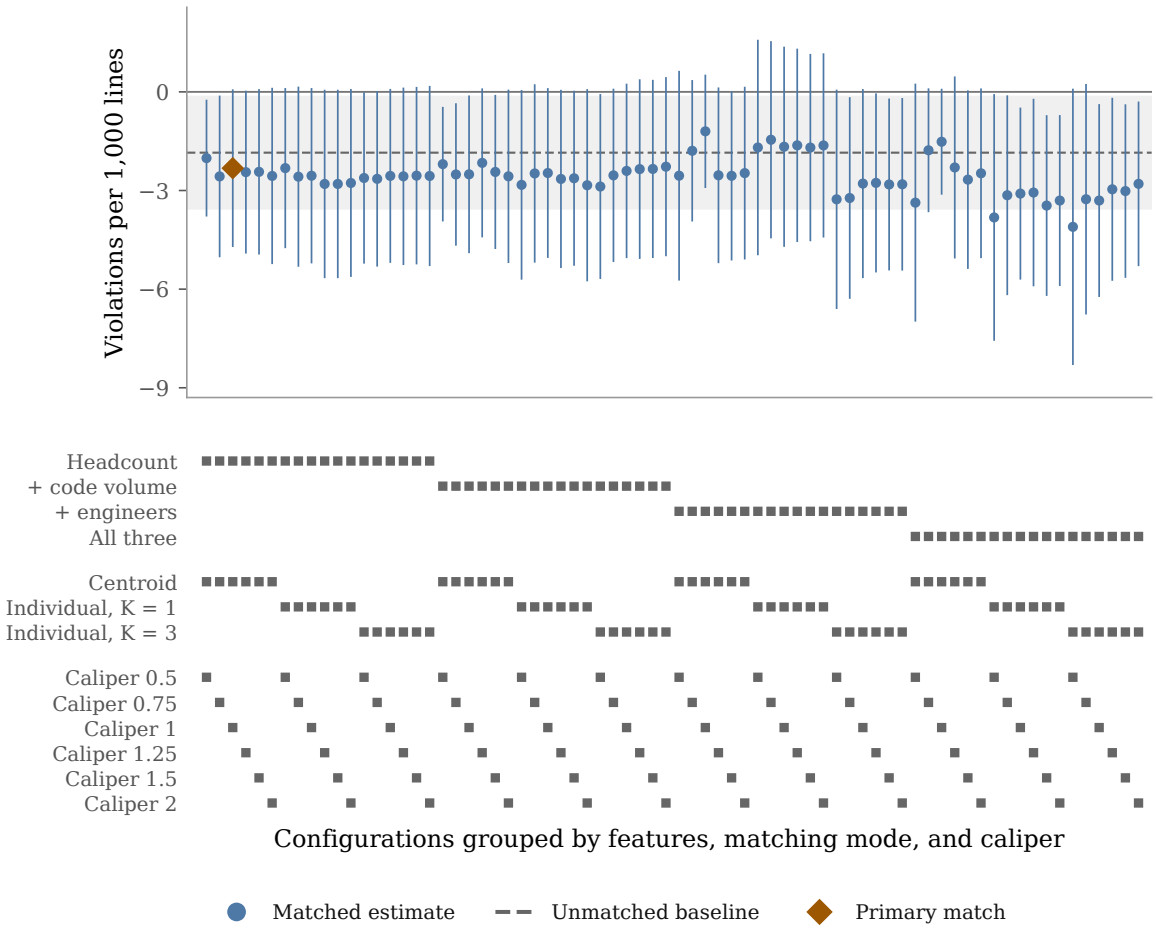


Figure 19: Specification Curve: Critical Violations per 1,000 LoC. Estimates across 72 matching configurations, identified in the lower panel. The dashed line and shaded band show the unmatched estimate and its interval; the orange diamond marks the preferred matched specification. Bars show 95% confidence intervals; standard errors are clustered by cohort and firm.

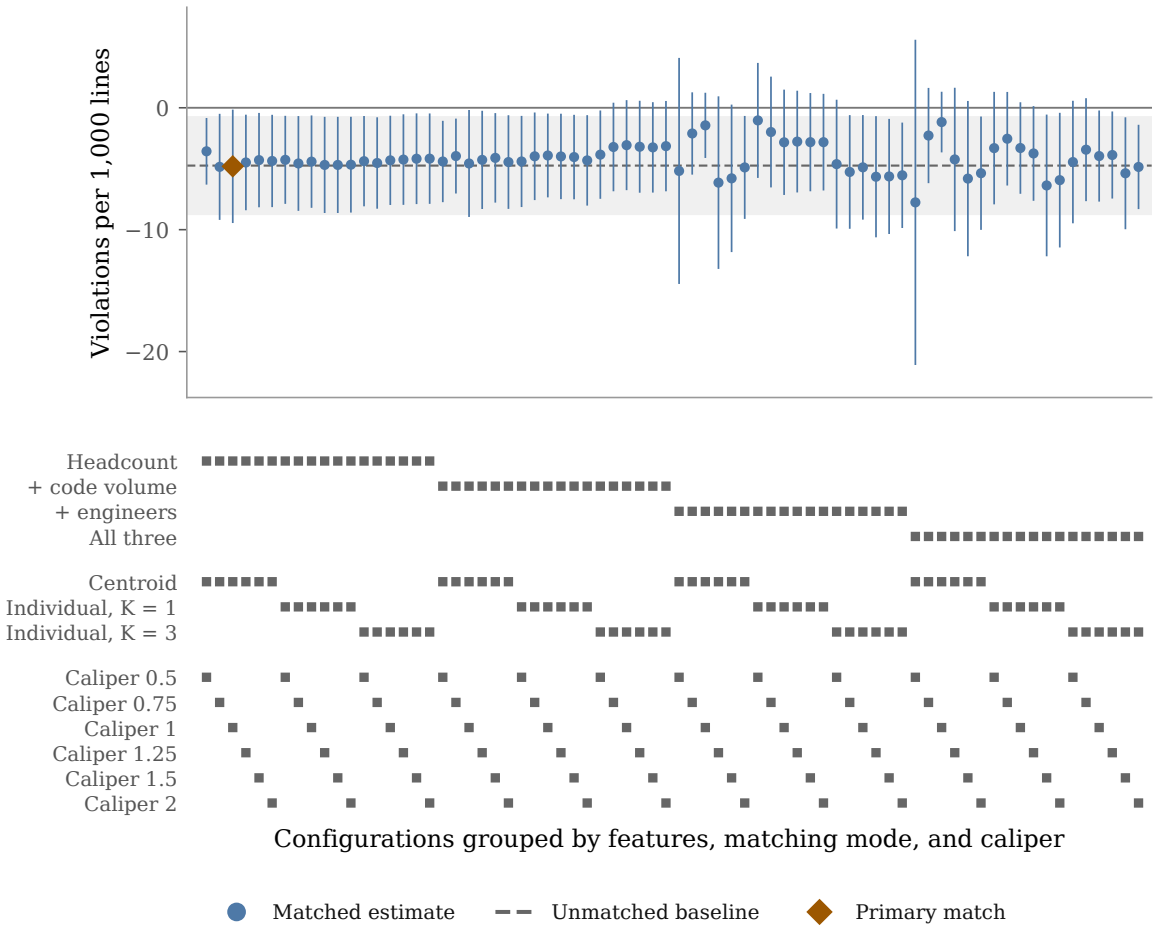


Figure 20: Specification Curve: Violations (All) per 1,000 LoC. See Figure 19 for a description of the layout. Bars show 95% confidence intervals; standard errors are clustered by cohort and firm.

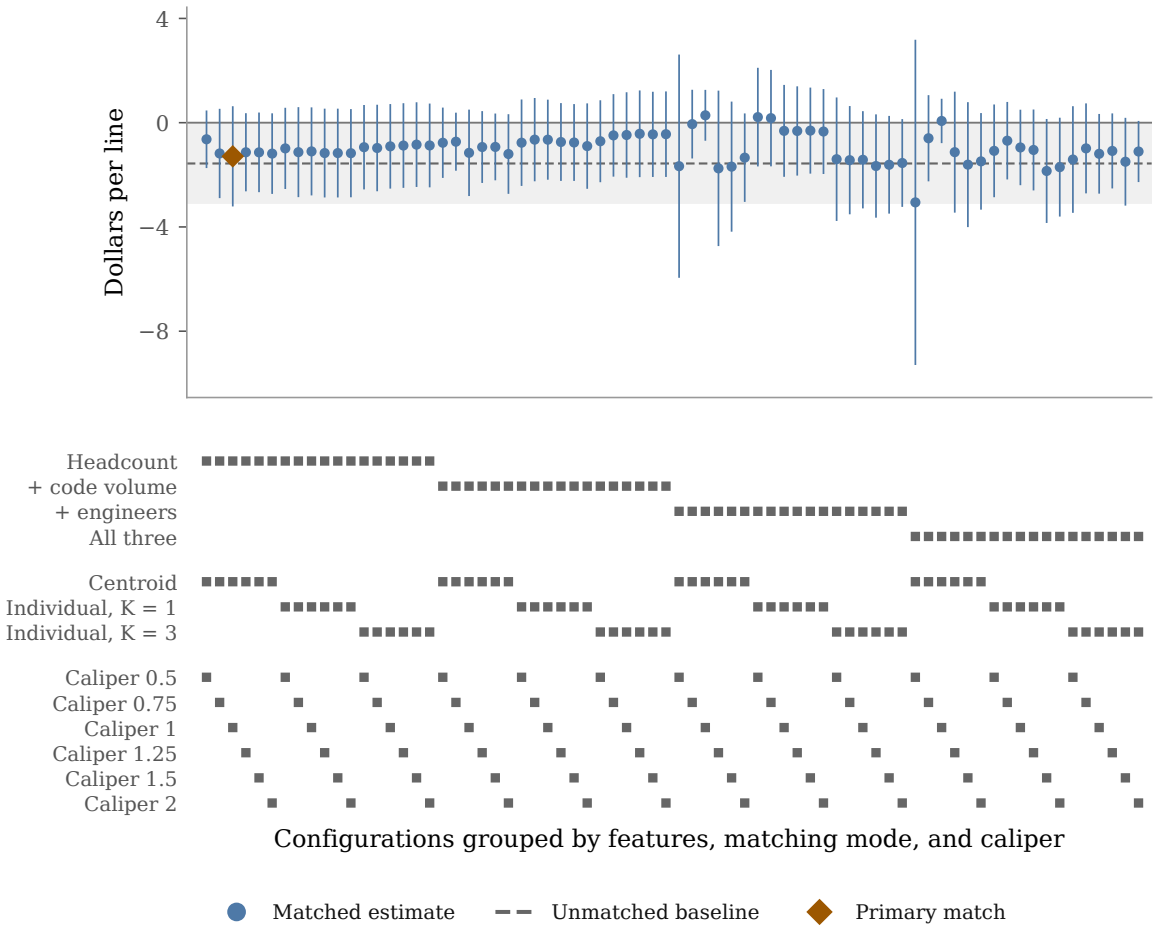


Figure 21: Specification Curve: Technical Debt per Line of Code. See Figure 19 for a description of the layout. Bars show 95% confidence intervals; standard errors are clustered by cohort and firm.

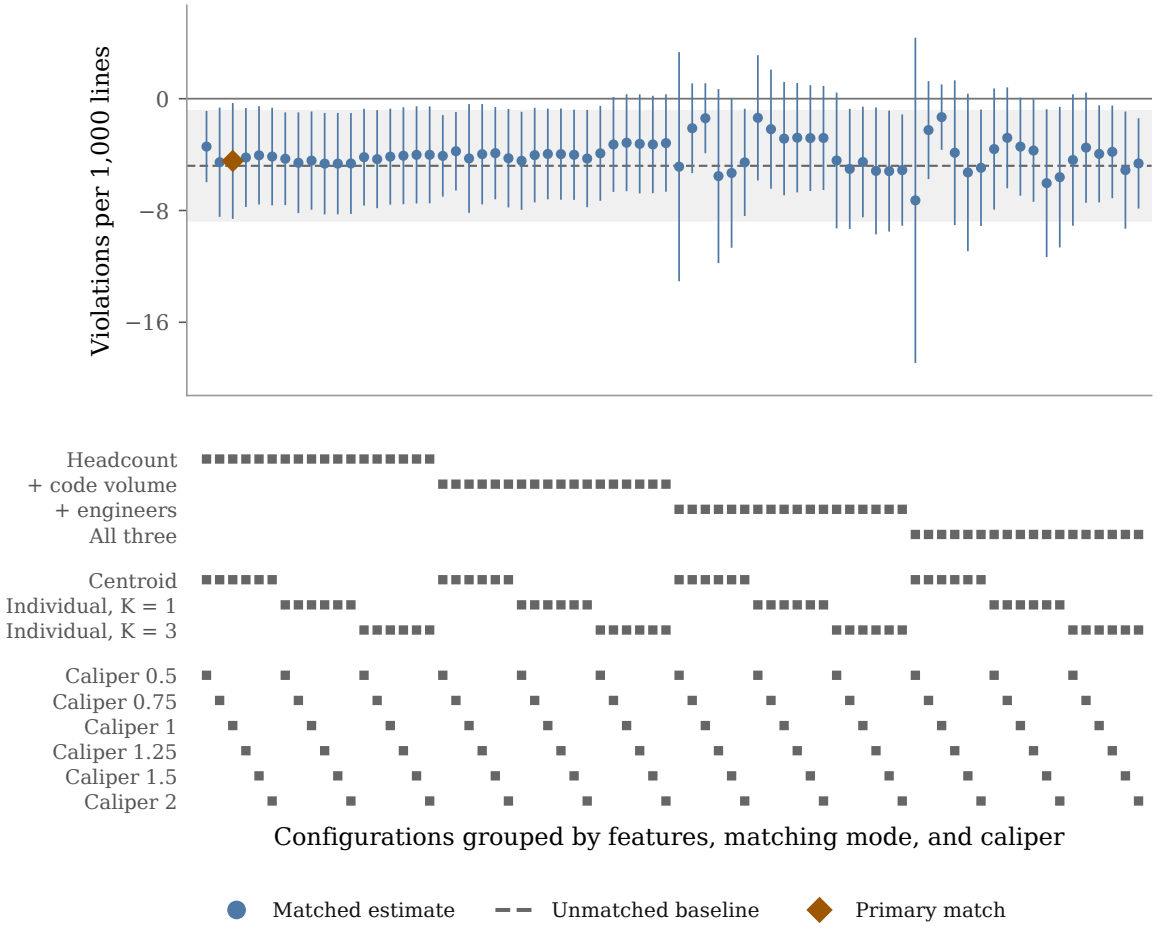


Figure 22: Specification Curve: Maintainability Violations per 1,000 LoC. See Figure 19 for a description of the layout. Bars show 95% confidence intervals; standard errors are clustered by cohort and firm.

E.1 Outcome Paths under Alternative Comparison Groups

To show how the choice of comparison group affects every outcome, not only those reported in Section 4.1, Figures 23–27 compare all 14 outcomes using the unmatched sample and the primary match on pre-financing total-headcount growth. Log-transformed outcomes include only observations with positive values.

The matching feature is the log of mean total headcount in months −4, −3, and −2 minus the log of mean total headcount in months −12 and −11. Within each cohort, controls are retained if their standardized distance from the eligible treated firms' mean growth is at most one. Both comparisons are unweighted. Appendix D gives the full eligibility and matching procedure. Panel notes report each regression's sample after missing-value and fixed-effect exclusions.

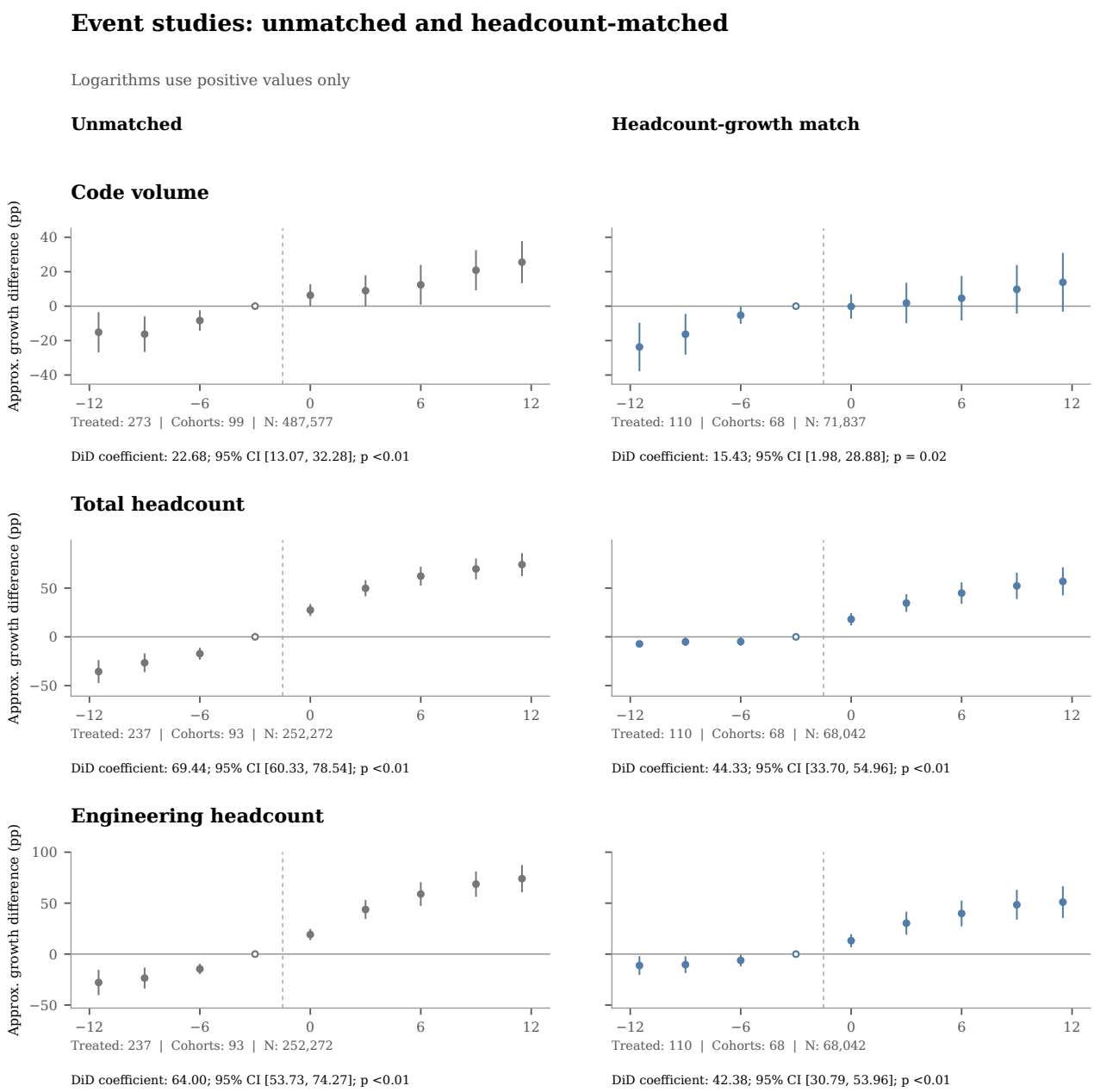


Figure 23: Alternative Comparison Groups: Part 1. Code volume, total headcount, and engineering headcount. Columns report unmatched stacked DiD and headcount-growth matching. Axes match within each row; error bars are 95% confidence intervals with standard errors clustered by cohort and firm. The dashed line marks the end of omitted months −4 through −2. Logged outcomes display approximate percentage-point growth differences relative to controls.

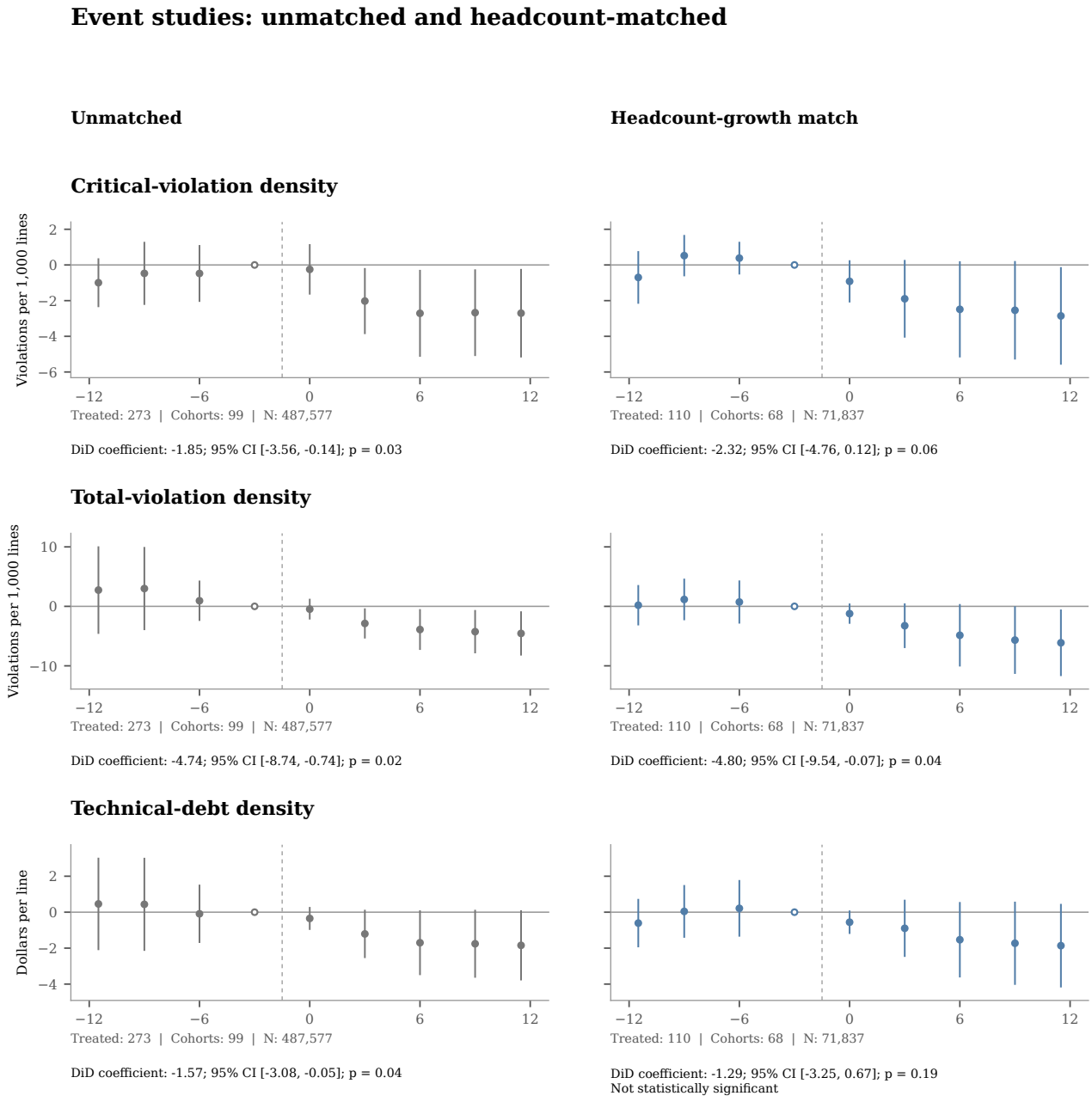


Figure 24: Alternative Comparison Groups: Part 2. Critical-violation density, total-violation density, and technical debt per line. Columns report unmatched stacked DiD and headcount-growth matching. Axes match within each row; error bars are 95% confidence intervals with standard errors clustered by cohort and firm. The dashed line marks the end of omitted months −4 through −2.

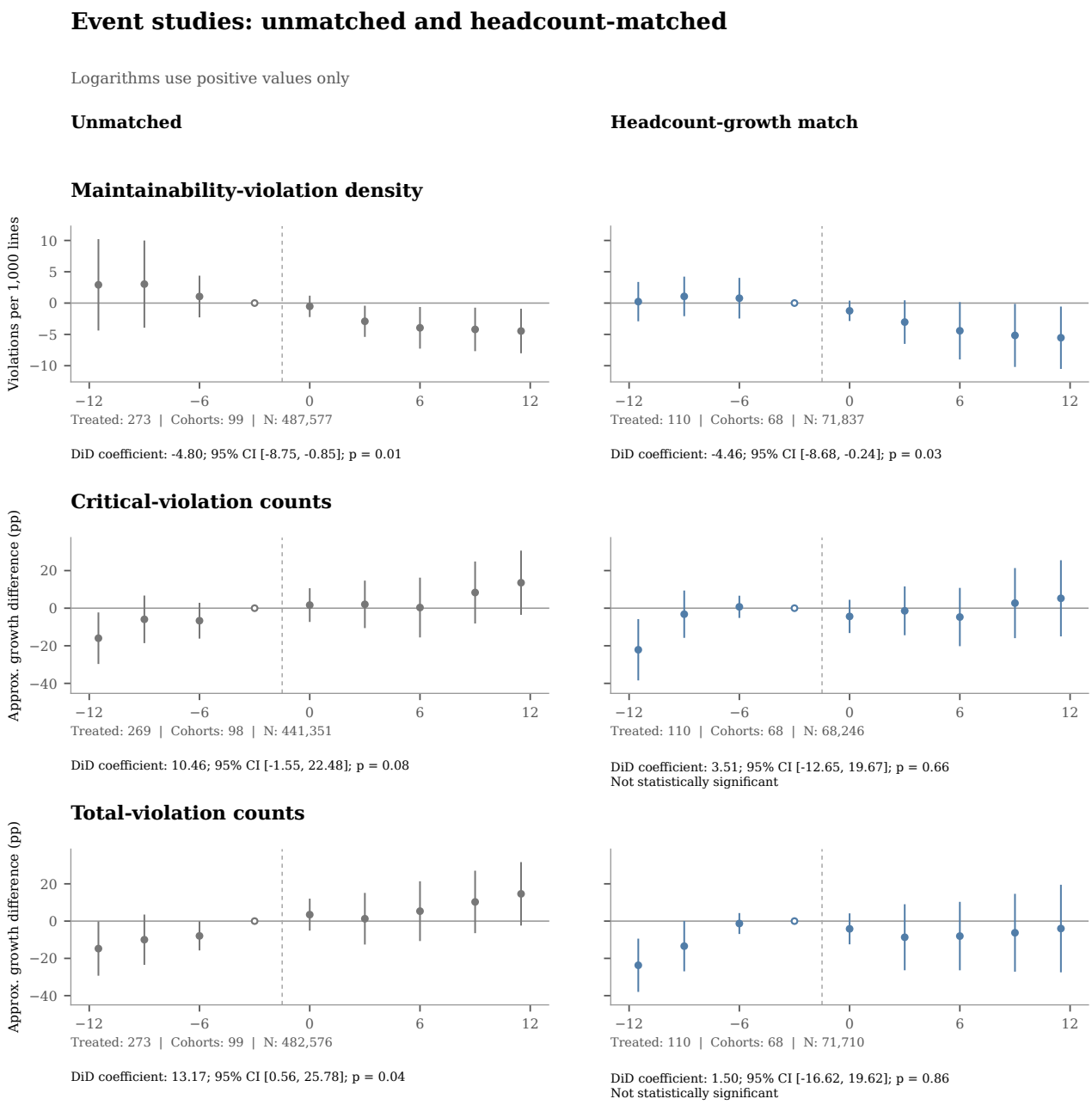


Figure 25: Alternative Comparison Groups: Part 3. Maintainability-violation density, critical-violation counts, and total-violation counts. Columns report unmatched stacked DiD and headcount-growth matching. Axes match within each row; error bars are 95% confidence intervals with standard errors clustered by cohort and firm. The dashed line marks the end of omitted months −4 through −2. Logged outcomes display approximate percentage-point growth differences relative to controls.

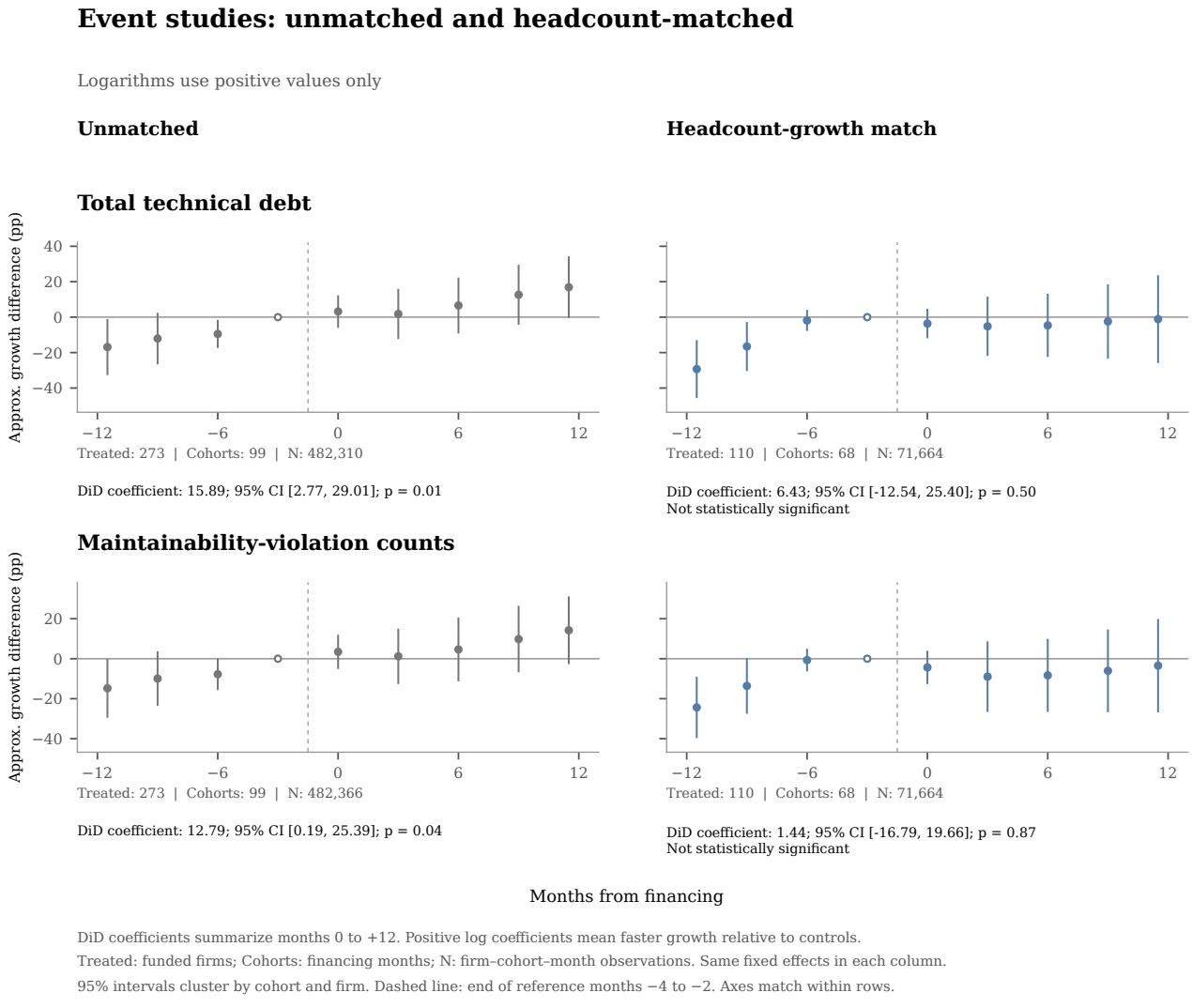


Figure 26: Alternative Comparison Groups: Part 4. Total technical debt and maintainability-violation counts. Columns report unmatched stacked DiD and headcount-growth matching. Axes match within each row; error bars are 95% confidence intervals with standard errors clustered by cohort and firm. The dashed line marks the end of omitted months −4 through −2. Logged outcomes display approximate percentage-point growth differences relative to controls.

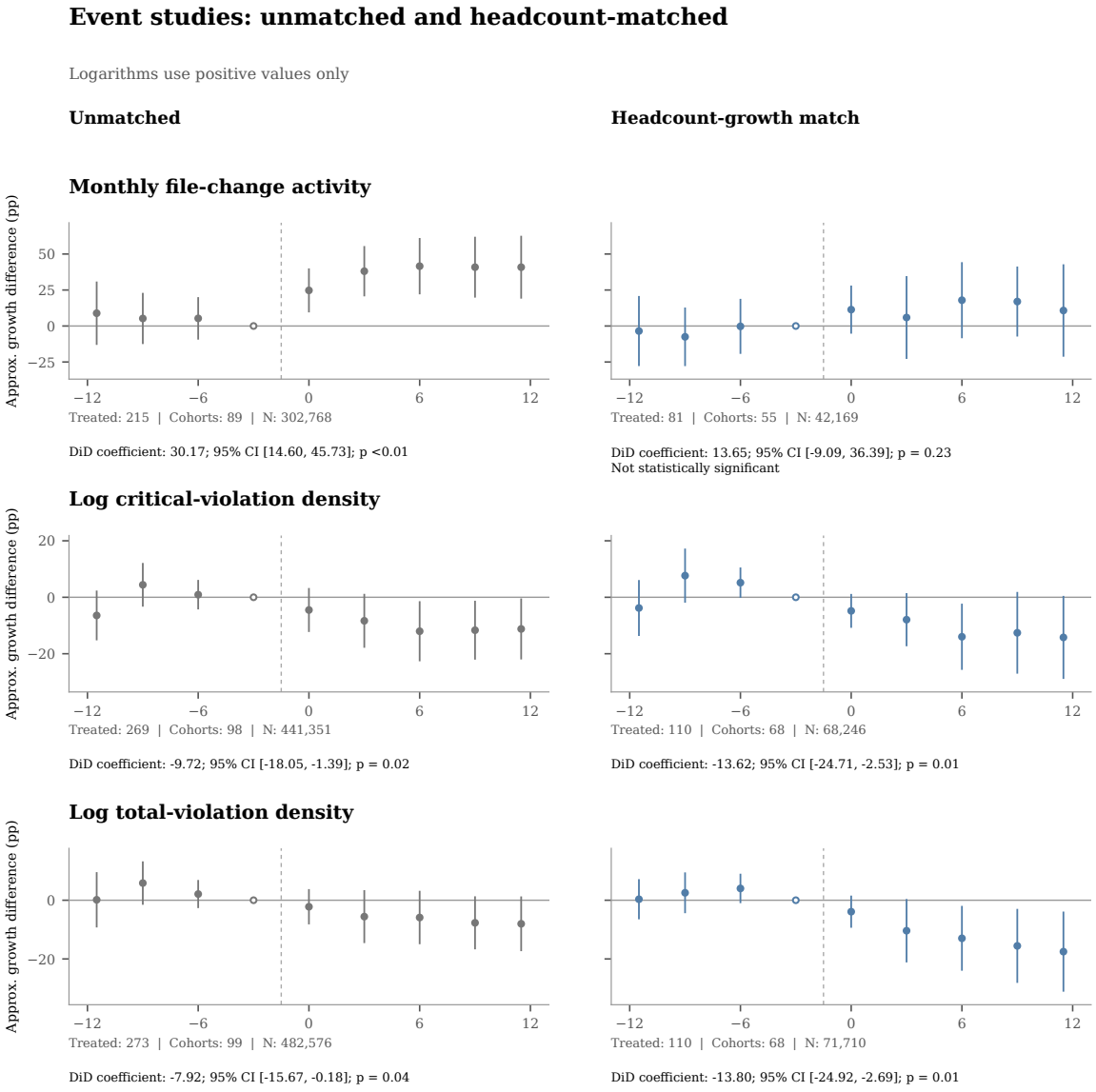


Figure 27: Alternative Comparison Groups: Part 5. Monthly file-change activity and logged critical- and total-violation densities. Columns report unmatched stacked DiD and headcount-growth matching. Axes match within each row; error bars are 95% confidence intervals with standard errors clustered by cohort and firm. The dashed line marks the end of omitted months −4 through −2. Logged outcomes display approximate percentage-point growth differences relative to controls. The file-change comparison uses main-sample firms with file-change histories.

F Founding-Team Heterogeneity

We examine whether the engineering response depends on technical leadership in the founding team and founders' prior executive experience. If these leadership roles provide knowledge that VC involvement would otherwise supply, firms whose founders hold such roles should have less scope to improve. A broader change in organizational practices could instead reach firms with either type of founding team. Using LinkedIn career histories, we identify individuals whose titles contain "founder" or "co-founder" and who were active at the firm within six months before or after first VC entry, then retrieve their prior employment records to construct two indicators.

Founder Technical Leadership equals one if any member of the founding team simultaneously held a technical leadership role—CTO, VP of Engineering, Head of Engineering, or Director of Engineering—at the same firm around the deal date.

Prior Executive Experience equals one if any founder previously held a CEO, Chief, Founder, or President role at a different company before the first VC deal date. Founders with these prior leadership roles may bring organizational practices from previous firms, leaving less scope for subsequent change.

We interact each indicator with the Treated × Post term, following the same triple-interaction specification used for the pre-treatment quality and deal size splits in the main analysis. The columns for founder characteristics include only firms with the required LinkedIn career histories, and cohorts without both treated and control firms are removed. Each indicator is computed at the firm's own first financing and retained across its cohort appearances. Eligible controls with observed founder indicators use their later financing dates, at least 13 months after the comparison cohort's event, while never-financed firms have no deal-dated indicator. Tables 20–22 report the full regression output. Each table presents six columns: columns (1)–(3) correspond to one outcome and columns (4)–(6) to a second outcome, with each set cycling through Large Codebase, Founder Technical Leadership and Prior Executive Experience.

Neither the Founder Technical Leadership nor the Prior Executive Experience interaction is statistically significant for any of the six outcomes. For technical debt per line, the Founder Technical Leadership interaction is \$0.12 and the Prior Executive Experience interaction is \$0.35 ($p = 0.94$ and $p = 0.85$, respectively). Thus, neither characteristic shows a systematic association with the engineering response.

Table 20: Founder Heterogeneity: Technical Debt per LoC and Technical Debt Ratio

	Technical Debt per LoC			**Technical Debt Ratio**		
	(1)	(2)	(3)	(4)	(5)	(6)
Treated × Post	−2.14	−1.05	−1.17	−7.77	−3.81	−4.25
	(1.31)	(1.13)	(1.62)	(4.75)	(4.09)	(5.90)
Post × Large Codebase (=1)	0.09			0.32		
	(0.42)			(1.51)		
Treated × Post × Large Codebase (=1)	1.29			4.68		
	(1.37)			(4.99)		
Post × Founder Technical Leadership (=1)		0.14			0.50	
		(0.82)			(2.99)	
Treated × Post × Founder Technical Leadership (=1)		0.12			0.44	
		(1.83)			(6.65)	
Post × Prior Executive Experience (=1)			0.53			1.93
			(0.93)			(3.37)
Treated × Post × Prior Executive Experience (=1)			0.35			1.29
			(1.89)			(6.86)
Cohort × Firm FE	Yes	Yes	Yes	Yes	Yes	Yes
Month × Cohort FE	Yes	Yes	Yes	Yes	Yes	Yes
# Obs	477,107	64,803	64,803	477,107	64,803	64,803
R^2	0.83	0.78	0.78	0.83	0.78	0.78

This table reports the coefficient β from the specification $M_{i,c,t} = \beta VC_{i,c,t} + \chi_{c,i} + \delta_{c,t} + \gamma X_{i,c,t} + \varepsilon_{i,c,t}$. Technical Debt per LoC is calibrated repair cost in dollars per line; the Technical Debt Ratio is that debt as a percentage of replacement-cost software assets. Columns (1) and (4) interact with a Large Codebase indicator (above-median lines of code). Columns (2) and (5) interact with a Founder Technical Leadership indicator. Columns (3) and (6) interact with a Prior Executive Experience indicator. See Appendix A, Table 17, for full variable definitions.

The employment-based indicators are available only for firms with LinkedIn coverage; those columns use the covered subsample, so Columns (2), (3), (5), and (6) have substantially smaller samples than Columns (1) and (4). Cohorts left without both treated and control firms after this restriction are removed.

Note: Standard errors in parentheses, clustered by cohort and firm. * p<0.1, ** p<0.05, *** p<0.01.

Table 21: Founder Heterogeneity: Violations per kLoC and Critical Violations per kLoC

	Violations per kLoC			**Critical Violations per kLoC**		
	(1)	(2)	(3)	(4)	(5)	(6)
Treated × Post	−6.35*	−2.92	−3.49	−2.35	−1.98	−2.75
	(3.42)	(2.23)	(2.85)	(1.49)	(1.63)	(2.21)
Post × Large Codebase (=1)	1.64**			0.53		
	(0.81)			(0.42)		
Treated × Post × Large Codebase (=1)	4.16			1.30		
	(3.54)			(1.59)		
Post × Founder Technical Leadership (=1)		−0.26			0.38	
		(1.93)			(0.83)	
Treated × Post × Founder Technical Leadership (=1)		−1.14			0.12	
		(4.28)			(2.73)	
Post × Prior Executive Experience (=1)			1.74			−0.50
			(1.99)			(0.90)
Treated × Post × Prior Executive Experience (=1)			0.47			1.45
			(3.71)			(2.77)
Cohort × Firm FE	Yes	Yes	Yes	Yes	Yes	Yes
Month × Cohort FE	Yes	Yes	Yes	Yes	Yes	Yes
# Obs	477,107	64,803	64,803	477,107	64,803	64,803
R^2	0.86	0.82	0.82	0.89	0.80	0.80

This table reports the coefficient β from the specification $M_{i,c,t} = \beta VC_{i,c,t} + \chi_{c,i} + \delta_{c,t} + \gamma X_{i,c,t} + \varepsilon_{i,c,t}$. Columns (1) and (4) interact with a Large Codebase indicator. Columns (2) and (5) interact with a Founder Technical Leadership indicator. Columns (3) and (6) interact with a Prior Executive Experience indicator. Violations per kLoC counts total rule violations across all severity levels per 1,000 lines of code. Critical Violations per kLoC counts violations classified as critical per 1,000 lines of code. See Appendix A, Table 17, for full variable definitions.

The employment-based indicators are available only for firms with LinkedIn coverage; those columns use the covered subsample, so Columns (2), (3), (5), and (6) have substantially smaller samples than Columns (1) and (4). Cohorts left without both treated and control firms after this restriction are removed.

Note: Standard errors in parentheses, clustered by cohort and firm. * p<0.1, ** p<0.05, *** p<0.01.

Table 22: Founder Heterogeneity: Bugs per kLoC and Reliability Issues (High) per kLoC

	Bugs per kLoC			**Reliability Issues (High) per kLoC**		
	(1)	(2)	(3)	(4)	(5)	(6)
Treated × Post	0.04	0.04	−0.04	−0.01	0.01	−0.03
	(0.20)	(0.16)	(0.15)	(0.03)	(0.02)	(0.02)
Post × Large Codebase (=1)	0.08			−0.02		
	(0.08)			(0.02)		
Treated × Post × Large Codebase (=1)	−0.08			0.00		
	(0.24)			(0.03)		
Post × Founder Technical Leadership (=1)		−0.10			0.02	
		(0.30)			(0.01)	
Treated × Post × Founder Technical Leadership (=1)		−0.63			−0.02	
		(0.46)			(0.03)	
Post × Prior Executive Experience (=1)			0.32			0.00
			(0.30)			(0.01)
Treated × Post × Prior Executive Experience (=1)			−0.24			0.05
			(0.38)			(0.03)
Cohort × Firm FE	Yes	Yes	Yes	Yes	Yes	Yes
Month × Cohort FE	Yes	Yes	Yes	Yes	Yes	Yes
# Obs	477,107	64,803	64,803	477,107	64,803	64,803
R^2	0.90	0.91	0.91	0.51	0.76	0.76

This table reports the coefficient β from the specification $M_{i,c,t} = \beta VC_{i,c,t} + \chi_{c,i} + \delta_{c,t} + \gamma X_{i,c,t} + \varepsilon_{i,c,t}$. Columns (1) and (4) interact with a Large Codebase indicator. Columns (2) and (5) interact with a Founder Technical Leadership indicator. Columns (3) and (6) interact with a Prior Executive Experience indicator. Bugs per kLoC counts patterns classified by SonarQube as likely to cause incorrect or unexpected behavior per 1,000 lines of code. Reliability Issues (High) per kLoC counts high-severity issues associated with a risk of incorrect behavior per 1,000 lines of code. Bugs use the traditional classification; Reliability Issues (High) uses the alternative classification described in Appendix M.2 and is not a subset of Bugs. See Appendix A, Table 17, for full variable definitions.

The employment-based indicators are available only for firms with LinkedIn coverage; those columns use the covered subsample, so Columns (2), (3), (5), and (6) have substantially smaller samples than Columns (1) and (4). Cohorts left without both treated and control firms after this restriction are removed.

Note: Standard errors in parentheses, clustered by cohort and firm. * p<0.1, ** p<0.05, *** p<0.01.

Interpretation. The quality gains are not systematically concentrated among firms without these founder leadership indicators. The pattern is consistent with organizational changes—such as formal tracking procedures and coordination across developers—that can benefit teams with different founder leadership roles. Section 3.8 examines the development practices of existing engineers directly.

Post-Financing Hiring Composition

We compare quality changes by hiring composition during the first year after financing. *Hired Technical Leader* identifies a CTO, VP of Engineering or Head of Engineering hire in the interval from the event through, but excluding, the date 12 calendar months later. *High Senior Ratio* identifies

firms whose senior-engineering hires as a share of all new hires in that interval exceed the treated-firm median of $1/7$. Both treated firms and controls are classified using the relevant cohort event. New hires require recorded start dates and no prior position at the firm before that event.

Tables 23 and 24 report the base, interaction and subgroup sum. Missing hiring records are excluded, and cohorts without both treated and control firms are removed. The leader comparison contains 31,925 observations, 161 firms and 137 treated firms; the senior-share comparison contains 26,451 observations, 159 firms and 135 treated firms. Both retain 63 cohorts.

Table 23: First-Year Hiring Composition: Critical and Total Violations

	Critical Violations / kLoC		**Violations / kLoC**	
	Leader	Senior share	Leader	Senior share
	(1)	(2)	(3)	(4)
Baseline on the same sample, no hiring interactions	−1.39	−1.13	−1.27	−0.90
	(2.22)	(2.28)	(3.49)	(3.58)
Treated × Post	−2.87	−2.01	−4.16	−2.51
	(3.13)	(3.49)	(4.05)	(4.47)
Post × Hiring indicator	0.20	0.03	−3.33	−2.22
	(1.29)	(1.25)	(3.25)	(2.56)
Treated × Post × Hiring indicator	3.38	1.63	8.29	3.56
	(3.30)	(4.07)	(5.14)	(5.06)
Hiring-group effect (base + interaction)	0.51	−0.38	4.12	1.04
	(2.00)	(2.51)	(5.07)	(4.35)
95% interval, hiring-group effect	[−3.48, 4.51]	[−5.40, 4.64]	[−6.01, 14.25]	[−7.66, 9.75]
Cohort × Firm FE	Yes	Yes	Yes	Yes
Month × Cohort FE	Yes	Yes	Yes	Yes
# Obs	31,925	26,451	31,925	26,451
# Firms	161	159	161	159
# Treated	137	135	137	135

Critical and total violations are counts per 1,000 lines of code. Leader and senior-share indicators are constructed from LinkedIn hiring during [event, event + 12 months), using the cohort event for both treated and control firms. Senior share is senior-engineering hires divided by all distinct new hires and is missing when no hires are recorded. The hiring-group effect is the sum of the base and interaction; its standard error and 95% interval use their full covariance. Both comparisons retain 63 cohorts.

Note: Standard errors in parentheses, clustered by cohort and firm. * p<0.1, ** p<0.05, *** p<0.01.

Table 24: First-Year Hiring Composition: Technical Debt and Maintainability

	Technical Debt per LoC ($)		**Maintainability / kLoC**	
	Leader	Senior share	Leader	Senior share
	(1)	(2)	(3)	(4)
Baseline on the same sample, no hiring interactions	−0.41	−0.11	−1.49	−1.16
	(1.61)	(1.65)	(3.26)	(3.34)
Treated × Post	−1.50	−0.88	−4.07	−2.74
	(1.98)	(2.47)	(3.85)	(4.35)
Post × Hiring indicator	−1.44	−0.74	−2.43	−1.89
	(1.31)	(1.25)	(2.79)	(2.25)
Treated × Post × Hiring indicator	3.21	1.61	7.16	3.43
	(2.57)	(2.92)	(4.75)	(4.85)
Hiring-group effect (base + interaction)	1.71	0.73	3.09	0.69
	(2.28)	(1.94)	(4.54)	(3.94)
95% interval, hiring-group effect	[−2.85, 6.27]	[−3.14, 4.61]	[−5.99, 12.18]	[−7.19, 8.56]
Cohort × Firm FE	Yes	Yes	Yes	Yes
Month × Cohort FE	Yes	Yes	Yes	Yes
# Obs	31,925	26,451	31,925	26,451
# Firms	161	159	161	159
# Treated	137	135	137	135

Maintainability violations are counts per 1,000 lines of code; technical debt is calibrated dollars per line. Leader and senior-share indicators are constructed from LinkedIn hiring during [event, event + 12 months), using the cohort event for both treated and control firms. Senior share is senior-engineering hires divided by all distinct new hires and is missing when no hires are recorded. The hiring-group effect is the sum of the base and interaction; its standard error and 95% interval use their full covariance. Both comparisons retain 63 cohorts.

Note: Standard errors in parentheses, clustered by cohort and firm. * p<0.1, ** p<0.05, *** p<0.01.

The point estimates do not show larger quality improvements among firms that hire technical leaders or a high share of senior engineers. The hiring interactions are positive across all four outcomes, and none is statistically significant. The estimates for firms without these hiring indicators are negative and are also not statistically significant. Tables 23 and 24 report the group effects and their confidence intervals. Section 3.8 examines the development records of existing engineers directly.

Table 25 reports the hiring-composition outcomes discussed in Section 3.7.

Table 25: Hiring Composition After First VC Entry

	Pre/Post 2×2		Quarterly Panel	
	Tech. Leader (pp)	Non-tech. Leader (pp)	SWE Share (pp)	SWE Seniority
	(1)	(2)	(3)	(4)
Treated × Post	0.64	−6.90	5.78**	−0.11
	(5.06)	(6.99)	(2.90)	(0.10)
Cohort × Firm FE	Yes	Yes	Yes	Yes
Post FE	Yes	Yes	—	—
Quarter × Cohort FE	—	—	Yes	Yes
# Obs	8,298	8,298	31,339	20,889
# Firms	218	218	260	217
R^2	0.74	0.75	0.42	0.49

This table reports the coefficient on Treated × Post from a DiD specification for four hiring-composition outcomes constructed from LinkedIn position histories. Columns (1) and (2) compare two periods for each firm–cohort pair—pre = relative month ∈ [−12, −1] and post = relative month ∈ [0, 11]—and estimate $Y_{i,c,p} = \beta \cdot VC_{i,c,p} + \chi_{c,i} + \delta_p + \varepsilon_{i,c,p}$. $Y_{i,c,p}$ is the corresponding leadership-hiring indicator. We retain firm–cohort pairs observed in every month of both 12-month periods, exclude fixed-effect singletons, and require both treated and control firms in each cohort. Each leadership comparison contains 218 firms, 112 treated firms and 63 cohorts. Leadership coefficients and standard errors in Columns (1) and (2) are displayed in percentage points. *Tech. Leader* is an indicator equal to one if the firm hired a CTO, VP of Engineering, Head/Director of Engineering, or VP of Technology during the period. *Non-tech. Leader* is the analogous indicator for a nontechnical executive (chief executive, operating, financial, marketing, product, revenue, or customer officer, or a head, vice president, or director of a non-engineering function).

Columns (3) and (4) estimate the quarterly stacked DiD, $M_{i,c,t} = \beta \cdot VC_{i,c,t} + \chi_{c,i} + \delta_{c,t} + \varepsilon_{i,c,t}$, at firm–cohort-quarter frequency. *SWE Share* is the share of distinct new hires in the quarter classified as software engineers in nonmanagerial roles; its coefficient and standard error are displayed in percentage points. *SWE Seniority* is the mean LinkedIn seniority (1–7 scale) of those hires. Both ratios are missing when the denominator is zero, and firms without LinkedIn coverage are excluded from all four columns. See Appendix A, Table 17, for full variable definitions.

The relative change in SWE Share divides the estimate in Column (3) by the treated pre-treatment mean in that column's estimation sample (30.4%).

Note: Standard errors in parentheses, clustered by cohort and firm. * p<0.1, ** p<0.05, *** p<0.01.

G Levels versus Density

The main results concern the quality of software production, measured by violations per unit of code. This normalization is analogous to a manufacturing defect rate: a producer can make more units and accumulate more defects in total while reducing the share of output that is defective. In software, each addition or modification to a codebase can introduce a new issue, so violation density helps distinguish the quality of engineering practices from the amount of code produced. We examine that distinction by estimating the baseline specification on total critical violations, total violations, and total technical debt, transformed as log(Critical), log(Violations), log(Debt), respectively.

Table 26 compares the per-line and total estimates. Critical violations fall by 1.85 per 1,000 lines of code ($p = 0.03$; Column (1)), while the transformed critical-violation count has a positive coefficient of 0.10 ($p = 0.08$; Column (3)). Technical debt falls by \$1.57 per line ($p = 0.04$; Column (2)). The growth rate in total technical debt in dollars is approximately 16 percentage points greater than in controls ($p = 0.01$; Column (4)). The growth rate in total violations is approximately 13 percentage points greater than in controls ($p = 0.04$; not tabulated). These findings show why counts and densities answer different economic questions: the accumulated burden can increase as the product grows even when each unit of software carries less debt.

Table 26: Critical Violations and Technical Debt: Per-Line Measures and Totals

	Density		**Log totals**	
	Critical Violations / kLoC	Technical Debt per LoC (\$)	Critical Violations	Technical Debt (\$)
	(1)	(2)	(3)	(4)
Treated × Post	−1.85**	−1.57**	0.10*	0.16**
	(0.86)	(0.77)	(0.06)	(0.07)
Cohort × Firm FE	Yes	Yes	Yes	Yes
Month × Cohort FE	Yes	Yes	Yes	Yes
# Obs	487,577	487,577	441,351	482,310
R^2	0.88	0.83	0.94	0.92

This table compares the stacked DiD estimates for critical-violation density and technical debt per line with estimates for total critical violations and total technical debt. Columns (1)–(2) reproduce the baseline per-LoC results from Tables 3 and 7. Columns (3)–(4) estimate the same specification using log(Critical) and log(Debt). The log specifications use only positive values. All specifications include Firm × Cohort and Month × Cohort fixed effects. See Appendix A, Table 17, for full variable definitions.

Note: Standard errors in parentheses, clustered by cohort and firm. * p<0.1, ** p<0.05, *** p<0.01.

A constant violation rate provides a useful benchmark for these magnitudes. If violation counts expanded proportionately with code, their growth rates would approximately match the estimated code-volume growth rate. The growth rate in code volume is approximately 23 percentage points greater than in controls ($p < 0.01$). The growth rates in critical violations, total violations, and total technical debt are below that benchmark. Using the same observations as each outcome, we estimate code volume coefficients of 0.20 for critical violations ($N = 441,351$), 0.21 for total violations ($N = 482,576$), and 0.21 for technical debt in dollars ($N = 482,310$). The comparison is consistent with firms adding code to a higher standard as they expand, and it explains how statistically significant density improvements can coexist with no statistically significant reduction in total critical-violation counts.[9]

The count-density comparison motivates a further check among firms with low pre-financing codebase growth, reported in Appendix H. Appendix I presents event studies for issue counts and total

[9]The transformations follow Table 17.

technical debt, while the contribution-level analysis in Section 3.8 provides separate evidence that contributions from people active before financing are less often linked to later fixes.

H Decomposing the Quality Improvement

We examine whether engineering quality improves among firms whose codebases had previously grown slowly. We then separate changes in violations per line into changes in the number of violations and changes in codebase size.

Pre-Treatment Codebase Growth

For each financing cohort, we measure codebase growth by comparing average log lines of code in months -12 through -7 with months -6 through -1. A firm can enter several cohort comparisons, first as a control and later as a treated firm. Among these observations for firms that eventually receive financing, we identify the lowest-growth quartile. We retain a firm if it falls in this bottom quartile in any cohort comparison, including an earlier comparison in which it serves as a control, and retain the controls in its cohorts. Every growth measure used for selection precedes its cohort's financing date, so selection does not depend on growth after financing. Across these observations, pre- and post-financing growth have a correlation of 0.57, and 62% of observations in the lowest pre-financing growth quartile remain in the lowest quartile afterward.

Table 27 shows that the critical-violation estimate strengthens in this subsample. Compared with 1.85 in the full sample, critical violations decline by 2.85 per 1,000 lines of code ($p = 0.06$). Technical debt per line declines by \$1.64 ($p = 0.05$), and total violations by 3.7 per 1,000 lines ($p = 0.04$). Joint tests do not reject zero pre-treatment coefficients for these outcomes. The log-code-volume coefficient, which measures growth relative to controls, is -0.02 ($p = 0.73$; not statistically significant). The treated firms' median change in their own mean log code volume from the pre- to the post-financing period is +0.13, approximately 13% growth in their own codebases in this sample. The comparison therefore documents quality gains among firms selected using low prior growth while allowing their codebases to expand.

The larger critical-violation estimate also has an economic interpretation. Firms whose codebases expand slowly may have more scope to devote engineering effort to existing software, including remediation and improvements in development procedures. Alternatively, the full-sample estimate may combine these gains with the additional coordination demands of rapid feature development. The low-growth result is consistent with both possibilities: improved engineering discipline need not depend on producing large amounts of additional code.

Table 27: Engineering Quality Among Firms with Low Pre-Treatment Codebase Growth

	Critical Violations / kLoC		Technical Debt per LoC	
	Full Sample	Low Pre-Treatment Growth (bottom quartile)	Full Sample	Low Pre-Treatment Growth (bottom quartile)
	(1)	(2)	(3)	(4)
Treated × Post	−1.85**	−2.85*	−1.57**	−1.64*
	(0.86)	(1.53)	(0.77)	(0.83)
Cohort × Firm FE	Yes	Yes	Yes	Yes
Month × Cohort FE	Yes	Yes	Yes	Yes
# Obs	487,577	361,439	487,577	361,439

This table compares the baseline per-LoC estimates (Columns 1 and 3) with estimates for treated firms that fall in the bottom quartile of codebase growth in at least one cohort comparison, including an earlier comparison in which they serve as controls (Columns 2 and 4). Growth compares average log lines of code in months $[-12, -7]$ with months $[-6, -1]$ before the relevant cohort event. We retain all control firms in the selected firms' cohorts. In this subsample, the coefficient for log(LoC) is −0.02 and is not statistically significant ($p = 0.73$). Cohorts left with only treated or only control firms after the restriction are dropped. Total violations per kLoC in the same subsample: −3.7 ($p = 0.04$). Joint tests do not reject zero pre-treatment coefficients for the reported outcomes. See Appendix A, Table 17, for variable definitions.

Note: Standard errors in parentheses, clustered by cohort and firm. * p<0.1, ** p<0.05, *** p<0.01.

Log-Density Decomposition

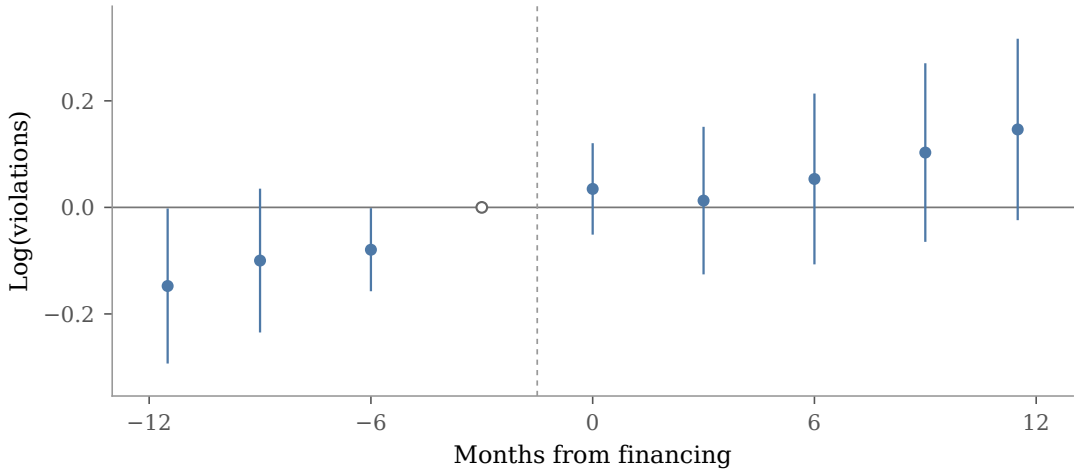
We also estimate the baseline specification on log-density measures. The critical-violation outcome is $\log(\text{Critical}) - \log(\text{LoC})$, and the total-violation outcome is $\log(\text{Violations}) - \log(\text{LoC})$. The critical-violation coefficient is −0.10 ($p = 0.02$; $N = 441,351$), and the total-violation coefficient is −0.08 ($p = 0.04$; $N = 482,576$). The corresponding joint pre-treatment tests give $p = 0.05$ for critical violations and $p = 0.15$ for total violations; the total-violation joint test is not statistically significant. The log specifications include only positive counts.

The critical-violation estimate decomposes into a count coefficient of +0.10 ($p = 0.08$) and a code-volume coefficient of +0.20 ($p < 0.01$). The identity makes the source of the transformed density decline explicit: the estimated count change is smaller than the estimated change in code volume. This decomposition describes the accounting relationship between the two components, while the low-growth comparison above shows that the density decline also occurs among firms selected using low prior code growth.

I Level Outcomes: Event-Study Figures

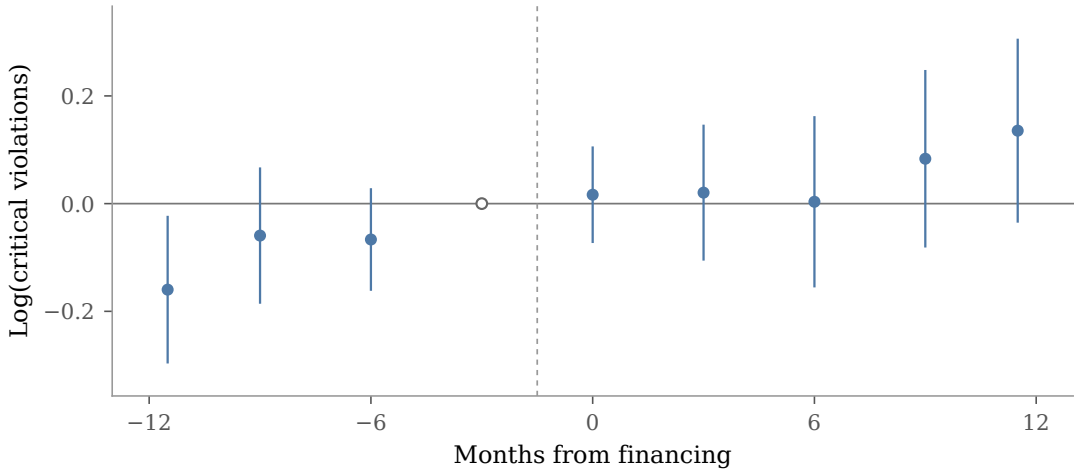
Figures 28–31 present event-study estimates for total code violations and total technical debt. We take logarithms of total violations, critical violations, total maintainability violations, and technical

debt in dollars. Observations with zero values are excluded. These measures describe changes in the total burden as the codebase grows; the per-line measures in the main text describe its density. Before financing, the estimated differences between treated and control firms generally rise toward zero, indicating that their total violations and technical debt were already growing at different rates.



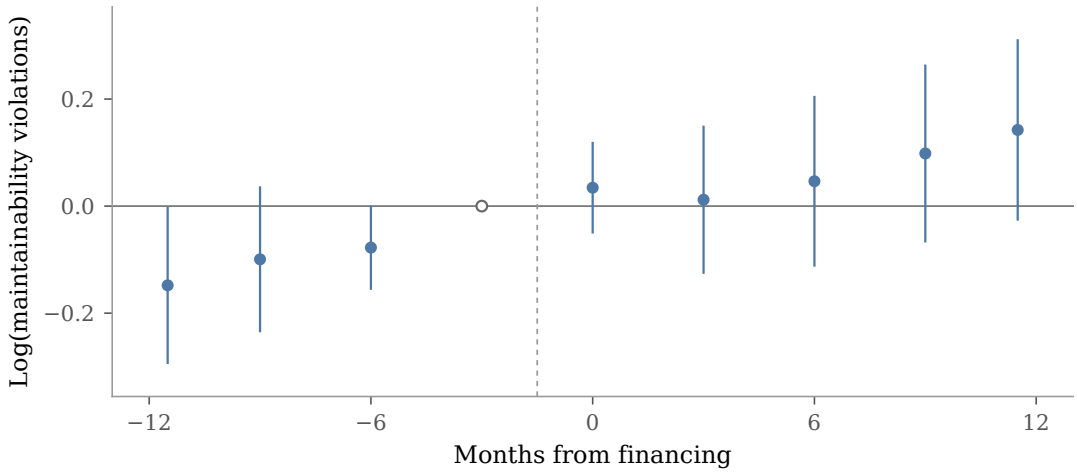
Positive values; N=482,576. Reference: months −4 to −2.

Figure 28: Log(Total Violations). Positive-value sample, $N = 482,576$. Treated firms complete their first Seed round, or their first Series A round when no completed Seed is recorded; controls remain without a qualifying first round through the event window. Quarterly estimates are relative to months −4 to −2; the bin plotted at month 0 spans months −1 to +1. The dashed line marks the end of the reference months. Bars show 95% confidence intervals; standard errors are clustered by cohort and firm.



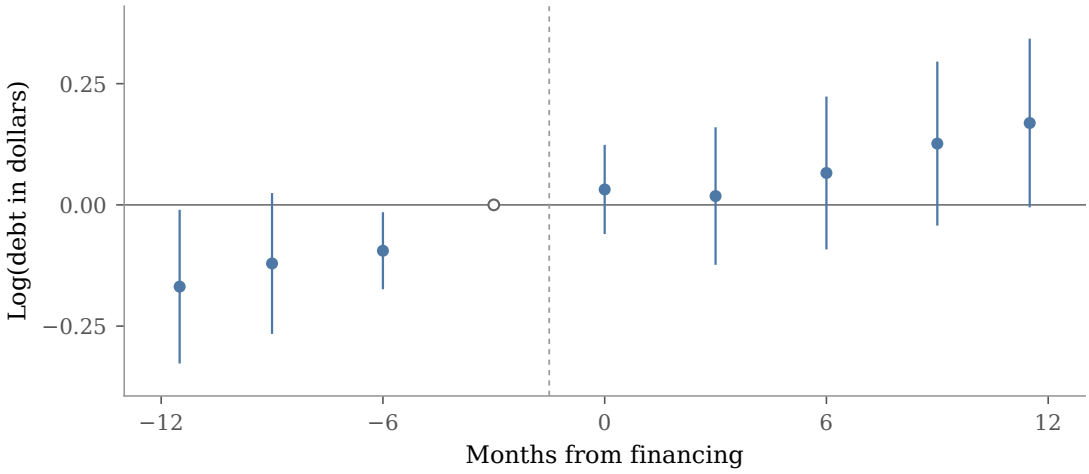
Positive values; N=441,351. Reference: months −4 to −2.

Figure 29: Log(Critical Violations). Positive-value sample, $N = 441,351$. Treated firms complete their first Seed round, or their first Series A round when no completed Seed is recorded; controls remain without a qualifying first round through the event window. Quarterly estimates are relative to months −4 to −2; the bin plotted at month 0 spans months −1 to +1. The dashed line marks the end of the reference months. Bars show 95% confidence intervals; standard errors are clustered by cohort and firm.



Positive values; N=482,366. Reference: months −4 to −2.

Figure 30: Log(Total Maintainability Violations). Positive-value sample, $N = 482,366$. Treated firms complete their first Seed round, or their first Series A round when no completed Seed is recorded; controls remain without a qualifying first round through the event window. Quarterly estimates are relative to months −4 to −2; the bin plotted at month 0 spans months −1 to +1. The dashed line marks the end of the reference months. Bars show 95% confidence intervals; standard errors are clustered by cohort and firm.



Positive values; N=482,310. Reference: months −4 to −2.

Figure 31: Log(Technical Debt $). Positive-value sample, $N = 482,310$. Treated firms complete their first Seed round, or their first Series A round when no completed Seed is recorded; controls remain without a qualifying first round through the event window. Quarterly estimates are relative to months −4 to −2; the bin plotted at month 0 spans months −1 to +1. The dashed line marks the end of the reference months. Bars show 95% confidence intervals; standard errors are clustered by cohort and firm.

J Series B+ Placebo Estimates: Later Financing Rounds

Figures 32–38 trace engineering-quality outcomes around each firm's first completed Series B-or-later round. Tables 28–32 report the corresponding DiD estimates and heterogeneity comparisons.

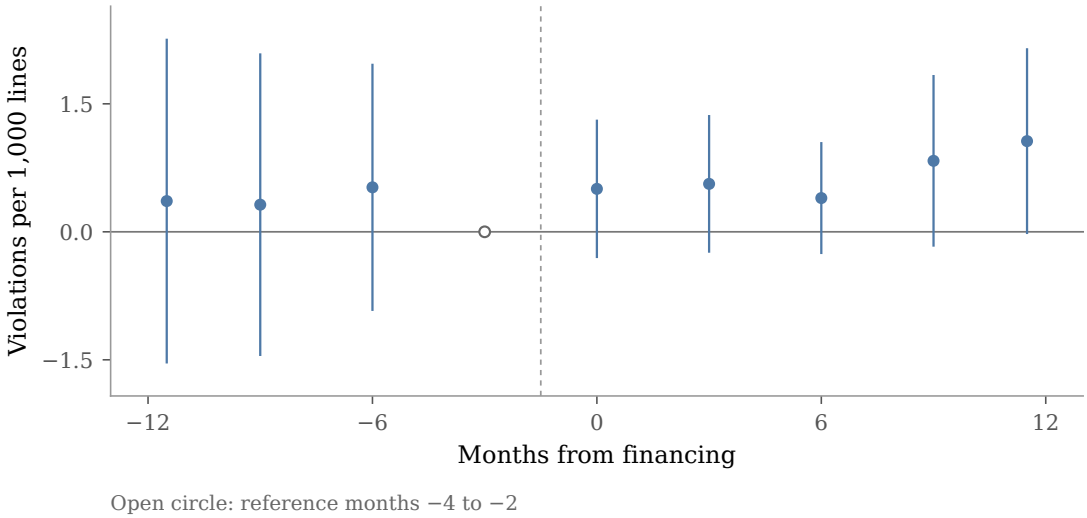


Figure 32: Later Financing: Critical Violations per 1,000 Lines of Code. Treated firms complete their first Series B-or-later round. Controls remain without a completed Seed or Series A round through the event window. Quarterly estimates are relative to months −4 to −2; the bin plotted at month 0 spans months −1 to +1. The dashed line marks the end of the reference months. Bars show 95% confidence intervals; standard errors are clustered by cohort and firm.

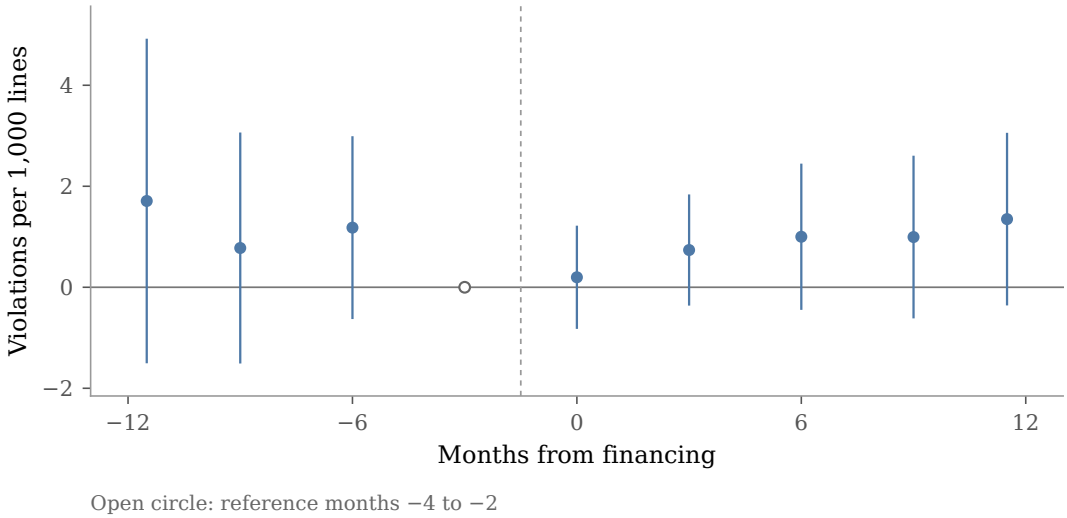


Figure 33: Later Financing: Violations (All Severities) per 1,000 Lines of Code. Treated firms complete their first Series B-or-later round. Controls remain without a completed Seed or Series A round through the event window. Quarterly estimates are relative to months −4 to −2; the bin plotted at month 0 spans months −1 to +1. The dashed line marks the end of the reference months. Bars show 95% confidence intervals; standard errors are clustered by cohort and firm.

Table 28: Series B+ Placebo Difference-in-Differences Estimates for Critical Violations and All Violations

	Critical Violations			**Violations**		
	(1)	(2)	(3)	(4)	(5)	(6)
Treated × Post	0.37	0.30*	0.51*	0.13	−0.26	−0.07
	(0.57)	(0.16)	(0.27)	(0.96)	(0.54)	(0.73)
Post × High Violations (=1)		−1.45***			−2.05*	
		(0.37)			(1.06)	
Treated × Post × High Violations (=1)		0.29			1.02	
		(1.17)			(1.88)	
Post × Large Round			−0.29			−1.38
			(0.46)			(0.85)
Treated × Post × Large Round			−0.10			1.52
			(1.29)			(1.92)
Cohort × Firm FE	Yes	Yes	Yes	Yes	Yes	Yes
Month × Cohort FE	Yes	Yes	Yes	Yes	Yes	Yes
# Obs	331,893	330,508	331,893	331,893	330,508	331,893
R^2	0.87	0.87	0.87	0.87	0.87	0.87

This table reports Series B+ placebo estimates from the specification $M_{i,c,t} = \beta VC_{i,c,t} + \chi_{c,i} + \delta_{c,t} + \gamma X_{i,c,t} + \varepsilon_{i,c,t}$, where $VC_{i,c,t}$ is the Series B+ Treated × Post indicator. *Critical Violations* count severe SonarQube rule violations per 1,000 lines of code, including risky code structures with a potentially substantial impact on reliability, security, or maintainability. *Violations (All)* denote total deviations from engineering best practices across all severity levels per 1,000 lines of code. See Appendix A, Table 17, for full variable definitions.

High Violations is defined from critical-violation density in the month before the placebo event. Large Round refers to the amount raised at the earlier Seed or Series A treatment round. The quality cutoff uses baseline firm–cohort observations of firms that appear as placebo-treated; the round-size cutoff uses one observation per placebo-treated firm.

Note: Standard errors in parentheses. * p<0.1, ** p<0.05, *** p<0.01. Standard errors are clustered by cohort and firm.

Table 29: Series B+ Placebo Difference-in-Differences Estimates for Maintainability Violations and Severity-Specific Issues

	Maintainability Violations (All)			**Maintainability Issues (High)**			**Maintainability Issues (Low)**		
	(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)
Treated × Post	0.19	−0.28	−0.06	0.38	0.30*	0.53*	−0.39	−0.20	−0.58
	(0.86)	(0.53)	(0.69)	(0.57)	(0.16)	(0.27)	(0.47)	(0.26)	(0.49)
Post × High Violations (=1)		−1.92*			−1.46***			0.28	
		(1.06)			(0.37)			(0.72)	
Treated × Post × High Violations (=1)		1.17			0.30			−0.42	
		(1.64)			(1.17)			(0.81)	
Post × Large Round			−1.47*			−0.29			−0.86*
			(0.83)			(0.46)			(0.46)
Treated × Post × Large Round			1.70			−0.13			1.10**
			(1.59)			(1.29)			(0.53)
Cohort × Firm FE	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Month × Cohort FE	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
# Obs	331,893	330,508	331,893	331,893	330,508	331,893	331,893	330,508	331,893
R^2	0.86	0.86	0.86	0.87	0.87	0.87	0.84	0.85	0.84

This table reports Series B+ placebo estimates from the specification $M_{i,c,t} = \beta VC_{i,c,t} + \chi_{c,i} + \delta_{c,t} + \gamma X_{i,c,t} + \varepsilon_{i,c,t}$, where $VC_{i,c,t}$ is the Series B+ Treated × Post indicator. Maintainability Violations (All) capture total maintainability-related rule violations per 1,000 lines of code under the traditional classification, as in Table 4. High-Severity Maintainability Issues identify maintainability problems assigned high impact, while Low-Severity Maintainability Issues capture more minor structural or readability deficiencies; both are counts per 1,000 lines of code under the alternative classification described in Appendix M.2 and are not a breakdown of the traditional overall count. See Appendix A, Table 17, for full variable definitions.

High Violations is defined from critical-violation density in the month before the placebo event. Large Round refers to the amount raised at the earlier Seed or Series A treatment round. The quality cutoff uses baseline firm–cohort observations of firms that appear as placebo-treated; the round-size cutoff uses one observation per placebo-treated firm.

Note: Standard errors in parentheses. * p<0.1, ** p<0.05, *** p<0.01. Standard errors are clustered by cohort and firm.

Table 30: Series B+ Placebo Difference-in-Differences Estimates for Security Hotspots

	Security Hotspots		
	(1)	(2)	(3)
Treated × Post	0.07	0.04	0.09
	(0.08)	(0.13)	(0.09)
Post × High Violations (=1)		0.08	
		(0.11)	
Treated × Post × High Violations (=1)		0.07	
		(0.12)	
Post × Large Round			−0.11
			(0.11)
Treated × Post × Large Round			0.06
			(0.12)
Cohort × Firm FE	Yes	Yes	Yes
Month × Cohort FE	Yes	Yes	Yes
# Obs	331,893	330,508	331,893
R^2	0.70	0.70	0.70

This table reports Series B+ placebo estimates from the specification $M_{i,c,t} = \beta VC_{i,c,t} + \chi_{c,i} + \delta_{c,t} + \gamma X_{i,c,t} + \varepsilon_{i,c,t}$, where $VC_{i,c,t}$ is the Series B+ Treated × Post indicator. *Security Hotspots* measure security-sensitive code segments requiring human review per 1,000 lines of code. See Appendix A, Table 17, for full variable definitions.

High Violations is defined from critical-violation density in the month before the placebo event. Large Round refers to the amount raised at the earlier Seed or Series A treatment round. The quality cutoff uses baseline firm–cohort observations of firms that appear as placebo-treated; the round-size cutoff uses one observation per placebo-treated firm.

Note: Standard errors in parentheses. * p<0.1, ** p<0.05, *** p<0.01. Standard errors are clustered by cohort and firm.

Table 31: Series B+ Placebo Difference-in-Differences Estimates for Bugs and Reliability Issues

	Bugs			**Reliability Issues (High)**			**Reliability Issues (Low)**		
	(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)
Treated × Post	−0.13	−0.05	−0.06	−0.01	−0.01	−0.02	−0.15	−0.20	−0.15
	(0.16)	(0.06)	(0.10)	(0.01)	(0.01)	(0.01)	(0.13)	(0.19)	(0.13)
Post × High Violations (=1)		−0.19**			0.00			−0.02	
		(0.09)			(0.02)			(0.14)	
Treated × Post × High Violations (=1)		−0.16			0.00			0.11	
		(0.33)			(0.02)			(0.22)	
Post × Large Round			0.08			0.00			−0.04
			(0.07)			(0.01)			(0.10)
Treated × Post × Large Round			−0.22			0.02			0.03
			(0.38)			(0.02)			(0.21)
Cohort × Firm FE	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Month × Cohort FE	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
# Obs	331,893	330,508	331,893	331,893	330,508	331,893	331,893	330,508	331,893
R^2	0.91	0.91	0.91	0.50	0.50	0.50	0.89	0.89	0.89

This table reports Series B+ placebo estimates from the specification $M_{i,c,t} = \beta VC_{i,c,t} + \chi_{c,i} + \delta_{c,t} + \gamma X_{i,c,t} + \varepsilon_{i,c,t}$, where $VC_{i,c,t}$ is the Series B+ Treated × Post indicator. *Bugs* count, per 1,000 lines of code, patterns that SonarQube classifies as likely to cause incorrect or unexpected behavior, such as invalid operations or failures to handle an input. *High-* and *Low-Severity Reliability Issues* count, per 1,000 lines of code, issues at the stated severity associated with a risk of incorrect behavior. Bugs use the traditional classification; the reliability measures use the alternative classification described in Appendix M.2 and are not a breakdown of Bugs. See Appendix A, Table 17, for full variable definitions.

High Violations is defined from critical-violation density in the month before the placebo event. Large Round refers to the amount raised at the earlier Seed or Series A treatment round. The quality cutoff uses baseline firm–cohort observations of firms that appear as placebo-treated; the round-size cutoff uses one observation per placebo-treated firm.

Note: Standard errors in parentheses. * p<0.1, ** p<0.05, *** p<0.01. Standard errors are clustered by cohort and firm.

Table 32: Series B+ Placebo Difference-in-Differences Estimates for Technical Debt

	Technical Debt per Line ($)		
	(1)	(2)	(3)
Treated × Post	−0.13	−0.15	−0.32
	(0.41)	(0.21)	(0.40)
Post × High Violations (=1)		−0.37	
		(0.62)	
Treated × Post × High Violations (=1)		0.09	
		(0.76)	
Post × Large Round			−0.56
			(0.36)
Treated × Post × Large Round			0.85
			(0.62)
Cohort × Firm FE	Yes	Yes	Yes
Month × Cohort FE	Yes	Yes	Yes
# Obs	331,893	330,508	331,893
R^2	0.84	0.84	0.84

This table reports Series B+ placebo estimates from the specification $M_{i,c,t} = \beta VC_{i,c,t} + \chi_{c,i} + \delta_{c,t} + \gamma X_{i,c,t} + \varepsilon_{i,c,t}$, where $VC_{i,c,t}$ is the Series B+ Treated × Post indicator. *Technical Debt per Line of Code* measures calibrated repair costs in dollars per line of code. The Technical Debt Ratio equals $40/11$ times this outcome on the same observations, so the table reports the dollar unit. See Appendix A, Table 17, for full variable definitions.

High Violations is defined from critical-violation density in the month before the placebo event. Large Round refers to the amount raised at the earlier Seed or Series A treatment round. The quality cutoff uses baseline firm–cohort observations of firms that appear as placebo-treated; the round-size cutoff uses one observation per placebo-treated firm.

Note: Standard errors in parentheses. * p<0.1, ** p<0.05, *** p<0.01. Standard errors are clustered by cohort and firm.

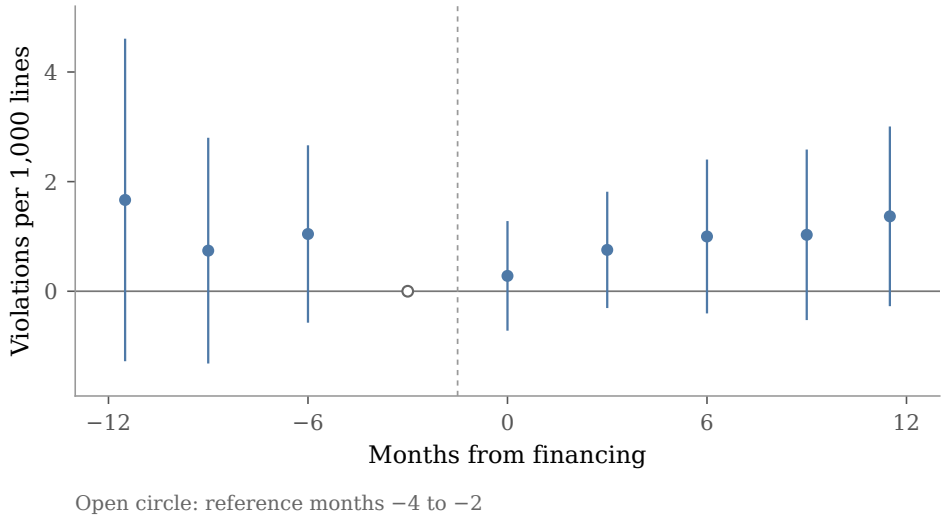


Figure 34: Later Financing: Maintainability Violations per 1,000 Lines of Code. Treated firms complete their first Series B-or-later round. Controls remain without a completed Seed or Series A round through the event window. Quarterly estimates are relative to months −4 to −2; the bin plotted at month 0 spans months −1 to +1. The dashed line marks the end of the reference months. Bars show 95% confidence intervals; standard errors are clustered by cohort and firm.

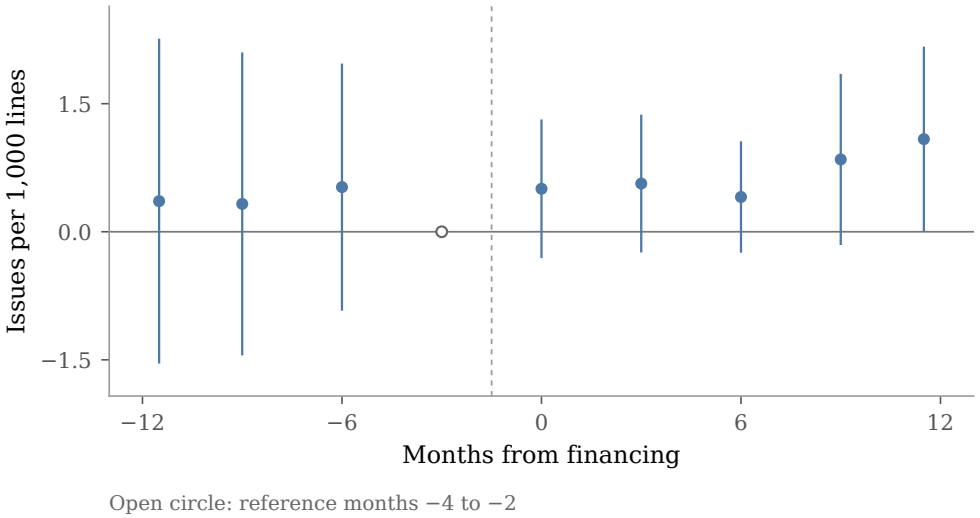


Figure 35: Later Financing: Maintainability Issues (High-Severity) per 1,000 Lines of Code. This measure uses the alternative classification described in Appendix M.2. Treated firms complete their first Series B-or-later round. Controls remain without a completed Seed or Series A round through the event window. Quarterly estimates are relative to months −4 to −2; the bin plotted at month 0 spans months −1 to +1. The dashed line marks the end of the reference months. Bars show 95% confidence intervals; standard errors are clustered by cohort and firm.

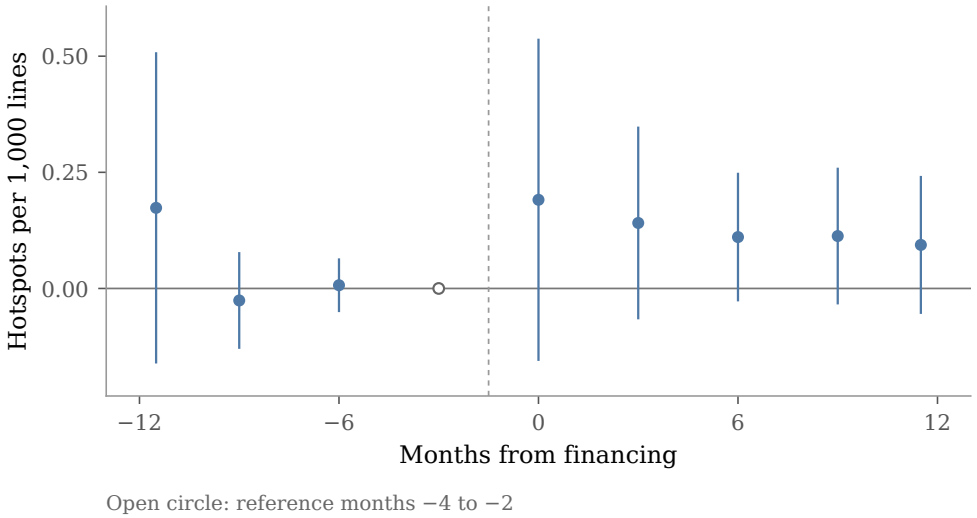
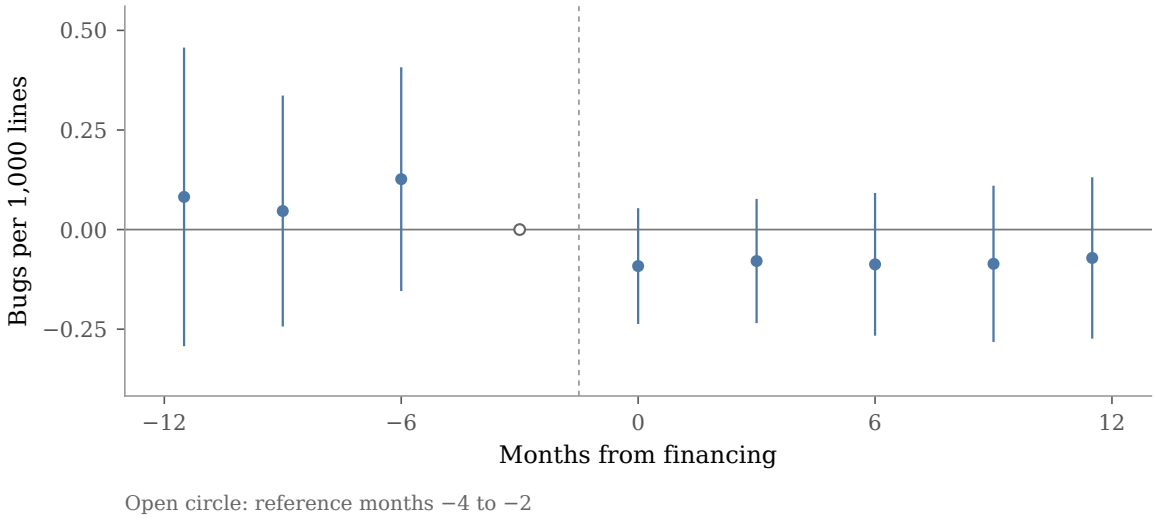
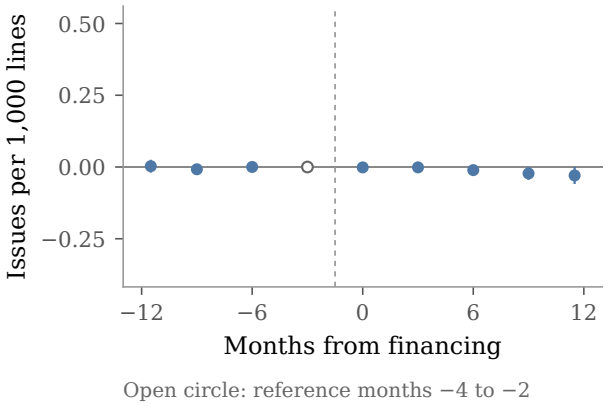


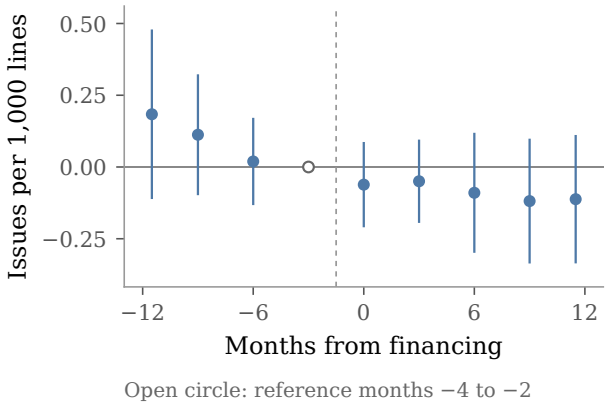
Figure 36: Later Financing: Security Hotspots per 1,000 Lines of Code. Treated firms complete their first Series B-or-later round. Controls remain without a completed Seed or Series A round through the event window. Quarterly estimates are relative to months −4 to −2; the bin plotted at month 0 spans months −1 to +1. The dashed line marks the end of the reference months. Bars show 95% confidence intervals; standard errors are clustered by cohort and firm.



(a) Bugs per 1,000 lines of code



(b) Reliability Issues (High Severity) per 1,000 lines of code



(c) Reliability Issues (Low Severity) per 1,000 lines of code

Figure 37: Later Financing: Bugs and Reliability Issues. Bugs use the traditional classification; the reliability measures use the alternative classification described in Appendix M.2. Treated firms complete their first Series B-or-later round. Controls remain without a completed Seed or Series A round through the event window. Quarterly estimates are relative to months −4 to −2; the bin plotted at month 0 spans months −1 to +1. The dashed line marks the end of the reference months. Bars show 95% confidence intervals; standard errors are clustered by cohort and firm.

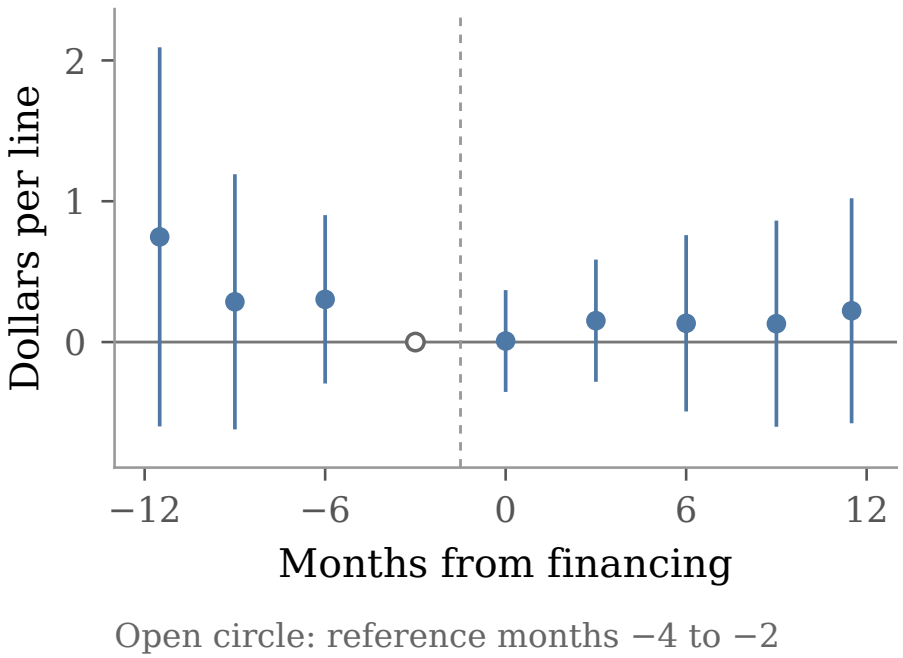


Figure 38: Later Financing: Technical Debt per Line of Code ($). Treated firms complete their first Series B-or-later round. Controls remain without a completed Seed or Series A round through the event window. Quarterly estimates are relative to months −4 to −2; the bin plotted at month 0 spans months −1 to +1. The dashed line marks the end of the reference months. Bars show 95% confidence intervals; standard errors are clustered by cohort and firm.

K Lifecycle and Commercialization: Additional Evidence

This appendix examines whether the changes around first institutional financing coincide with a shift toward commercialization. It complements the commercial-stage comparisons in Section 4.3 with LinkedIn employment records and commit histories. We use the estimator in Equation (7). The employment comparisons cover main-sample firms with LinkedIn coverage. The commit-history comparisons cover main-sample firms with commit histories and pool each firm's covered repositories. After restricting to observed commit measures and removing fixed-effect singletons, the commit-message regressions contain 300,380 stacked observations from 371 firms, including 213 treated firms, in 89 cohorts; the changed-file regressions contain 302,768 stacked observations from 373 firms, including 215 treated firms, in 89 cohorts. Each table reports its own coverage.

K.1 Headcount Composition

We use employment records to test whether commercialization shifts resources from engineering toward selling. We estimate Equation (7) on each role group's share of active headcount. Table 33

reports a 2.8 percentage-point increase in the business-development share and a 7.4-point increase in the engineering share ($p < 0.01$ for each). Business-development roles also account for 5.1 percentage points more of monthly hires ($p < 0.01$). The point estimates show commercial expansion alongside a larger increase in engineering's share of employment. Figure 39 traces both shares around financing.

Table 33: Business-Development and Engineering Shares of Headcount and Hires

	Business Development Share of Headcount	**Engineering Share of Headcount**	**Business Development Share of Monthly Hires**
	(1)	(2)	(3)
Treated × Post	2.84***	7.40***	5.06***
	(0.60)	(1.31)	(1.04)
Cohort × Firm FE	Yes	Yes	Yes
Month × Cohort FE	Yes	Yes	Yes
# Obs	260,004	260,004	128,044
# Firms	382	382	355

This table estimates the baseline stacked difference-in-differences specification of Equation (7) on LinkedIn employment spells for main-sample firms with LinkedIn coverage. The outcomes in Columns (1) and (2) are the business-development and software-engineering categories' shares of a firm's active headcount in a month; the two categories are mutually exclusive. Column (3) uses the flow of role starts rather than the stock of active employees. The estimation samples contain 260,004 observations from 382 firms, including 237 treated firms, in 93 cohorts for Columns (1) and (2), and 128,044 observations from 355 firms, including 222 treated firms, in 89 cohorts for Column (3). Joint tests do not reject zero pre-treatment coefficients for these three share outcomes.

Coefficients and standard errors are reported in percentage points.

Note: Standard errors in parentheses. * p<0.1, ** p<0.05, *** p<0.01. Standard errors are clustered by cohort and firm.

Table 34 decomposes the commercial expansion into sales-related and marketing roles and identifies human-resources staff within the residual category. Sales-related roles account for 1.9 percentage points of the increase in headcount share ($p < 0.01$), and marketing for 1.0 points ($p < 0.01$). The human-resources estimates are not statistically significant ($p = 0.67$ for headcount and $p = 0.61$ for hires). The share of all other classified roles falls by 8.4 points ($p < 0.01$), and the sales-related, marketing and all-other shares of monthly hires change in the same direction. These changes indicate joint investment in engineering and commercial capacity, with no transfer of employment share from engineering toward selling.

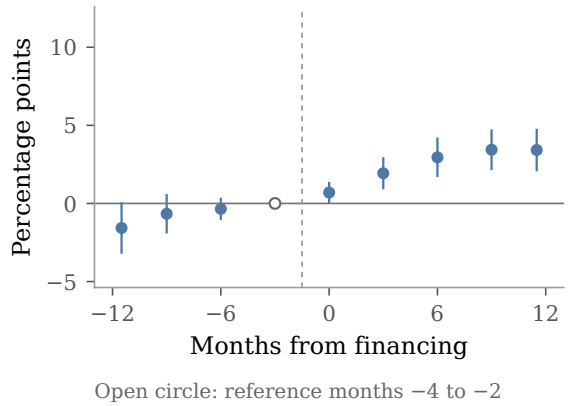
Table 34: Decomposition of the Business-Development Share of Headcount and Hires

	Sales-Related	**Marketing**	**Human Resources**	**All Other Roles**
	(1)	(2)	(3)	(4)
Panel A: Share of active headcount				
Treated × Post	1.87***	0.99***	−0.22	−8.37***
	(0.54)	(0.26)	(0.51)	(1.28)
# Obs	260,004	260,004	260,004	260,004
# Firms	382	382	382	382
Panel B: Share of monthly hires				
Treated × Post	2.55**	2.51***	0.29	−9.95***
	(1.03)	(0.48)	(0.57)	(2.06)
# Obs	128,044	128,044	128,044	128,044
# Firms	355	355	355	355
Cohort × Firm FE	Yes	Yes	Yes	Yes
Month × Cohort FE	Yes	Yes	Yes	Yes

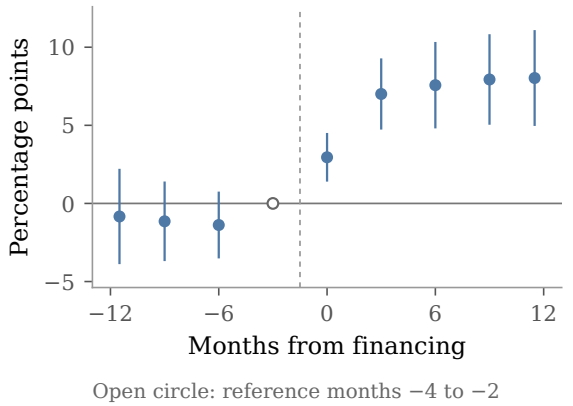
This table decomposes the business-development share from Table 33, using the same stacked difference-in-differences specification, sample, and employment records. Columns (1) and (2) split the business-development group into sales-related roles (sales, account management, sales development, business-development titles, partnerships, customer success, go-to-market, and revenue operations) and marketing roles (marketing, product marketing, growth, and demand generation); employees who hold roles in both components are counted in each. Column (3) is a human-resources group (human resources, recruiting, talent acquisition, and people operations) separated from the residual category, and Column (4) is the remaining residual. Engineering-management roles are excluded from every group, following the role classification in Table 33, so the reported shares and the engineering share do not sum to one. Panel A uses each group's share of a firm's active headcount; Panel B uses the share of monthly role starts. Joint tests do not reject zero pre-treatment coefficients for the reported outcomes.

Coefficients and standard errors are reported in percentage points.

Note: Standard errors in parentheses. * p<0.1, ** p<0.05, *** p<0.01. Standard errors are clustered by cohort and firm.



(a) Business-development share of headcount

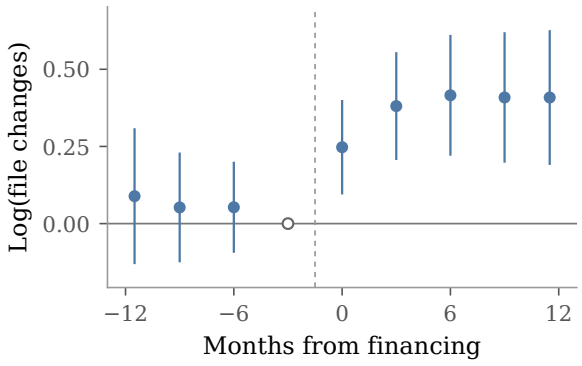


(b) Engineering share of headcount

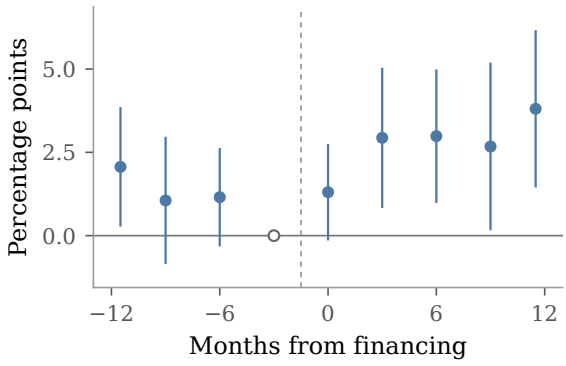
Figure 39: Headcount Composition. Main-sample firms with LinkedIn coverage; shares in percentage points. Quarterly estimates are relative to months −4 to −2; the bin plotted at month 0 spans months −1 to +1. The dashed line marks the end of the reference months. Bars show 95% confidence intervals; standard errors are clustered by cohort and firm.

K.2 Commercialization in the Codebase

We also examine commercialization within the software asset using commit messages, the names and folder locations of changed files, and dependency manifests—the files that list the external software packages used by a project. Table 16 shows no statistically significant increase in messages mentioning billing, payments, metering, subscriptions, or upgrades, or in the share of changed files associated with commercial features, payment libraries, enterprise functions, or growth. In the year after financing, 0.4% of repositories add a payment library, while 8.8% add a frontend framework used to build the user-facing interface. Development activity nevertheless expands. The growth rate in monthly file changes is approximately 30 percentage points greater than in controls ($p < 0.01$). Engineers modify files in more top-level folders, and the share of changes to frontend code increases. Figure 40 traces these changes over the event window. The observable expansion is therefore broader than a shift toward the measured monetization features.



(a) Log file changes per month



(b) Frontend share of changed files

Figure 40: Development Activity and the Frontend Share of Changed Files. Main-sample firms with file-change histories; shares in percentage points. Quarterly estimates are relative to months −4 to −2; the bin plotted at month 0 spans months −1 to +1. The dashed line marks the end of the reference months. Bars show 95% confidence intervals; standard errors are clustered by cohort and firm.

Together, the employment and codebase evidence does not support a narrow lifecycle account in which firms shift resources from engineering toward selling at their first round. Engineering gains employment share, development expands, and the public codebase shows no detectable increase in the measured monetization features. Combined with the quality improvements among firms already generating revenue in Section 4.3, these findings indicate that the engineering response extends beyond the initial transition to commercialization.

L Engineering Practice around VC Entry

This appendix examines the changes in development practices behind Section 3.8. We report commit-message components, headquarters-local weekend comparisons, and bug introduction among engineers active before financing. Several message measures use trailers: standardized lines, such as "Signed-off-by:" or "Co-authored-by:", that record review, sign-off, acknowledgement, approval or coauthorship. We link firms to their listed commercialized open-source projects using a fixed company–project mapping, held constant across event months. These analyses aggregate the available contribution histories to the firm level. Outcome coverage depends on the histories and measurements available for each firm. Table 35 reports observations, firms, treated firms and cohorts separately for each analysis.

Table 35: Sample Coverage of Contribution-Based Comparisons

Comparison	Observations	Firms	Treated	Cohorts
Primary message-marker comparisons	300,380	371	213	89
Primary headquarters-local weekend share	171,116	273	199	85
Primary bug introduction by pre-financing engineers: rate and components	450,203	451	249	98
Primary fixed-roster pre/post comparisons	37,938	450	245	96
Common sample for core outcomes	260,714	343	193	86
Common sample for broader outcomes	151,957	252	181	82
Changed-file comparisons	302,768	373	215	89
Firm-wide bug-introduction rate	457,480	467	265	98
Post-entry newcomer gap	198,118	448	246	98
Headquarters-local weekend repair comparison	4,026,140	284	—	—

All firms represented in this table belong to the main 484-firm sample. The primary analyses and the changed-file comparisons pool available contribution histories across each firm's covered repositories. Their samples vary with outcome measurement and contribution availability. The core and broader common samples use identical firm–cohort–month observations across their included outcomes (Table 36). Attribution measurement is available for 469 firms before the pre-financing-engineer and cohort restrictions.
The fixed-roster row counts one observation before and one after financing for each firm–cohort unit; the weekend repair row counts contributions and has no financing-cohort treatment. Other rows count firm–cohort–months. The primary analyses exclude singleton observations and cohorts without both treated and control firms. The firm-wide bug-introduction, newcomer and weekend-repair comparisons use their own samples.

More frequent contribution documentation and lower bug-introduction and critical-violation rates persist when we estimate the outcomes on identical firm–cohort–month observations (Table 36). In the comparison that includes additional process measures, the shares of contributions referencing numbered records or pull requests also rise, and the weekend share falls.

Table 36: Process and Engineering Quality on Identical Observations

Measure	Core estimate	p-value	Broader estimate	p-value
Contribution documentation (combined)	3.00***	< 0.01	3.01***	< 0.01
	(0.98)		(1.05)	
Developer sign-offs	1.89*	0.05	1.86*	0.07
	(0.96)		(1.02)	
Coauthor credits	1.30**	0.01	1.36**	0.01
	(0.54)		(0.57)	
Review acknowledgements	−0.03	0.27	−0.04	0.26
	(0.02)		(0.04)	
Pre-financing-engineer bug introduction	−1.70**	0.01	−1.40*	0.05
	(0.68)		(0.73)	
Critical violations per 1,000 lines	−2.21**	0.04	−2.44**	0.03
	(1.08)		(1.11)	
Numbered issue references	—	—	4.54**	0.03
			(2.14)	
Pull-request references	—	—	5.12**	0.01
			(2.13)	
Merge-or-squash markers	—	—	5.28***	< 0.01
			(1.98)	
Headquarters-local weekend share	—	—	−1.54**	0.01
			(0.64)	
Explicit pull-request URLs	—	—	0.07	0.29
			(0.07)	
Observations	260,714		151,957	
Firms / treated firms / cohorts	343 / 193 / 86		252 / 181 / 82	

Every outcome within a column uses the same firm–cohort–month observations, excluding singleton observations and cohorts without both treated and control firms. The core common sample combines trailer components, bug introduction among pre-financing engineers and critical violations. The broader common sample additionally requires numbered issue references, pull-request references, merge-or-squash markers, explicit pull-request URLs and headquarters-local weekend shares. Control-role counts are 295 and 211; roles can overlap across cohorts. Shares and bug-introduction rates are percentage points; critical violations are counts per 1,000 lines. Both columns use firm-by-cohort and calendar-month-by-cohort fixed effects.

Coefficients and standard errors are in percentage points unless otherwise stated. Standard errors in parentheses are clustered by cohort and firm. * $p < 0.10$, ** $p < 0.05$, *** $p < 0.01$.

L.1 Engineering Process Measures

We use commit messages to examine how engineers document and coordinate changes. The primary comparisons report contribution-documentation markers, issue and pull-request references, merge-or-squash markers and headquarters-local weekend activity (Tables 10 and 38). A separate table reports individual message components (Table 37).

The later-round comparison produces a different pattern. Around Series B and later rounds, the issue-reference share, which counts numbered references and numbers following words such as "fixes" or "refs," changes by −0.3 percentage points and the merge-or-squash share by −1.8 percentage

points; neither change is statistically significant ($p = 0.90$ and $p = 0.42$, respectively). The latter compares with a 6.5-point increase at first entry (Table 38). The integration marker therefore does not show recurring gains with each financing round, consistent with an initial change in how development is organized under institutional backing.

Table 37: Message-Marker Components on Identical Observations

Measure	Estimate (pp)	p-value
Combined trailer share	3.06***	< 0.01
	(0.95)	
Reviewed-by	0.02	0.77
	(0.05)	
Signed-off-by	1.86*	0.05
	(0.94)	
Acked-by	0.01	0.31
	(0.01)	
Approved-by	< 0.01	0.32
	(< 0.01)	
Co-authored-by	1.35***	< 0.01
	(0.50)	
Different reviewer/approver email	0.03	0.39
	(0.03)	
Author email in review/approval trailer	−0.02	0.66
	(0.04)	
Explicit pull-request URLs	−0.02	0.75
	(0.08)	
Numbered issue references	5.51***	< 0.01
	(1.89)	
External task-tracking identifiers	1.06**	0.03
	(0.49)	

Every outcome is the fraction of a firm's monthly commits with the corresponding marker, pooling commits across each firm's covered projects: 300,380 observations, 371 firms, 213 treated firms and 89 cohorts. The different-email category requires a Reviewed-by, Acked-by or Approved-by line containing an email other than the commit author's normalized email. Explicit pull-request URLs are matched separately from generic numbered references and external task-tracking identifiers. Markers can overlap within a commit. Coefficients and standard errors are reported in percentage points.
Note: Standard errors in parentheses, clustered by cohort and firm. * p<0.1, ** p<0.05, *** p<0.01.

Table 38: Commit-Message Workflow Comparisons

Measure	Estimate (pp)	p-value
Numbered issue references	5.51***	< 0.01
	(1.89)	
Pull-request references	6.23***	< 0.01
	(1.89)	
Tracker references (any type)	5.96***	< 0.01
	(1.91)	
Merge-or-squash markers	6.47***	< 0.01
	(1.76)	

Monthly shares pool qualifying contributions across the mapped projects covered for each firm. Numbered references match a hash sign followed by digits anywhere in a message. The pull-request-reference marker matches a parenthesized numbered reference such as `(#123)`, the words "pull request" separated by spaces or line breaks, or `/pull/` followed by digits, ignoring case. Explicit pull-request URLs are a separate measure in Table 37. The tracker-reference marker matches issue-tracker keys such as `ABC-123`, a number following a closing word such as "fixes" or "resolved," a link to a hosted issue tracker, or a pull-request reference. The merge-or-squash marker equals one when a commit is a recorded merge or when its first message line ends in a parenthesized number such as `(#123)`, the default title of a pull request consolidated into a single commit. Each outcome uses 300,380 identical firm–cohort–month observations, 371 firms, 213 treated firms, 308 control-role firms and 89 cohorts. Firm-by-cohort and calendar-month-by-cohort fixed effects are absorbed, and singleton observations and cohorts without both treated and control firms are excluded.
Coefficients and standard errors are percentage points. Standard errors in parentheses are clustered by cohort and firm. * $p < 0.10$, ** $p < 0.05$, *** $p < 0.01$.

Commit records provide a separate measure of merges, which join parallel versions of the code. The monthly merge-commit share changes by +0.5 percentage points ($p = 0.52$; not statistically significant; 95% interval $[-1.0, +1.9]$), using 300,380 observations, 371 firms, 213 treated firms and 89 cohorts. This outcome counts recorded merges alone, whereas the merge-or-squash marker in Table 38 also counts pull requests consolidated into a single commit.

L.2 Weekend Contributions and Subsequent Repairs

We compare weekend and weekday contributions using Saturdays and Sundays at each firm's reported headquarters. We use the time each contribution was committed, converted to the time zone of the firm's headquarters and accounting for daylight-saving changes. The outcome records linkage to a bug fix within 730 elapsed days. The regression compares contributions by the same developer in the same repository, with calendar-month fixed effects and controls for message-length deciles. Table 39 reports a 0.7-percentage-point higher repair probability for weekend contributions ($p < 0.01$).

Table 39: Headquarters-Local Weekend Contributions and Subsequent Repairs

Dep Var:	**Bug repair within 730 days**
Headquarters-local weekend	0.67***
	(0.15)
Author × Repository FE	Yes
Calendar-month FE	Yes
Message-length deciles	Yes
# Obs	4,026,140
# Firms / Repositories	284 / 356
# Authors	62,249

The dependent variable equals one when a non-merge, non-bot contribution is linked to a bug fix within 730 elapsed days. The contribution sample ends in June 2024, and we track associated bug fixes through June 2026. Weekend equals Saturday or Sunday at the firm's reported headquarters; weekday is the comparison category. Conversion uses the time the contribution was committed and the headquarters time zone, accounting for daylight-saving changes. The model includes author-by-repository and calendar-month fixed effects and message-length-decile controls. The estimation sample contains 4,026,140 contributions, 284 firms, 356 repositories and 62,249 authors. The coefficient and standard error are reported in percentage points.
Note: Standard errors in parentheses, clustered by firm. * p<0.1, ** p<0.05, *** p<0.01.

L.3 Bug Introduction Among Pre-Financing Engineers

We trace bug-fix commits to the earlier contributions that last modified the repaired lines, using `git blame`, the version-control operation that identifies the last recorded editor of each line. The resulting numerator counts attribution links, once per introducing commit, rather than repaired lines. Fix commits are identified by bug-fix message markers, excluding merge and squash commits. Attribution ignores differences in spacing and follows moved or copied code; fixes touching more than 30 files or deleting more than 1,000 lines are excluded. We classify engineers as pre-financing if their first contribution precedes the firm event, or the cohort event for controls. Newcomers first contribute at or after that event.

We exclude merge commits and accounts identified as bots from their names and email addresses. Across a firm's projects, we link developers using recorded email addresses or normalized usernames extracted from GitHub's anonymized email addresses when repository records support the match; otherwise, we keep their identities separate across repositories. We classify engineers using their first recorded contribution across the firm's included projects, rather than a LinkedIn employment match.

Table 40 reports a 1.6-percentage-point decline in bug introduction among engineers active before financing, or 10.6% of the treated pre-treatment mean ($p < 0.01$). The within-engineer component accounts for 0.9 points ($p < 0.01$), while changes in engineers' participation and shares of work account for 0.6 points ($p = 0.07$). All three estimates use the same 450,203 observations, 451

firms, 249 treated firms and 98 cohorts. Figure 41 traces the bug-introduction rate among pre-financing engineers around the financing event. The firm-wide rate declines by 1.1 points ($p = 0.02$).

In a separate comparison, we subtract the rate for pre-financing engineers from newcomers' rate and compare this difference between treated and control firms in post-event months with both groups observed. The difference between treated and control firms is not statistically significant ($p = 0.83$). The classification uses each firm's own cohort event and the first contribution month. With r^N_{ice} and r^I_{ice} denoting the newcomer and pre-financing-engineer rates, we estimate $r^N_{ice} - r^I_{ice} = \gamma \mathbf{1}\{E_i = c\} + \lambda_{ce} + u_{ice}$ for $e \geq 0$, retaining a firm–cohort–month only when both groups contributed in that month. Thus γ is the treated-versus-control difference in the observed post-entry group gap.

Table 40: Bug Introduction Among Pre-Financing Engineers and Post-Entry Group Gaps

Measure	Estimate (pp)	p-value	# Obs	# Firms	# Treated
Pre-financing engineers, total	−1.57*** (0.54)	< 0.01	450,203	451	249
Within-engineer component	−0.93*** (0.32)	< 0.01	450,203	451	249
Work-composition component	−0.65* (0.36)	0.07	450,203	451	249
Post-entry newcomer gap	0.09 (0.45)	0.83	198,118	448	246
Firm-wide rate	−1.14** (0.51)	0.02	457,480	467	265

The bug-introduction rate is the fraction of an engineer group's commits identified by tracing later bug fixes to earlier contributions, at calendar-month lags of 0–24. Pre-financing engineers first appear across included projects before the firm event cutoff; controls are classified relative to their cohort event. The total and its two additive components use identical monthly observations, firm-by-cohort and month-by-cohort fixed effects, and 98 cohorts. The decomposition compares each month's observed engineer rates and shares with the group's recorded pre-event history. For engineers active at both endpoints, it averages the endpoint shares for the rate component and the endpoint rates for the share component, splitting the rate–share interaction equally. Activity observed at only one endpoint is allocated to composition. The components sum to the primary total; inference retains their joint covariance. The treated pre-treatment mean of the total rate is 14.9%. Event windows end by June 2024. The post-entry gap uses only months with both groups observed and regresses newcomers' rate minus the pre-financing-engineer rate on treated status with cohort-by-calendar-month fixed effects, in 98 cohorts. Newcomer status uses the firm's own cohort event and the engineer's first contribution month. The firm-wide rate pools engineers and uses 98 cohorts. Contributions and introducing commits are pooled across each firm's projects, counting each commit once. Zero contribution denominators yield missing rates. Cohorts lacking either treated or control firms are excluded in every comparison. Coefficients and standard errors are reported in percentage points.
Note: Standard errors in parentheses, clustered by cohort and firm. * p<0.1, ** p<0.05, *** p<0.01.

Fixed-roster comparisons. Table 41 compares the same pre-financing engineers before and after financing. The share of contributions introducing bugs declines by 1.1 percentage points when each engineer receives equal weight (7.8% of the pre-treatment mean; $p = 0.05$) and by 2.3 points when engineers are weighted by their contribution shares before financing (14.7% of the pre-treatment mean; $p < 0.01$). These comparisons complement the monthly decomposition by holding both the team and the engineers' weights fixed.

Table 41: Bug Introduction with the Pre-Financing Engineer Roster Fixed

Weighting	Estimate (pp)	95% interval (pp)	Observations
Equal engineer weights	−1.09*	[−2.20, 0.02]	37,938
	(0.56)		
Fixed pre-financing contribution weights	−2.34***	[−3.42, −1.26]	37,938
	(0.54)		

An engineer enters the fixed roster if active before the cohort event and if total contributions are positive in each of two periods: months −12 through −1 and months 0–12. We compute each engineer's bug-introduction rate in each phase counting an introducing commit once if any recorded fix qualifies at calendar-month lags 0–24, then subtract the pre rate from the post rate. Aggregation either weights these same engineers equally in both phases or uses their pre-phase commit shares in both phases. The comparisons retain 450 firms, 245 treated firms and 96 cohorts, and event windows end by June 2024. Each of the 18,969 firm–cohort units contributes one observation before and one after financing. Coefficients, standard errors and intervals are percentage points; standard errors are clustered by cohort and firm. * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$.

Firm-wide rates. Across 457,480 observations, firm-wide bug-introduction rates measured over 12 calendar months change by −0.9 percentage points ($p = 0.08$), over 24 calendar months by −1.1 points ($p = 0.02$) and over 730 days by −1.1 points ($p = 0.03$). These comparisons use the non-merge commits of all contributors.

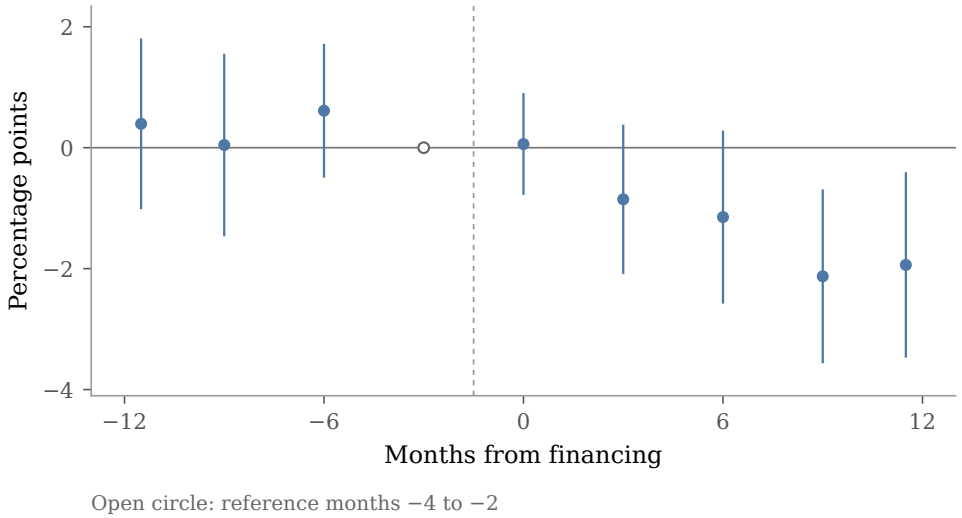


Figure 41: Bug Introduction Among Pre-Financing Engineers. The rate is the share of contributions linked to a later bug fix among engineers active before the financing event. Treated firms complete their first Seed round, or their first Series A round when no completed Seed is recorded; controls remain without a qualifying first round through the event window. Shares in percentage points. Quarterly estimates are relative to months −4 to −2; the bin plotted at month 0 spans months −1 to +1. The dashed line marks the end of the reference months. Bars show 95% confidence intervals; standard errors are clustered by cohort and firm.

Together, the findings on documentation, weekend activity and existing engineers support the

professionalization interpretation. Existing engineers become less likely to introduce bugs, reflecting improvements within engineers and changes in who does how much work.

M Design Sensitivity, SonarQube Classifications and Engineer Retention

This appendix reports sensitivity to the event window, treatment timing, and control reuse, followed by a comparison of SonarQube's two classifications, and evidence on pre-financing-engineer retention.

M.1 Design and Inference Sensitivity

Table 42 collects the principal variations of the baseline design. The tests address three distinct concerns: sensitivity to observations immediately around the deal, the period over which firms are compared, and the weight placed on repeatedly used controls. Estimates remain negative when the months around financing are excluded, under alternative windows, and under inverse-frequency weighting. The persistence across these choices indicates that the quality improvement is not confined to the financing month or to one construction of the comparison sample.

Table 42: Additional Design Sensitivity

Specification	Critical Violations	Technical Debt	Violations
Baseline (±12 months)	−1.85**	−1.57**	−4.74**
	(0.86)	(0.77)	(2.02)
Excluding months −1 to +1	−1.97**	−1.73**	−5.20**
	(0.94)	(0.85)	(2.22)
Excluding months −2 to +2	−2.06**	−1.84**	−5.62**
	(0.99)	(0.91)	(2.43)
±6 months	−1.71**	−0.92*	−2.47**
	(0.78)	(0.48)	(1.03)
±9 months	−1.85**	−1.33**	−3.88**
	(0.80)	(0.61)	(1.50)
±18 months	−2.25**	−2.23**	−6.14**
	(0.99)	(1.01)	(2.75)
±24 months	−2.30*	−1.71**	−4.33**
	(1.16)	(0.81)	(1.77)
Inverse control-frequency weights	−1.98**	−1.83**	−5.29**
	(0.89)	(0.92)	(2.52)

Critical Violations and Violations are counts per 1,000 lines of code; Technical Debt is calibrated dollars per line. The exclusion rows omit the financing month and one or two months on each side of it. Inverse-frequency weights down-weight controls in proportion to the number of cohorts in which they appear. Standard errors are clustered by cohort and firm. * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$.

Across 99 leave-one-cohort-out estimates, the coefficient ranges are $[-2.00, -1.25]$ for Critical Violations, $[-1.67, -1.06]$ for Technical Debt, and $[-5.00, -3.15]$ for Violations; every estimate is negative. Table 54 reports pointwise and familywise baseline tests.

M.2 Traditional and Alternative Classifications

SonarQube's traditional classification assigns issues to bugs, vulnerabilities or "code smells," the vendor's term for maintainability violations. The alternative classification is SonarQube's Multi-Quality Rule (MQR) classification. It classifies a rule's impacts on reliability, security and maintainability. A rule can affect more than one quality, so the alternative classification's domain counts overlap and should not be summed as unique violations. Both classifications are available in the same scans. Critical violations under the traditional classification and high-severity issues under the alternative classification provide closely aligned measures of serious violations, while estimated repair time for maintainability violations can differ between the classifications.

SonarQube's traditional measures report critical violations and each issue type separately, but not critical violations by type. The alternative classification reports high-severity issues by the quality

they affect, and its high-severity count closely matches critical violations: before financing, treated firms average 12.41 high-severity issues and 12.40 critical violations per 1,000 lines of code, and Table 43 shows that the two yield nearly identical estimates. Maintainability issues account for 12.29 of these 12.41 high-severity issues. The traditional counts alone imply that maintainability violations make up at least 83% of critical violations, because treated firms average only 2.06 bugs and security vulnerabilities combined across all severity levels.

Table 43 compares the outcomes on identical baseline and matched observations. The critical and high-severity estimates are both −1.85 in the baseline and −2.32 under the preferred matching specification, and the maintainability-count estimates differ by less than 0.1 violation per 1,000 lines. Baseline calibrated repair cost declines by $1.57 per line under the traditional classification and $1.46 under the alternative classification, with a standard error of $0.77 in both cases. Under the preferred matching specification, the corresponding declines are $1.29 ($p = 0.19$) and $1.25 ($p = 0.21$); neither estimate is statistically significant. The paired difference has $p = 0.26$ in the baseline and $p = 0.67$ after matching (neither difference is statistically significant). Across these classifications, the estimated quality gains are closely aligned in magnitude, direction and uncertainty.

Table 43: Traditional and Alternative Classifications on Identical Samples

Outcome	Traditional classification	Alternative classification	Difference p
Panel A: Baseline			
Critical / high severity	−1.85**	−1.85**	0.15
	(0.86)	(0.86)	
Maintainability violations	−4.80**	−4.76**	0.48
	(1.99)	(1.99)	
Repair cost, dollars per line	−1.57**	−1.46*	0.26
	(0.77)	(0.77)	
Panel B: Preferred matching			
Critical / high severity	−2.32*	−2.32*	0.44
	(1.22)	(1.22)	
Maintainability violations	−4.46**	−4.41**	0.55
	(2.11)	(2.11)	
Repair cost, dollars per line	−1.29	−1.25	0.67
	(0.98)	(0.98)	

Critical violations under the traditional classification correspond to high-severity issues under the alternative classification. For maintainability, the traditional classification counts issues classified as code smells; the alternative classification counts maintainability impacts. Counts are per 1,000 lines of code. Repair cost uses the same fixed calibration under both classifications and is dollars per line. Each pair uses identical observations, regression design and calibration; Difference p uses the alternative estimate minus the traditional estimate and their paired covariance. The baseline has 487,577 observations, 483 firms, 273 treated firms and 99 cohorts; matching has 71,837 observations, 249 firms, 110 treated firms and 68 cohorts. Standard errors in parentheses are clustered by cohort and firm. * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$.

M.3 Continued Contribution

We test whether the improvement among pre-financing engineers reflects the selective retention of engineers whose prior commits were less frequently linked to later fixes. Fixing the engineer cohort at $t = -1$ and controlling for pre-period activity, we find no statistically significant change in 12-month retention ($p = 0.29$). A direct comparison of retained engineers with leavers shows no statistically significant treated-firm difference in pre-period bug-introduction rates ($p = 0.91$). The fixed-roster comparisons in Table 41 hold the engineer roster and pre-financing contribution weights fixed and retain a decline in bug-introduction rates.

N Observation Support, Aggregation and Subgroup Comparisons

N.1 Observation Support and Aggregation

Table 44 traces sample construction from the initial universe of 649 firms. The company–repository mapping links 609 firms to 703 distinct repositories; 569 firms supply covered codebase months, and 484 enter the stacked panel. The 274 treated and 412 control roles overlap because firms can serve as controls before their own financing. Fixed-effect singleton deletion leaves 273 treated firms in the baseline estimation sample.

Table 44: Sample Construction and Regression Coverage

Sample stage	Firms	Difference from preceding row
Initial universe (OSSRank)	649	—
Company–repository mapping	609	40
At least one month with positive lines of code	569	40
Estimation sample before singleton deletion	484	85
Estimation sample after singleton deletion	483	1

The mapping links 703 distinct repositories. Differences are counts between the listed groups of firms. Before singleton removal, the stacked panel has 274 treated and 412 control firms in 99 cohorts. The estimation sample has 273 treated and 412 control firms in 99 cohorts, after deleting 18 fixed-effect singleton observations. Treated and control roles can overlap across cohorts.

Table 46 reports the observation and repository counts by event month. Table 45 compares the baseline DiD coefficient with complete observation windows, paired changes, nonnegative treated-firm aggregation, and fixed repository sets. Each restriction defines its own sample.

Table 45: Quality Estimates by Observation Support and Aggregation

Specification	Critical	Total	Debt	Maintainability	N	Treated firms	Cohorts
Baseline	−1.85**	−4.74**	−1.57**	−4.80**	487,577	273	99
	(0.86)	(2.02)	(0.77)	(1.99)			
Observed before and after	−1.85**	−4.74**	−1.57**	−4.80**	483,358	253	98
	(0.86)	(2.02)	(0.77)	(1.99)			
Complete 25-month windows	−2.19**	−5.61**	−1.87**	−5.60**	375,175	182	90
	(1.05)	(2.45)	(0.89)	(2.42)			
Balanced, treated-firm weights	−2.19**	−5.58**	−1.87**	−5.57**	15,007	182	90
	(1.05)	(2.43)	(0.88)	(2.40)			
Paired changes from month −1	−2.37**	−3.46**	−1.26	−3.44**	248,792	244	95
	(1.16)	(1.56)	(0.76)	(1.51)			
Fixed baseline repositories	−1.89**	−5.07**	−1.70**	−5.09**	476,758	247	98
	(0.87)	(2.04)	(0.78)	(2.02)			
Fixed baseline repositories, all observed	−2.01**	−5.21**	−1.77**	−5.24**	458,153	247	98
	(0.90)	(2.10)	(0.80)	(2.08)			
Full codebase, baseline repositories observed	−1.99**	−5.00**	−1.68**	−5.06**	458,153	247	98
	(0.90)	(2.10)	(0.80)	(2.07)			

Critical, total and maintainability outcomes are issue counts per 1,000 lines of code; debt is calibrated dollars per line. Observed before and after requires at least one observed month on each side. Complete windows require every month −12 through +12. Balanced aggregation subtracts each firm's mean in months −12 to −1 from its mean in months 0–12, with cohort weights proportional to treated-firm counts. Its N counts firm–cohort units. Paired changes compare each post month with the same firm's month −1 observation, give post months equal weight and fix cohort weights at baseline treated-firm counts; its N counts paired post observations. Repository sets are fixed to repositories observed with positive lines at month −1. The last two rows require each baseline repository to remain observed; the last row measures the firm's full codebase on those observations. Cohorts without both treated and control firms are dropped after each restriction.

Note: Standard errors in parentheses, clustered by cohort and firm. * p<0.1, ** p<0.05, *** p<0.01.

Table 46: Codebase Coverage by Event Month

Event month	Treated observations	Control observations	Control firms	Repos, treated	Repos, control
−12	199	17,057	367	1.4	1.7
−11	201	17,281	369	1.4	1.7
−10	208	17,528	376	1.4	1.7
−9	217	17,806	377	1.4	1.7
−8	221	18,060	380	1.4	1.7
−7	223	18,225	382	1.4	1.7
−6	226	18,434	384	1.4	1.7
−5	232	18,595	390	1.4	1.7
−4	237	18,755	392	1.4	1.7
−3	235	19,024	399	1.4	1.7
−2	241	19,228	402	1.4	1.7
−1	247	19,686	410	1.4	1.7
0	245	19,414	410	1.4	1.6
1	245	19,347	409	1.4	1.6
2	245	19,364	409	1.4	1.6
3	245	19,336	409	1.4	1.6
4	241	19,349	408	1.4	1.6
5	243	19,332	408	1.4	1.6
6	246	19,319	408	1.4	1.6
7	245	19,320	407	1.4	1.6
8	244	19,302	409	1.4	1.6
9	245	19,307	409	1.4	1.6
10	241	19,314	409	1.4	1.6
11	245	19,267	408	1.4	1.6
12	243	19,248	408	1.4	1.6

Counts cover firm–cohort–months with an observed codebase and positive lines for which a repository set can be defined at month −1. A treated firm enters its own cohort once, so treated observations also count treated firms. Controls can appear in several cohorts in the same event month; control observations count those cohort appearances, while control firms counts unique firms. Repos is the mean number of observed baseline repositories per firm–cohort observation. A firm–month counts as observed when any of its repositories is observed; this table does not require every baseline repository to be observed. The all-repositories-observed restriction is reported separately in Table 45.

N.2 Repository and Language Composition

A firm's aggregate violation density reflects both quality within its repositories and their relative sizes. The density can fall if violations decline within repositories or if repositories with fewer violations account for more of the observed codebase. To examine the first channel, we hold each repository's share of lines of code at its value in the month before financing. Quality can then change within the pre-financing repositories while their weights in the firm aggregate remain fixed.

Panel A of Table 47 compares the current-share and fixed-share measures on the same firm–cohort–month observations. We retain a month only when every repository observed in the month before financing remains observed. Critical violations decline by 2.0 per 1,000 lines with current shares ($p = 0.02$) and by 1.8 with fixed repository shares ($p = 0.04$). Total violations also decline

under both measures ($p = 0.01$ for each), as does technical debt ($p = 0.03$ for each). The estimated response of the difference between the current-share and fixed-share measures is −0.18 for critical violations and is not statistically significant ($p = 0.29$). The quality gains therefore persist with the pre-financing repository mix.

We next ask whether quality improves among firms whose main programming language remains unchanged. We define the main language as the language accounting for the most observed lines and retain firm–cohorts for which it is the same in every observed month of the event window. Panel B of Table 47 reports this comparison on its own sample. Critical violations decline by 1.4 per 1,000 lines ($p = 0.03$) and total violations by 2.3 ($p = 0.02$). The technical-debt estimate is negative but not statistically significant ($p = 0.16$). This restriction holds the identity of the main language fixed while allowing the shares of other languages to change.

Table 47: Quality Estimates under Repository and Language Restrictions

Comparison	**Critical**	**Total**	**Debt**	**N**	**Treated firms**	**Cohorts**
Panel A: Fixed repository shares, identical observations						
Current line shares	−1.99**	−5.00**	−1.68**	458,153	247	98
	(0.90)	(2.10)	(0.80)			
Fixed pre-financing shares	−1.82**	−5.04**	−1.70**	458,153	247	98
	(0.88)	(2.09)	(0.80)			
Current minus fixed shares	−0.18	0.04	0.02	458,153	247	98
	(0.17)	(0.22)	(0.10)			
Panel B: Unchanged main language, separate sample						
Unchanged main language	−1.44**	−2.33**	−0.57	353,940	217	92
	(0.68)	(1.02)	(0.41)			

Critical and total violations are counts per 1,000 lines of code; debt is calibrated dollars per line of code. Panel A uses identical firm–cohort–month observations for all three rows, retaining a month only when every repository with positive lines at month −1 remains observed. The current-share measure weights repositories by their current line shares. The fixed-share measure weights each pre-financing repository's current density by its share of lines at month −1; repositories absent at month −1 receive zero weight. The difference row estimates the treatment response of the current-share measure minus the fixed-share measure on those observations. Panel B is estimated separately on firm–cohorts whose recorded main (dominant) language, the language with the most observed lines, is unchanged across all observed event-window months. This restriction does not require an observation in every month. Both panels include cohort-by-firm and cohort-by-calendar-month fixed effects and exclude cohorts without both treated and control firms; the treated counts are firms in the estimation sample.

Note: Standard errors in parentheses, clustered by cohort and firm. * p<0.1, ** p<0.05, *** p<0.01.

Finally, Table 48 measures changes in the language mix of the observed codebase. Each outcome is a language family's share of total lines of code. The JavaScript/TypeScript share falls by 0.9 percentage points after financing relative to controls ($p = 0.04$). The other shares show no statistically significant change.

Table 48: Changes in Programming-Language Shares of the Observed Codebase

Language family	**Percentage points**	Language family	**Percentage points**
JavaScript/TypeScript	−0.89**	Rust	−0.01
	(0.44)		(0.20)
Python	0.13	Ruby	0.02
	(0.51)		(0.12)
Java/JVM	0.22	Web markup	0.02
	(0.18)		(0.36)
C/C++	0.35	Other	−0.34
	(0.22)		(0.28)
Go	0.51		
	(0.40)		

Each outcome is a language family's count of lines of code divided by total lines of code in the covered firm–month codebase. Coefficients and standard errors are percentage points. The nine mutually exclusive families sum to the observed lines, so the treatment coefficients sum to zero. All estimates use the same 487,577 observations, 273 treated firms and 99 cohorts, with the baseline fixed effects. Java/JVM comprises Java, Kotlin and Scala, which run on the Java Virtual Machine (JVM).
Note: Standard errors in parentheses, clustered by cohort and firm. * p<0.1, ** p<0.05, *** p<0.01.

N.3 Baseline Quality and Concentration

The high-versus-low critical-violation difference remains negative when the threshold is defined from one baseline observation at each treated firm's own financing or from average pre-financing quality. Table 49 reports these group effects, and Table 50 reports the number of treated firms and the 95% interval for each group. Table 49 also reports the full sequence excluding the firms with the highest month −1 critical-violation densities. Removing the seven highest-density treated firms changes the critical-violation DiD coefficient from −1.85 ($p = 0.03$) to −0.64 ($p = 0.11$; not statistically significant), a 65.2% reduction in its magnitude. The average decline is therefore sensitive to firms with high initial critical-violation density.

Table 49: Initial Quality and the Concentration of Improvements

	Critical violations			Total violations		
Quality classification	**Low**	**High**	**Difference**	**Low**	**High**	**Difference**
Panel A: Response by baseline critical-violation density						
Baseline cutoff	−0.28	−4.14**	−3.87**	−4.60	−5.41**	−0.81
	(0.39)	(1.85)	(1.89)	(2.98)	(2.40)	(3.74)
Own-financing baseline cutoff	−0.28	−3.55**	−3.27*	−5.06	−4.74**	0.32
	(0.42)	(1.64)	(1.69)	(3.36)	(2.13)	(3.89)
Six-month pre-financing average	−0.12	−3.73**	−3.62**	−4.80	−4.91**	−0.12
	(0.18)	(1.66)	(1.67)	(3.27)	(2.16)	(3.83)
Average (at least three observed pre-financing months)	−0.09	−3.63**	−3.54**	−4.86	−4.79**	0.07
	(0.17)	(1.69)	(1.70)	(3.31)	(2.20)	(3.89)

Panel B: Exclusions ranked by month −1 critical density				
	Critical	**95% interval**	**High minus low**	**N**
Exclude highest 1	−1.75**	[−3.47, −0.04]	−3.64*	487,563
	(0.86)		(1.91)	
Exclude highest 3	−1.61*	[−3.30, 0.08]	−3.35*	486,498
	(0.85)		(1.91)	
Exclude highest 5	−1.30*	[−2.81, 0.22]	−2.65	481,099
	(0.76)		(1.74)	
Exclude highest 7	−0.64	[−1.46, 0.17]	−1.11	480,114
	(0.41)		(0.87)	

All outcomes are issue counts per 1,000 lines of code. Panel A includes the Post interaction with high baseline critical-violation density and reports critical and total violation densities. The baseline cutoff uses the median across baseline firm–cohort observations of firms treated in any cohort. The own-financing cutoff uses one month −1 observation per treated firm. The average uses months −6 through −1; the restricted version requires observations in at least three of those months. Group-effect standard errors account for the covariance of the base and treatment interaction. Panel B ranks treated firms at their own financing and removes each listed firm's treated and control appearances. It reports the critical-violation DiD coefficient and the high-minus-low critical interaction using the baseline cutoff. Each resulting sample retains both treated and control firms in every cohort.

Note: Standard errors in parentheses, clustered by cohort and firm. * p<0.1, ** p<0.05, *** p<0.01.

Table 50: Baseline-Quality Groups: Support and Effect Intervals

Classification	Low, treated	High, treated	N	Low, 95% interval	High, 95% interval
Baseline cutoff	141	106	477,107	[−1.05, 0.50]	[−7.82, −0.47]
Own-financing baseline cutoff	124	123	477,107	[−1.12, 0.55]	[−6.79, −0.30]
Six-month pre-financing average	126	126	483,145	[−0.46, 0.23]	[−7.02, −0.44]
Average (at least three observed pre-financing months)	121	120	475,847	[−0.43, 0.26]	[−6.99, −0.27]

Intervals concern the critical-violation effects per 1,000 lines of code reported in Panel A of Table 49. High-group intervals use the covariance of the base coefficient and treatment interaction. Counts describe unique fitted treated firms in each group; all four comparisons retain 98 cohorts. Cohort-by-firm and cohort-by-calendar-month fixed effects, and clustering by cohort and firm, follow the corresponding interaction regression.

N.4 Matching Eligibility and Timing

Table 51 separates the requirement for observable growth features from the selection of comparable controls. Its second row retains precisely the treated firms entering the matched regression while leaving their eligible candidate controls unmatched. The common-population specification comparison in Appendix E separately fixes the treated population and cohort weights across all matching configurations.

Table 51: Matching Eligibility and Control Selection

Comparison	**Critical**	**Total**	**Debt**	**Maintainability**	**N**	**Treated**	**Controls**	**Cohorts**
Matching-eligible, unmatched	−2.52*	−4.26**	−1.14	−4.05**	177,278	112	206	70
	(1.43)	(1.86)	(0.82)	(1.75)				
Final treated firms, unmatched	−2.57*	−4.36**	−1.14	−4.15**	172,134	110	206	68
	(1.45)	(1.88)	(0.83)	(1.78)				
Preferred matched comparison	−2.32*	−4.80**	−1.29	−4.46**	71,837	110	196	68
	(1.22)	(2.37)	(0.98)	(2.11)				

Critical, total and maintainability outcomes are counts per 1,000 lines of code; debt is calibrated dollars per line. Eligibility requires an observation in month −1 and finite headcount-growth measures for treated and control firms. The final-treated unmatched row uses the 110 treated firms retained after the caliper and cohort support rule with all eligible controls. The preferred comparison selects controls within a Mahalanobis-distance caliper of one from the cohort's average treated headcount-growth profile. Counts are unique firms in the estimation sample by treatment or control role; roles can overlap across cohorts.

Note: Standard errors in parentheses, clustered by cohort and firm. * p<0.1, ** p<0.05, *** p<0.01.

The main quarterly event study places month −1 in the same bin as financing month 0; Table 52 estimates monthly coefficients that separate the two.

Table 52: Monthly Timing Relative to Month −4

Event month	**Critical**	**Total**	**Debt**	**Maintainability**
Month −3	0.07	−0.17	−0.01	−0.23
	(0.19)	(0.34)	(0.12)	(0.29)
Month −2	0.14	−0.27	−0.07	−0.34
	(0.22)	(0.45)	(0.16)	(0.41)
Month −1	0.33	0.13	−0.16	0.03
	(0.66)	(0.89)	(0.29)	(0.85)
Month 0	−0.24	−0.65	−0.35	−0.83
	(0.88)	(1.13)	(0.39)	(1.08)
Month 12	−2.60**	−4.75**	−1.87*	−4.67**
	(1.28)	(1.94)	(1.01)	(1.85)
All pre-financing months, joint p	0.93	0.81	0.83	0.74
Months −3, −2, −1, joint p	0.91	0.88	0.94	0.79

Critical, total and maintainability outcomes are counts per 1,000 lines of code; debt is calibrated dollars per line. Monthly treatment interactions omit month −4, preserving a separate coefficient for month −1. The regression uses the baseline 487,577 observations across 99 cohorts. Joint tests of zero pre-financing coefficients use the full two-way-clustered covariance matrix and a chi-square reference distribution, covering all eleven nonreference pre-financing months and, separately, months −3, −2, −1.

Note: Standard errors in parentheses, clustered by cohort and firm. * p<0.1, ** p<0.05, *** p<0.01.

N.5 Financing Amount and Low-Severity Maintainability Issues

Table 53 reports the low-severity maintainability estimates for smaller and larger first rounds from Table 4, Column (9). Low-severity maintainability issues decline by 1.0 per 1,000 lines in the smaller-round group ($p = 0.05$), while the larger-round estimate is close to zero and not statistically significant ($p = 0.83$). The groups differ by 1.1 issues per 1,000 lines ($p = 0.03$). Restricting to recorded financing amounts retains a positive difference of 1.1 ($p = 0.02$).

Table 53: First-Round Size and Low-Severity Maintainability Issues

Sample	Smaller round	Larger round	Difference	Treated small / large	N
All observations	−1.03*	0.08	1.11**	157 / 116	487,577
	[−2.09, 0.04]	[−0.70, 0.86]	[0.08, 2.13]		
Recorded amounts only	−0.92**	0.20	1.13**	138 / 116	132,023
	[−1.79, −0.06]	[−0.43, 0.84]	[0.13, 2.13]		

Outcomes are low-severity maintainability issue counts under the alternative classification per 1,000 lines of code. Large Round uses the treated-firm median of the first qualifying financing amount; missing amounts are coded as smaller rounds. The second row removes observations with a missing amount. The larger-round effect sums the base and interaction with their covariance. The two rows retain 99 and 95 cohorts, respectively. Brackets are 95% confidence intervals; standard errors are clustered by cohort and firm. * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$.

N.6 Outcome Families and Inference

Stars in the regression tables report two-sided pointwise tests using standard errors clustered by cohort and firm.

A conjunctive claim requires both critical-violation density and technical debt to decline. Its null is that at least one of these effects is nonnegative. An intersection–union test rejects only when each component test rejects in the negative direction. Using the retained two-sided tests, both negative baseline estimates meet the 5% threshold required for this conjunction, without assuming that the outcomes are independent.

Table 54 instead reports Holm adjustments for separate discoveries within the stated outcome families, controlling the probability of any false individual rejection. Technical debt per line and the Technical Debt Ratio express the same outcome in different units on the same observations and enter a family once. The two headline outcomes both have Holm-adjusted $p = 0.06$; adding total violations or maintainability leaves the adjusted values between 0.06 and 0.07.

Table 54: Pointwise and Holm-Adjusted Baseline Tests

Outcome	Pointwise p	Two outcomes	Three outcomes	Four outcomes
Critical violations	0.03	0.06	0.06	0.07
Technical debt per line	0.04	0.06	0.06	0.07
Total violations	0.02	—	0.06	0.07
Maintainability violations	0.01	—	—	0.07

The two-outcome family contains critical violations and technical debt per line. The three-outcome family adds total violations; the four-outcome family also adds the maintainability count. The Technical Debt Ratio is a fixed rescaling of debt per line on the same observations and is counted once. Holm adjustments control familywise rejection using the retained two-sided pointwise tests without assuming independent outcomes. All tests use the same 487,577 observations, 483 firms and 99 cohorts and the firm-and-cohort clustered covariance.

N.7 Baseline Selection Regression

We examine whether pre-financing engineering quality and codebase size predict which firms receive financing within a cohort. The dependent variable equals one for that cohort's treated firms and zero for its controls. The regressions use 19,933 firm–cohort observations at month -1, representing 462 firms and 98 cohorts. Both specifications absorb cohort fixed effects and cluster standard errors by firm. The quality-only specification includes critical-violation density, total-violation density and debt per line; the second adds $\log(\text{LoC})$. The Technical Debt Ratio is collinear with debt per line and adds no independent regressor. Joint tests use $F(k, 461)$, with k equal to the number of retained tested coefficients and 461 equal to the number of firm clusters minus one. The tests give $F(3, 461) = 1.38$ ($p = 0.24$; not statistically significant) and $F(4, 461) = 3.86$ ($p < 0.01$). The corresponding within-cohort explained variance is 0.02% and 0.1%.

N.8 Denominators for Relative Effect Sizes

We express each quality coefficient as a percentage of that outcome's own treated pre-financing mean in the estimation sample: $100\widehat{\beta}/\bar{Y}_{T,\text{pre}}$. The mean averages the available treated observations in months -12 through -1, giving each observation equal weight. Table 55 reports the denominators separately for the baseline and preferred matching comparisons, including both maintainability classifications.

Table 55: Pre-Financing Means Used for Relative Effect Sizes

Outcome	Pre-financing mean	Change (%)	Treated pre-financing observations	Regression observations
Baseline				
Critical violations	12.40	-14.9	2,719	487,577
Total violations	32.89	-14.4	2,719	487,577
Maintainability (traditional classification)	30.83	-15.6	2,719	487,577
Maintainability (alternative classification)	30.70	-15.5	2,719	487,577
Debt, dollars per line	13.09	-12.0	2,719	487,577
Preferred matching				
Critical violations	16.09	-14.4	1,315	71,837
Total violations	36.62	-13.1	1,315	71,837
Maintainability (traditional classification)	34.54	-12.9	1,315	71,837
Maintainability (alternative classification)	34.09	-12.9	1,315	71,837
Debt, dollars per line	14.81	-8.7	1,315	71,837

Critical, total and traditional maintainability means are violations per 1,000 lines of code; alternative maintainability means are issues per 1,000 lines. The debt mean is calibrated dollars per line. Each reported treated pre-financing count is the number of observations used to calculate that mean. Baseline comparisons have 273 treated firms in 99 cohorts; preferred matching has 110 treated firms in 68 cohorts. The Technical Debt Ratio is a fixed rescaling of debt per line on these observations and has the same relative change.

N.9 Calendar-Month Dependence

The baseline estimation sample covers 178 calendar months, 99 financing cohorts and 483 firms. Table 56 keeps the coefficients and observations fixed and adds calendar-month clustering to the firm and cohort dimensions. Critical violations retain $p = 0.03$ and technical debt retains $p = 0.04$; their confidence intervals are slightly narrower than under baseline clustering.

Table 56: Calendar-Month Clustering on Identical Baseline Observations

Clustering	Critical violations		Technical debt	
	Estimate (standard error)	95% interval	Estimate (standard error)	95% interval
Firm and cohort	−1.85 (0.86)	[−3.56, −0.14]	−1.57 (0.77)	[−3.08, −0.05]
Firm and calendar month	−1.85 (0.78)	[−3.39, −0.31]	−1.57 (0.73)	[−3.01, −0.12]
Firm, cohort and calendar month	−1.85 (0.85)	[−3.53, −0.17]	−1.57 (0.76)	[−3.07, −0.06]

All specifications use the same 487,577 observations and baseline fixed effects. Critical violations are counts per 1,000 lines of code; technical debt is calibrated dollars per line. Calendar months use the observation's calendar year and month. Covariance combines score cross-products for every clustering dimension and intersection by inclusion–exclusion, preserving repeated observations of each firm. The small-sample cluster adjustment and Student-t reference follow the baseline convention: degrees of freedom are 98 for rows including cohort and 177 for firm/calendar clustering.